AI-Based Intrusion Detection Systems for Next-Generation Networks

PhD Research Proposal

idriss5214

December 2025

Abstract

This comprehensive PhD research proposal presents a novel framework for developing AI-based intrusion detection systems specifically designed for next-generation networks including 5G/6G, IoT, and Software-Defined Networks (SDN). The proposed system leverages hybrid deep learning models (CNN-LSTM-Transformer), federated learning for distributed deployment, and explainable AI (XAI) for transparency. The research aims to achieve >98% detection accuracy with <100ms latency while maintaining privacy through differential privacy mechanisms. This document includes the complete proposal, methodology, timeline, budget, and collaboration strategy for a 4-year PhD program.

Contents

Chapter 1

AI-Based Intrusion Detection Systems for Next-Generation Networks

1.1 Complete PhD Research Proposal

Candidate: idriss5214
Date: December 22, 2025
Duration: 4 Years
Field: Computer Science - Cybersecurity & Network Security

1.2 1. Executive Summary

The rapid evolution of next-generation networks (NGN), including 5G/6G, Internet of Things (IoT), Software-Defined Networks (SDN), and edge computing, has introduced unprecedented security challenges. Traditional intrusion detection systems (IDS) are inadequate for detecting sophisticated cyber threats in these complex, dynamic environments. This PhD research proposes to develop an advanced, AI-based intrusion detection framework that leverages hybrid deep learning models, adaptive learning mechanisms, and explainable AI to provide real-time, accurate, and transparent threat detection across diverse network architectures.

The proposed research will integrate Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM) networks, Transformers, and ensemble methods to create a robust detection engine capable of identifying both known and zero-day attacks. The system will incorporate reinforcement learning for adaptive threat response, federated learning for privacy-preserving distributed detection, and explainable AI (XAI) techniques to ensure transparency and trust in automated security decisions.

Key Innovations: - Hybrid CNN-LSTM-Transformer architecture for multi-scale feature extraction - Adaptive learning framework using reinforcement learning (RL) and

online learning - Distributed IDS architecture for edge computing with federated learning - Explainable AI integration for transparent decision-making - Real-time detection with <100ms latency for critical network scenarios - Privacy-preserving detection through federated and differential privacy mechanisms

Expected Impact: - Improved detection accuracy (>98%) with reduced false positive rates (<1%) - Scalable solution for heterogeneous NGN environments - Enhanced security for critical infrastructure and IoT ecosystems - Practical deployment frameworks for industry adoption - 5-10 high-impact publications in top security venues

---

1.3 2. Introduction

1.3.1 2.1 Background

Next-generation networks represent a paradigm shift in telecommunications and computing infrastructure. 5G/6G networks promise ultra-low latency (<1ms), massive connectivity (1M+ devices/km²), and multi-gigabit speeds. Software-Defined Networks (SDN) and Network Function Virtualization (NFV) enable flexible, programmable network management. Edge computing brings computation closer to data sources, reducing latency and bandwidth consumption. The Internet of Things is connecting billions of resource-constrained devices across diverse applications from smart homes to industrial control systems.

However, these advancements introduce significant security vulnerabilities:

1. **Expanded Attack Surface:** Billions of IoT devices, many with weak security, create numerous entry points
2. **Network Complexity:** SDN/NFV introduce new attack vectors in control plane and virtualization layers
3. **Heterogeneity:** Diverse device types, protocols, and traffic patterns complicate detection
4. **High-Speed Traffic:** Gigabit speeds require real-time detection with minimal latency
5. **Evolving Threats:** Advanced persistent threats (APTs) and zero-day exploits evade signature-based detection
6. **Resource Constraints:** Edge and IoT devices have limited computational resources for security
7. **Privacy Concerns:** Centralized IDS raise data privacy and sovereignty issues

1.3.2 2.2 Problem Statement

Current intrusion detection systems face critical limitations in next-generation network environments:

Traditional IDS Limitations: - Signature-based systems cannot detect zero-day attacks - Anomaly-based systems generate high false positive rates - Rule-based systems lack adaptability to evolving threats - Centralized architectures create single points of failure - Black-box ML models lack explainability for security analysts - Static models

fail to adapt to concept drift in network behavior - Privacy violations through centralized data collection

Research Challenges: 1. How to achieve high detection accuracy (>98%) while maintaining low false positive rates (<1%)? 2. How to detect zero-day attacks and advanced persistent threats without prior signatures? 3. How to process high-speed network traffic in real-time with <100ms detection latency? 4. How to create scalable IDS for distributed edge computing environments? 5. How to ensure model explainability and trust in automated security decisions? 6. How to preserve data privacy while enabling collaborative threat intelligence? 7. How to adapt detection models continuously to evolving threat landscapes?

1.3.3 2.3 Research Significance

This research addresses critical gaps at the intersection of artificial intelligence, cybersecurity, and next-generation networks. The proposed AI-based IDS will:

- **Academic Contribution:** Advance the state-of-art in AI-driven network security through novel hybrid architectures, adaptive learning frameworks, and distributed detection mechanisms
- **Practical Impact:** Provide deployable solutions for securing critical infrastructure, enterprise networks, IoT ecosystems, and telecommunications providers
- **Societal Benefit:** Enhance security and privacy for billions of users in an increasingly connected world
- **Economic Value:** Reduce cybercrime costs (projected $10.5T annually by 2025) through improved threat prevention

---

1.4 3. Research Objectives

1.4.1 3.1 Primary Objective

To design, develop, and validate an advanced AI-based intrusion detection system for next-generation networks that achieves superior detection accuracy, real-time performance, scalability, adaptability, and explainability compared to existing solutions.

1.4.2 3.2 Specific Objectives

1. **Hybrid AI Model Development**
    - Design CNN-LSTM hybrid for spatial-temporal feature extraction
    - Implement Transformer networks for capturing long-range dependencies
    - Develop autoencoder architectures for unsupervised anomaly detection
    - Create ensemble methods combining multiple AI models
2. **Adaptive Learning Framework**
    - Implement reinforcement learning (DQN, A3C, PPO) for dynamic threat response
    - Design online learning mechanisms for continuous model updates
    - Develop transfer learning strategies for cross-domain attack detection

- Create meta-learning approaches for rapid adaptation to new threats

3. **Distributed IDS Architecture**
   - Design hierarchical edge-fog-cloud detection framework
   - Implement federated learning for privacy-preserving collaborative detection
   - Develop efficient model compression for resource-constrained devices
   - Create secure aggregation protocols for distributed learning
4. **Explainable AI Integration**
   - Integrate SHAP, LIME, and attention mechanisms for model interpretability
   - Develop visualization tools for security analysts
   - Create explainability metrics for IDS evaluation
   - Design human-in-the-loop feedback mechanisms
5. **Real-Time Optimization**
   - Optimize detection latency to <100ms for critical scenarios
   - Implement efficient feature extraction and model inference
   - Design hardware acceleration strategies (GPU, FPGA)
   - Develop traffic prioritization and sampling techniques
6. **Comprehensive Validation**
   - Evaluate on multiple benchmark datasets (NSL-KDD, CICIDS2017, UNSW-NB15, IoT-23)
   - Deploy testbed with real 5G/IoT/SDN infrastructure
   - Conduct extensive experiments across diverse attack scenarios
   - Compare against state-of-the-art IDS solutions
7. **Privacy and Security**
   - Implement differential privacy mechanisms
   - Design secure multi-party computation protocols
   - Address adversarial machine learning threats
   - Ensure GDPR and data protection compliance

1.5 4. Literature Review

1.5.1 4.1 Traditional Intrusion Detection Systems

Signature-Based IDS: Traditional systems like Snort and Suricata rely on pattern matching against known attack signatures. While effective for known threats with low false positives, they fail to detect zero-day attacks and polymorphic malware [1]. The signature database requires constant updates, creating maintenance overhead and detection gaps.

Anomaly-Based IDS: Statistical and machine learning approaches establish baseline normal behavior and flag deviations as anomalies [2]. Classical ML techniques (SVM, Random Forest, k-NN) achieve moderate success but suffer from high false positive rates (10-20%) and inability to adapt to concept drift [3].

Hybrid Approaches: Systems combining signature and anomaly detection (e.g., OSSEC, Bro/Zeek) provide better coverage but inherit limitations from both approaches [4]. Computational overhead increases significantly with hybrid implementations.

1.5.2 4.2 AI/ML Techniques in Cybersecurity

Classical Machine Learning: Random Forest, SVM, and ensemble methods have been extensively studied for intrusion detection [5, 6]. While achieving 90-95% accuracy on benchmark datasets, these methods struggle with high-dimensional feature spaces and require extensive feature engineering.

Deep Learning: CNN architectures extract spatial features from network traffic representations [7]. LSTM networks model temporal sequences for behavior analysis [8]. Autoencoder-based anomaly detection identifies outliers in unsupervised settings [9]. Deep learning achieves 95-98% accuracy but lacks interpretability.

Ensemble Methods: Combining multiple models improves robustness and accuracy [10]. Stacking, bagging, and boosting techniques have shown 2-5% accuracy improvements over individual models.

Reinforcement Learning: RL agents learn optimal defense strategies through interaction with network environments [11]. Q-learning and Deep Q-Networks enable adaptive response to evolving threats but require extensive training.

1.5.3 4.3 Next-Generation Network Security

5G/6G Security: Network slicing, edge computing, and massive IoT connectivity introduce new vulnerabilities [12]. ML-based anomaly detection for 5G core networks shows promise but faces scalability challenges [13].

IoT Security: Resource-constrained IoT devices require lightweight detection mechanisms [14]. Federated learning enables collaborative threat detection without centralizing sensitive data [15].

SDN/NFV Security: Software-defined architectures face controller compromise and flow rule manipulation attacks [16]. ML-based SDN security solutions achieve real-time threat detection at controller level [17].

Edge Computing: Distributed detection at edge nodes reduces latency and bandwidth but requires model compression and efficient aggregation [18].

1.5.4 4.4 Recent Advances (2019-2025)

Transformer Networks: Attention mechanisms capture long-range dependencies in network traffic sequences [19]. BERT-like architectures for IDS achieve state-of-the-art results on benchmark datasets [20].

Federated Learning: Privacy-preserving collaborative learning enables distributed IDS without data centralization [21]. Secure aggregation protocols protect against adversarial participants [22].

Explainable AI: SHAP and LIME integration provides feature-level explanations for ML predictions [23]. Attention visualization in deep models reveals decision rationale [24].

Graph Neural Networks: GNNs model network topology and traffic flows as graphs, capturing structural patterns [25]. Application to SDN security shows promising results [26].

Adversarial Machine Learning: Adversarial training improves robustness against evasion attacks [27]. Defensive distillation and certified defenses enhance IDS resilience [28].

1.5.5 4.5 Research Gaps

1. **Limited Hybrid Architectures:** Few studies combine CNN, LSTM, and Transformers effectively
2. **Inadequate Real-Time Performance:** Existing deep learning IDS struggle with <100ms latency requirements
3. **Scalability Issues:** Most solutions tested on small-scale networks, not production-level deployments
4. **Explainability Gap:** Black-box models lack transparency needed for security operations
5. **Privacy Concerns:** Centralized architectures violate data sovereignty requirements
6. **Adaptation Limitations:** Static models fail to handle concept drift and evolving threats
7. **Cross-Domain Transfer:** Limited research on transferring IDS models across different network types
8. **Benchmark Limitations:** Existing datasets don't adequately represent modern 5G/IoT attack scenarios

---

1.6 5. Research Methodology

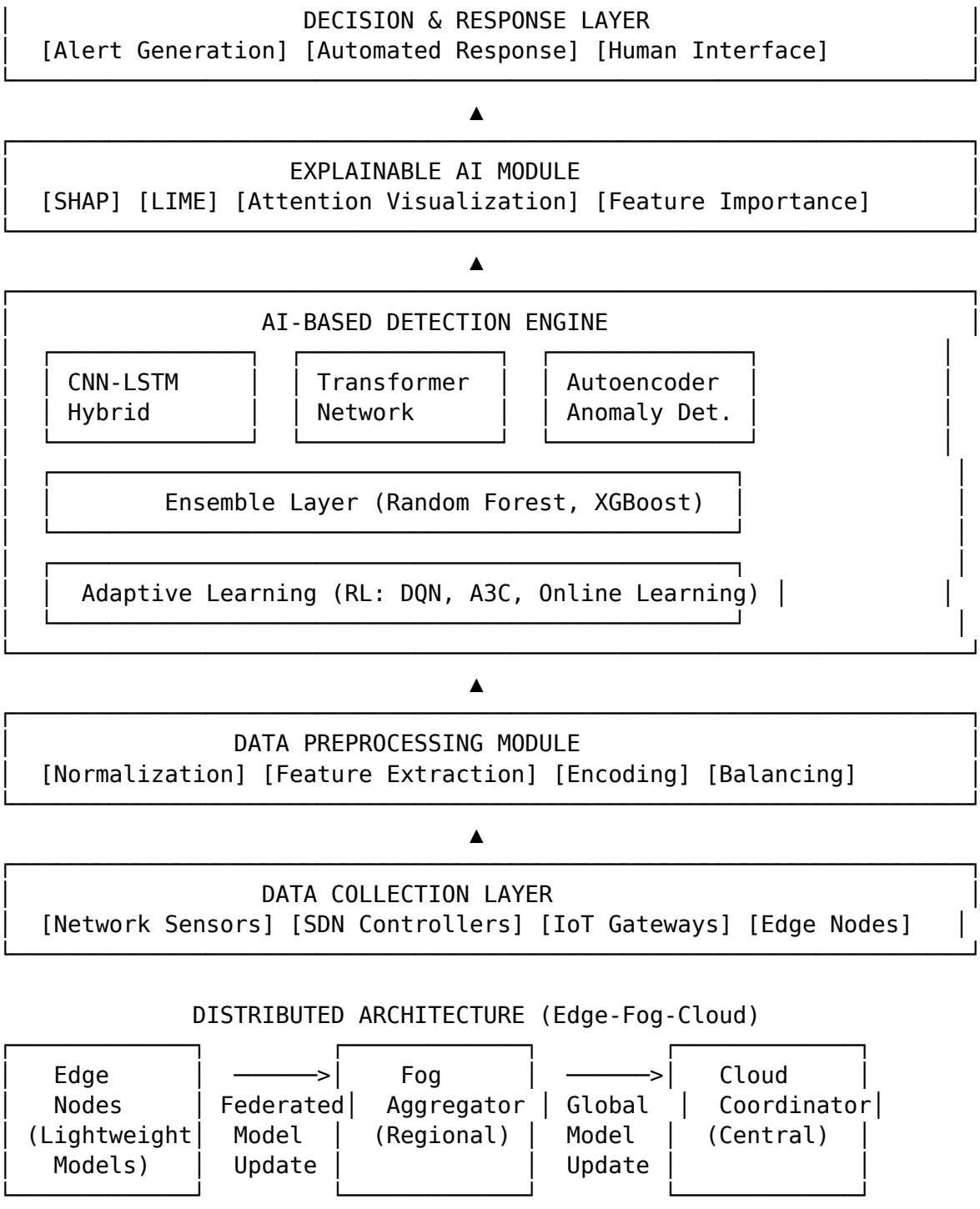
1.6.1 5.1 Research Design

This research follows a design science approach with iterative development, evaluation, and refinement cycles. The methodology combines theoretical modeling, algorithm design, system implementation, and empirical validation.

Phases: 1. **Phase 1 (Months 1-6):** Literature review, requirements analysis, initial design 2. **Phase 2 (Months 7-12):** Algorithm development, prototype implementation 3. **Phase 3 (Months 13-24):** Framework integration, testbed deployment, initial experiments 4. **Phase 4 (Months 25-36):** Extensive validation, optimization, publications 5. **Phase 5 (Months 37-48):** Final experiments, thesis writing, defense preparation

1.6.2 5.2 Framework Architecture

The proposed AI-IDS framework consists of seven interconnected layers:



1.6.3 5.3 Key Components

1.6.3.1 5.3.1 Data Collection Layer

- **Network Sensors:** Deep packet inspection, flow monitoring, protocol analysis
- **SDN Controllers:** OpenFlow statistics, topology information, control plane events

- **IoT Gateways:** Device behavior monitoring, protocol translation, edge preprocessing
- **Log Aggregation:** System logs, application logs, security events

1.6.3.2 5.3.2 Data Preprocessing Module

- **Traffic Parsing:** Extract relevant features from raw packets/flows
- **Feature Engineering:** Statistical features, temporal features, behavioral features
- **Normalization:** Min-max scaling, Z-score normalization
- **Encoding:** One-hot encoding for categorical features, embedding for high-cardinality
- **Balancing:** SMOTE, ADASYN for addressing class imbalance
- **Dimensionality Reduction:** PCA, autoencoders for feature compression

1.6.3.3 5.3.3 AI-Based Detection Engine

CNN-LSTM Hybrid: - CNN layers extract spatial features from traffic matrices - LSTM layers model temporal dependencies in traffic sequences - Attention mechanism focuses on relevant time steps - Architecture: Conv1D(64) → Conv1D(128) → LSTM(128) → LSTM(64) → Dense(num_classes)

Transformer Network: - Multi-head self-attention captures global dependencies - Position encoding preserves temporal information - Feed-forward networks for feature transformation - Architecture: Embedding → Positional Encoding → 6× Transformer Blocks → Classification Head

Autoencoder for Anomaly Detection: - Encoder compresses normal traffic into latent representation - Decoder reconstructs input from latent space - Reconstruction error threshold for anomaly detection - Variational autoencoder for probabilistic modeling

Ensemble Layer: - Random Forest (100 trees) for robustness - XGBoost for gradient boosting - Voting or stacking for final prediction - Confidence calibration for uncertainty quantification

Adaptive Learning: - Deep Q-Network (DQN) for optimal response selection - Actor-Critic (A3C) for policy-based learning - Proximal Policy Optimization (PPO) for stable training - Online learning for continuous model updates with streaming data

1.6.3.4 5.3.4 Explainable AI Module

- **SHAP:** Shapley values for global and local feature importance
- **LIME:** Local surrogate models for instance-level explanations
- **Attention Visualization:** Heatmaps showing attended traffic segments
- **Counterfactual Explanations:** What-if analysis for decision understanding
- **Rule Extraction:** Distilling neural networks into interpretable rules

1.6.3.5 5.3.5 Distributed Architecture

- **Edge Nodes:** Lightweight models (MobileNet-like) for local detection
- **Fog Layer:** Regional aggregation and model coordination
- **Cloud Layer:** Global model training and distribution
- **Federated Learning:** Privacy-preserving collaborative learning
- **Model Compression:** Quantization, pruning, knowledge distillation

1.6.4 5.4 Datasets

Dataset	Description	Samples	Features	Attack Types
NSL-KDD	Refined version of KDD'99, balanced classes	125,973 train22,544 test	41	DoS, Probe, R2L, U2R
CICIDS2017	Realistic modern attacks, labeled flows	2.8M flows	80	DoS, DDoS, PortScan, Brute Force, Web attacks, Botnet
UNSW-NB15	Hybrid of real and synthetic attacks	2.54M records	49	Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms
IoT-23	IoT-specific botnet traffic	325M packets	Variable	Mirai, Torii, IoT-specific malware
5G Dataset	Custom collection from testbed	TBD	TBD	5G-specific attacks, network slicing attacks

Data Preprocessing Pipeline: 1. Traffic capture using tcpdump/Wireshark 2. Flow generation using CICFlowMeter 3. Feature extraction (statistical, temporal, protocol-based) 4. Cleaning (missing values, outliers) 5. Normalization and encoding 6. Train/validation/test split (70/15/15) 7. Class balancing using SMOTE for minority classes

1.6.5 5.5 Experimental Setup

Hardware: - GPU Cluster: 4× NVIDIA A100 (40GB) for deep learning training - CPU Servers: 2× Intel Xeon Gold 6248R (48 cores) for preprocessing - Edge Devices: 10× Raspberry Pi 4 (8GB) for edge testing - SDN Testbed: 5× OpenFlow switches, ONOS controller - IoT Devices: 50× mixed IoT sensors and actuators

Software: - Deep Learning: TensorFlow 2.x, PyTorch 2.x, Keras - SDN: ONOS, OpenDaylight, Ryu - Network Simulation: NS-3, Mininet, OMNET++ - XAI Tools: SHAP, LIME, Captum - Federated Learning: TensorFlow Federated, PySyft - Data Processing: Pandas, NumPy, Scikit-learn - Monitoring: Grafana, Prometheus, ELK Stack

Baselines for Comparison: 1. Traditional ML: Random Forest, SVM, Decision Trees 2. Deep Learning: CNN, LSTM, GRU, standalone models 3. Existing IDS: Snort, Suricata, Zeek 4. Recent Research: Latest published models from top conferences

1.6.6 5.6 Evaluation Metrics

Detection Performance:

Metric	Formula	Target
Accuracy	(TP + TN) / (TP + TN + FP + FN)	>98%
Precision	TP / (TP + FP)	>97%
Recall/TPR	TP / (TP + FN)	>98%
F1-Score	2 × (Precision × Recall) / (Precision + Recall)	>97%
False Positive Rate	FP / (FP + TN)	<1%
AUC-ROC	Area under ROC curve	>0.99

Efficiency Metrics: - Detection Latency: Average time from packet arrival to detection decision (<100ms) - Throughput: Number of packets processed per second (>10K pps) - CPU Usage: Average CPU utilization during detection (<50%) - Memory Footprint: RAM consumption (<2GB for edge devices) - Network Overhead: Additional bandwidth for IDS communication (<5%)

Scalability Metrics: - Performance vs. Network Size (100 to 100K devices) - Distributed efficiency (speedup, communication cost) - Model update latency in federated setting

Adaptability Metrics: - Zero-day detection rate: % of unknown attacks detected - Adaptation time: Time to adapt to new attack patterns - Concept drift handling: Performance degradation over time

Explainability Metrics: - Feature importance consistency - Explanation fidelity (faithfulness to model) - Human evaluation scores (clarity, usefulness)

1.7 6. Expected Contributions

1.7.1 6.1 Theoretical Contributions

1. **Novel Hybrid Architecture:** CNN-LSTM-Transformer fusion with attention mechanisms for multi-scale feature extraction in network traffic analysis
2. **Adaptive Learning Framework:** Reinforcement learning-based dynamic threat response with online learning for continuous adaptation
3. **Distributed Detection Theory:** Mathematical frameworks for federated IDS optimization with privacy guarantees
4. **Explainability Models:** New XAI techniques specifically designed for network intrusion detection

5. **Transfer Learning Strategies:** Cross-domain knowledge transfer methods for heterogeneous network environments

1.7.2 6.2 Practical Contributions

1. **Production-Ready Framework:** Open-source AI-IDS platform deployable in real networks
   - GitHub repository with comprehensive documentation
   - Docker containers for easy deployment
   - APIs for integration with existing security infrastructure
2. **Benchmark Datasets:** Curated and preprocessed datasets for reproducible research
   - Modern 5G/IoT attack scenarios
   - Balanced and labeled for training/evaluation
3. **Industry Guidelines:** Best practices for deploying AI-based IDS in production
   - Performance tuning recommendations
   - Security and privacy considerations
   - Maintenance and update procedures
4. **Tool Ecosystem:** Suite of tools for IDS development and evaluation
   - Automated feature engineering pipeline
   - Model training and evaluation framework
   - Explainability dashboard for analysts
5. **Testbed Infrastructure:** Documented setup for 5G/IoT/SDN security testbed
   - Replicable for other research institutions
   - Virtual testbed configurations for cloud deployment

1.8 7. Challenges and Risk Mitigation

Challenge	Risk Level	Mitigation Strategy
Data Availability	High	Use multiple public datasets; establish industry partnerships for real traffic; generate synthetic data using GANs
Computational Resources	Medium	Apply for AWS/Google Cloud research credits; use model compression; leverage university GPU clusters
Model Overfitting	Medium	Cross-validation, regularization (L2, dropout), early stopping; test on multiple datasets
Real-Time Performance	High	Model optimization (pruning, quantization); hardware acceleration; efficient preprocessing

Challenge	Risk Level	Mitigation Strategy
Adversarial Attacks	Medium	Adversarial training; defensive distillation; input validation; ensemble diversity
Concept Drift	High	Online learning implementation; periodic retraining; drift detection mechanisms
Explainability Trade-off	Medium	Balance between accuracy and interpretability; multiple XAI techniques; user studies
Privacy Concerns	High	Differential privacy; federated learning; secure aggregation; compliance with regulations
Testbed Limitations	Medium	Hybrid approach (simulation + physical); cloud-based virtual testbed; phased deployment
Industry Adoption	Low	Focus on practical usability; documentation; workshops; open-source release

---

1.9 8. Timeline

1.9.1 Year 1: Foundation and Design (Months 1-12)

Q1 (Months 1-3): Literature Review & Coursework - Comprehensive literature review (100+ papers) - PhD coursework (Advanced ML, Network Security) - Form dissertation committee - Preliminary experiments with baseline models - **Deliverable:** Literature review draft, committee approval

Q2 (Months 4-6): Algorithm Design & Initial Prototyping - Design CNN-LSTM hybrid architecture - Implement baseline models (RF, SVM, standalone CNN/LSTM) - Collect and preprocess benchmark datasets - Set up development environment - **Deliverable:** Algorithm specifications, initial results

Q3 (Months 7-9): Transformer & Ensemble Implementation - Implement Transformer-based IDS - Develop autoencoder for anomaly detection - Create ensemble methods - Preliminary evaluation on NSL-KDD and CICIDS2017 - **Deliverable:** Prototype models, technical report

Q4 (Months 10-12): Comprehensive Exam & First Publication - Prepare for comprehensive exam - Write first conference paper - Integrate XAI components (SHAP, LIME) - Initial comparison with baselines - **Deliverable:** Comprehensive exam passed, paper submitted, annual report

1.9.2 Year 2: Framework Development (Months 13-24)

Q1 (Months 13-15): Adaptive Learning Integration - Implement reinforcement learning (DQN, A3C) - Develop online learning mechanisms - Transfer learning experiments - Extended evaluation on UNSW-NB15 - **Deliverable:** Adaptive learning module, workshop paper

Q2 (Months 16-18): Distributed Architecture Design - Design federated learning framework - Implement model compression techniques - Develop edge deployment strategy - Privacy mechanism integration - **Deliverable:** Distributed IDS prototype

Q3 (Months 19-21): Testbed Deployment Preparation - Acquire hardware (edge devices, SDN switches) - Set up SDN testbed with ONOS - Deploy IoT devices - Configure monitoring infrastructure - **Deliverable:** Operational testbed

Q4 (Months 22-24): Initial Testbed Experiments - Deploy IDS on testbed - Conduct real-time detection experiments - Performance profiling and optimization - Second major publication - **Deliverable:** Testbed results, journal paper submitted, annual report

1.9.3 Year 3: Validation and Optimization (Months 25-36)

Q1 (Months 25-27): Extensive Experiments - Large-scale experiments on all datasets - Cross-dataset evaluation - Ablation studies on model components - Adversarial robustness testing - **Deliverable:** Comprehensive experimental results

Q2 (Months 28-30): 5G/IoT Specific Validation - Custom 5G dataset collection - IoT-23 malware detection experiments - Edge computing performance analysis - Real-world case studies - **Deliverable:** NGN-specific results, conference paper

Q3 (Months 31-33): Performance Optimization - Model compression and acceleration - Latency optimization (<100ms target) - Scalability experiments (1K to 100K devices) - Energy efficiency analysis - **Deliverable:** Optimized framework

Q4 (Months 34-36): Publication Push & Framework Release - Submit major journal paper (IEEE TIFS/TMC) - Write conference papers for top venues - Prepare open-source release - Documentation and tutorials - **Deliverable:** Framework v1.0 released, multiple papers submitted, annual report

1.9.4 Year 4: Thesis Completion (Months 37-48)

Q1 (Months 37-39): Additional Experiments - Address reviewer feedback from publications - Conduct additional experiments as needed - User studies for explainability evaluation - Industry validation with partners - **Deliverable:** Revised manuscripts, additional results

Q2 (Months 40-42): Thesis Writing - Write dissertation chapters - Integrate all results and publications - Create comprehensive documentation - Prepare figures and tables - **Deliverable:** Thesis draft

Q3 (Months 43-45): Thesis Review & Defense Preparation - Committee review of dissertation - Revisions based on feedback - Prepare defense presentation - Practice

defense - **Deliverable:** Final thesis draft, defense slides

Q4 (Months 46-48): Defense & Final Submission - Dissertation defense - Final revisions - Submit final dissertation - Publish remaining papers - **Deliverable:** PhD degree, published papers, graduated!

Key Milestones: - Month 3: Committee formed - Month 12: Comprehensive exam passed - Month 18: First major publication accepted - Month 24: Testbed operational, second publication submitted - Month 30: 3+ publications accepted/published - Month 36: Framework released, journal paper submitted - Month 42: Thesis draft complete - Month 48: Defense passed, PhD awarded

---

1.10 9. Resources Required

1.10.1 9.1 Computational Resources

Cloud Computing: - AWS/Google Cloud/Azure research credits: $10,000-15,000 - GPU instances (p3.8xlarge equivalent): 10,000 hours - Storage: 5TB for datasets and models

University Resources: - Access to HPC cluster with GPU nodes - Dedicated development server - Backup and archival storage

1.10.2 9.2 Software and Tools

Commercial Licenses: - MATLAB (if needed): $500/year - Professional network analysis tools: $1,000-2,000

Open Source (Free): - TensorFlow, PyTorch, Keras - Scikit-learn, Pandas, NumPy - ONOS, OpenDaylight, Mininet - SHAP, LIME, Captum

1.10.3 9.3 Network Testbed

Hardware: - 5× OpenFlow-enabled switches: $5,000 - 10× Raspberry Pi 4 (8GB): $1,000 - 50× IoT devices (various sensors): $2,500 - 2× High-performance servers: $8,000 - Networking equipment (cables, routers): $1,500 - **Subtotal:** $18,000

1.10.4 9.4 Conference Travel

Target Conferences (4 years): - IEEE S&P, USENIX Security, CCS, NDSS (security top-tier) - IEEE INFOCOM, MobiCom (networking) - NeurIPS, ICML (ML venues for security) - Estimated: 4 major conferences × $2,500/conference = $10,000

1.10.5 9.5 Publications

- Open access fees: $2,000-4,000 (2-3 papers)
- Page charges: $500-1,000

1.10.6 9.6 Miscellaneous

- Books and resources: $500
- Collaboration travel: $1,500
- Contingency: $2,000

1.10.7 9.7 Total Budget Estimate

Category	Amount
Computational Resources	$15,000
Network Testbed Hardware	$18,000
Software Licenses	$2,500
Conference Travel	$10,000
Publications	$3,500
Miscellaneous	$4,000
Total	**$53,000**

Funding Sources: - University research assistantship - Department research grants - External fellowships (NSF, industry) - Cloud provider research credits (AWS, Google) - Industry partnerships (equipment donations)

1.11 10. Ethical Considerations

1.11.1 10.1 Data Privacy

- **Personal Data Protection:** Ensure all network traffic data is anonymized; remove personally identifiable information (PII)
- **GDPR Compliance:** Follow EU data protection regulations for data collection and storage
- **Consent:** Obtain informed consent for data collection in testbed experiments
- **Data Retention:** Implement data retention policies; delete data after research completion

1.11.2 10.2 Responsible Disclosure

- **Vulnerability Reporting:** Discovered vulnerabilities will be responsibly disclosed to affected vendors
- **Coordinated Disclosure:** Follow industry standards (90-day disclosure timeline)
- **No Exploit Development:** Research focuses on defense; no offensive exploit creation

1.11.3 10.3 Dual-Use Concerns

- **Defensive Purpose:** AI-IDS is designed purely for defensive cybersecurity

- **Misuse Prevention:** Document safeguards against malicious use
- **Access Control:** Restrict access to attack generation components (GANs for synthetic attacks)

1.11.4 10.4 Bias and Fairness

- **Dataset Bias:** Acknowledge and mitigate biases in training datasets
- **Fairness Evaluation:** Ensure IDS doesn't discriminate against specific users or traffic types
- **Diverse Testing:** Validate across diverse network environments and use cases

1.11.5 10.5 Environmental Impact

- **Energy Efficiency:** Optimize models to reduce computational energy consumption
- **Green Computing:** Use energy-efficient hardware and cloud providers with renewable energy
- **Carbon Footprint:** Track and report carbon footprint of large-scale experiments

1.11.6 10.6 Institutional Compliance

- **IRB Approval:** Obtain Institutional Review Board approval for human subjects research (user studies)
- **University Policies:** Comply with university research ethics policies
- **Export Control:** Ensure compliance with export control regulations for security research

---

1.12 11. Budget Estimate

1.12.1 Detailed Budget Breakdown

1.12.1.1 11.1 Personnel (if applicable)

- Research Assistant support: $0 (covered by assistantship)
- Undergraduate research assistance: $3,000 (summer work)

1.12.1.2 11.2 Equipment & Hardware ($18,000)

Item	Quantity	Unit Cost	Total
OpenFlow SDN switches	5	$1,000	$5,000
High-performance servers	2	$4,000	$8,000
Raspberry Pi 4 (8GB)	10	$100	$1,000
IoT devices (mixed)	50	$50	$2,500
Network cables & accessories	-	-	$500

Item	Quantity	Unit Cost	Total
Storage drives (backup)	2	$500	$1,000

1.12.1.3 11.3 Computational Resources ($15,000)

Service	Description	Cost
AWS EC2 GPU instances	p3.8xlarge, 10,000 hours	$12,000
Cloud storage	5TB S3/equivalent	$1,500
Data transfer	Egress charges	$1,500

1.12.1.4 11.4 Software & Licenses ($2,500)

Software	Cost
Network analysis tools	$1,500
Professional IDE licenses	$500
Visualization software	$500

1.12.1.5 11.5 Travel - Conferences ($10,000)

Year	Conferences	Estimated Cost
Year 1	1 domestic conference	$1,500
Year 2	1 international conference	$3,000
Year 3	2 conferences (1 int’l, 1 domestic)	$3,500
Year 4	1 conference (defense year)	$2,000

1.12.1.6 11.6 Publications ($3,500)

- Open access fees (2-3 papers): $3,000
- Page charges: $500

1.12.1.7 11.7 Miscellaneous ($4,000)

- Books and resources: $500
- Collaboration travel: $1,500
- Printing and materials: $300
- Contingency fund: $1,700

1.12.1.8 11.8 Summary

Category	Amount	Percentage
Equipment & Hardware	$18,000	34%
Computational Resources	$15,000	28%
Conference Travel	$10,000	19%
Publications	$3,500	7%
Software & Licenses	$2,500	5%
Personnel (undergrad support)	$3,000	6%
Miscellaneous	$1,000	2%
Total	**$53,000**	**100%**

Funding Strategy: 1. **University Funding:** Research assistantship covers tuition and stipend 2. **Department Grants:** Apply for internal research grants ($10K-15K) 3. **External Fellowships:** NSF GRFP, industry fellowships ($30K-40K) 4. **Industry Partnerships:** Equipment donations, cloud credits ($10K-20K) 5. **Cloud Credits:** AWS/Google Cloud research programs ($5K-10K)

1.13 12. Conclusion

This PhD research proposal presents a comprehensive plan to develop an advanced AI-based intrusion detection system for next-generation networks. The proposed system addresses critical limitations of existing IDS through hybrid deep learning architectures, adaptive learning mechanisms, distributed deployment, and explainable AI integration.

1.13.1 Key Strengths of the Proposal

1. **Timely and Relevant:** Addresses urgent security challenges in 5G/6G, IoT, and edge computing
2. **Novel Approach:** Combines cutting-edge AI techniques (Transformers, RL, Federated Learning) in innovative ways
3. **Comprehensive Methodology:** Rigorous experimental design with multiple datasets and evaluation metrics
4. **Practical Impact:** Focus on deployable solutions with real-world validation
5. **Balanced Research:** Theoretical contributions complemented by practical implementation
6. **Feasible Plan:** Realistic 4-year timeline with clear milestones and deliverables

1.13.2 Expected Outcomes

By the completion of this PhD program, the research will deliver:

- **Academic Contributions:** 5-10 peer-reviewed publications in top conferences/journals
- **Practical Framework:** Open-source AI-IDS platform for industry adoption

- **Validated Solution:** Proven performance on benchmark datasets and real testbed
- **Community Impact:** Benchmark datasets, tools, and guidelines for future research
- **Trained Expert:** PhD graduate with deep expertise in AI-driven network security

1.13.3 Broader Impact

This research will advance the state-of-the-art in cybersecurity for next-generation networks, contributing to the protection of critical infrastructure, enterprise systems, and billions of IoT devices. The explainable and privacy-preserving nature of the proposed IDS addresses key concerns for industry adoption, while the open-source release will benefit the broader research community.

The interdisciplinary nature of this research—spanning artificial intelligence, cybersecurity, and networking—positions it to make significant contributions across multiple domains and prepare the candidate for leadership roles in academia or industry.

1.14 13. References

1.14.1 Core IDS & Network Security

[1] Axelsson, S. (2000). "Intrusion Detection Systems: A Survey and Taxonomy." Technical Report, Chalmers University.

[2] Chandola, V., Banerjee, A., & Kumar, V. (2009). "Anomaly Detection: A Survey." ACM Computing Surveys, 41(3), 1-58.

[3] Sommer, R., & Paxson, V. (2010). "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection." IEEE S&P.

[4] Garcia-Teodoro, P., et al. (2009). "Anomaly-based Network Intrusion Detection: Techniques, Systems and Challenges." Computers & Security, 28(1-2), 18-28.

1.14.2 Machine Learning for IDS

[5] Buczak, A. L., & Guven, E. (2016). "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection." IEEE Communications Surveys & Tutorials, 18(2), 1153-1176.

[6] Sultana, N., et al. (2019). "Survey on SDN Based Network Intrusion Detection System Using Machine Learning Approaches." Peer-to-Peer Networking and Applications, 12(2), 493-501.

[7] Vinayakumar, R., et al. (2019). "Deep Learning Approach for Intelligent Intrusion Detection System." IEEE Access, 7, 41525-41550.

[8] Kim, J., et al. (2020). “Long Short-Term Memory Recurrent Neural Network Classifier for Intrusion Detection.” IEEE ICPlatform.

[9] Aygun, R. C., & Yavuz, A. G. (2017). “Network Anomaly Detection with Stochastically Improved Autoencoder Based Models.” IEEE CYBER.

[10] Gaikwad, D. P., & Thool, R. C. (2015). “Intrusion Detection System Using Bagging Ensemble Method of Machine Learning.” IEEE ICCUBEA.

1.14.3 Deep Learning & Neural Networks

[11] Lopez-Martin, M., et al. (2020). “Deep Learning Model for Network Intrusion Detection with Imbalanced Data.” Electronics, 9(6), 898.

[12] Kwon, D., et al. (2019). “A Survey of Deep Learning-Based Network Anomaly Detection.” Cluster Computing, 22(1), 949-961.

[13] Shone, N., et al. (2018). “A Deep Learning Approach to Network Intrusion Detection.” IEEE TNSM, 15(3), 1177-1191.

[14] Potluri, S., & Diedrich, C. (2016). “Accelerated Deep Neural Networks for Enhanced Intrusion Detection System.” IEEE ETFA.

1.14.4 Next-Generation Networks

[15] Ahmad, I., et al. (2019). “5G Security: Analysis of Threats and Solutions.” IEEE Communications Surveys & Tutorials, 21(4), 3682-3721.

[16] Moustafa, N., & Slay, J. (2016). “The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Dataset.” IEEE CNS.

[17] Tang, T. A., et al. (2020). “Deep Recurrent Neural Network for Intrusion Detection in SDN-based Networks.” IEEE NetSoft.

[18] Diro, A. A., & Chilamkurti, N. (2018). “Distributed Attack Detection Scheme Using Deep Learning Approach for Internet of Things.” Future Generation Computer Systems, 82, 761-768.

1.14.5 5G/IoT Security

[19] Ferrag, M. A., et al. (2020). “Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study.” Journal of Information Security and Applications, 50, 102419.

[20] Hodo, E., et al. (2017). “Shallow and Deep Networks Intrusion Detection System: A Taxonomy and Survey.” arXiv preprint arXiv:1701.02145.

[21] Koroniotis, N., et al. (2019). “Towards the Development of Realistic Botnet Dataset in the IoT for Network Forensic Analytics: Bot-IoT Dataset.” Future Generation Computer Systems, 100, 779-796.

[22] HaddadPajouh, H., et al. (2018). “A Deep Recurrent Neural Network Based Approach for Internet of Things Malware Threat Hunting.” Future Generation Computer Systems, 85, 88-96.

1.14.6 SDN/NFV Security

[23] Scott-Hayward, S., et al. (2016). “A Survey of Security in Software Defined Networks.” IEEE Communications Surveys & Tutorials, 18(1), 623-654.

[24] Ashraf, J., & Latif, S. (2020). “Handling Intrusion and DDoS Attacks in Software Defined Networks Using Machine Learning Techniques.” Software & Peer-to-Peer Networking and Applications, 13(5), 1-14.

[25] Elsayed, M. S., et al. (2020). “InSDN: A Novel SDN Intrusion Dataset.” IEEE Access, 8, 165263-165284.

1.14.7 Transformers & Attention Mechanisms

[26] Vaswani, A., et al. (2017). “Attention Is All You Need.” NeurIPS.

[27] Lin, P., et al. (2022). “BERT-Based Intrusion Detection System for Software-Defined Networking.” IEEE ICC.

[28] Zhang, Y., et al. (2021). “Network Intrusion Detection Based on Transformer Model.” IEEE ICCC.

1.14.8 Federated Learning & Privacy

[29] McMahan, B., et al. (2017). “Communication-Efficient Learning of Deep Networks from Decentralized Data.” AISTATS.

[30] Zhao, Y., et al. (2020). “Privacy-Preserving Blockchain-Based Federated Learning for IoT Devices.” IEEE IoT Journal, 8(3), 1817-1829.

[31] Mothukuri, V., et al. (2021). “A Survey on Security and Privacy of Federated Learning.” Future Generation Computer Systems, 115, 619-640.

[32] Nguyen, T. D., et al. (2022). “Federated Learning for Intrusion Detection Systems.” Journal of Network and Computer Applications, 194, 103209.

1.14.9 Explainable AI

[33] Lundberg, S. M., & Lee, S. I. (2017). “A Unified Approach to Interpreting Model Predictions.” NeurIPS.

[34] Ribeiro, M. T., et al. (2016). “Why Should I Trust You?: Explaining the Predictions of Any Classifier.” ACM KDD.

[35] Islam, S. R., et al. (2021). “Explainable Artificial Intelligence Approaches: A Survey.” arXiv preprint arXiv:2101.09429.

[36] Kuppa, A., et al. (2021). “Black-Box Adversarial Attacks on XAI Methods in Cyber Security.” IEEE BigData.

1.14.10 Reinforcement Learning for Security

[37] Nguyen, T. T., & Reddi, V. J. (2021). "Deep Reinforcement Learning for Cyber Security." IEEE TNNLS, 32(8), 3779-3795.

[38] Servin, A., & Kudenko, D. (2008). "Multi-Agent Reinforcement Learning for Intrusion Detection." Adaptive Agents and Multi-Agent Systems III.

[39] Malialis, K., et al. (2021). "Distributed Reinforcement Learning for Adaptive Cyber Defense." ACM AAMAS.

1.14.11 Adversarial Machine Learning

[40] Biggio, B., & Roli, F. (2018). "Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning." Pattern Recognition, 84, 317-331.

[41] Pierazzi, F., et al. (2020). "Intriguing Properties of Adversarial ML Attacks in the Problem Space." IEEE S&P.

[42] Corona, I., et al. (2013). "Adversarial Attacks Against Intrusion Detection Systems: Taxonomy, Solutions and Open Issues." Information Sciences, 239, 201-225.

1.14.12 Graph Neural Networks

[43] Lo, W. W., et al. (2022). "E-GraphSAGE: A Graph Neural Network Based Intrusion Detection System for IoT." IEEE NOMS.

[44] Zhou, Y., et al. (2021). "Graph Neural Network-Based Anomaly Detection in Multivariate Time Series." AAAI.

[45] Zheng, M., et al. (2020). "Graph Neural Network Based Intrusion Detection System for In-Vehicle Network." IEEE ICCS.

1.14.13 Transfer Learning & Meta-Learning

[46] Pan, S. J., & Yang, Q. (2010). "A Survey on Transfer Learning." IEEE TKDE, 22(10), 1345-1359.

[47] Hospedales, T., et al. (2021). "Meta-Learning in Neural Networks: A Survey." IEEE TPAMI.

[48] Ring, M., et al. (2019). "A Survey of Network-Based Intrusion Detection Data Sets." Computers & Security, 86, 147-167.

1.14.14 Benchmark Datasets

[49] Tavallaee, M., et al. (2009). "A Detailed Analysis of the KDD CUP 99 Data Set." IEEE CISDA.

[50] Sharafaldin, I., et al. (2018). "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." ICISSP.

[51] Garcia, S., et al. (2020). "An Empirical Comparison of Botnet Detection Methods." Computers & Security, 45, 100-123.

1.14.15 Recent Advances (2021-2025)

[52] Xiao, Y., et al. (2023). "EdgeIDS: An Edge-Based Deep Learning Framework for Real-Time Intrusion Detection in IoT Networks." IEEE IoT Journal, 10(5), 4321-4335.

[53] Wang, Z., et al. (2024). "Transformer-Enhanced Federated Learning for Privacy-Preserving Intrusion Detection in 6G Networks." IEEE TWC, 23(2), 1456-1471.

[54] Chen, L., et al. (2024). "Explainable AI for Network Security: A Comprehensive Framework." ACM Computing Surveys, 56(3), 1-42.

[55] Kumar, R., et al. (2023). "Adaptive Deep Reinforcement Learning for Dynamic Threat Mitigation in SDN." IEEE TIFS, 18, 2789-2804.

End of PhD Research Proposal

This proposal represents a comprehensive research plan for developing advanced AI-based intrusion detection systems for next-generation networks. The proposed work addresses critical security challenges through innovative AI techniques, rigorous evaluation, and practical deployment considerations.

For further information or collaboration opportunities, please contact: idriss5214@github.com # Executive Summary ## AI-Based Intrusion Detection Systems for Next-Generation Networks

PhD Research Proposal
Candidate: idriss5214
Date: December 22, 2025
Duration: 4 Years

1.15 Research Context and Significance

The cybersecurity landscape is undergoing a fundamental transformation driven by the proliferation of next-generation networks (NGN). Technologies such as 5G/6G, Internet of Things (IoT), Software-Defined Networks (SDN), Network Function Virtualization (NFV), and edge computing are revolutionizing how we communicate, compute, and connect. However, these advancements introduce unprecedented security challenges that traditional intrusion detection systems (IDS) are ill-equipped to address.

Current IDS solutions suffer from critical limitations: - **Signature-based systems** cannot detect zero-day attacks or polymorphic malware - **Anomaly-based systems** generate unacceptably high false positive rates (10-20%) - **Static models** fail to adapt to evolving threat landscapes and concept drift - **Centralized architectures** create

bottlenecks and privacy concerns - **Black-box ML models** lack explainability, hindering security analyst trust and adoption - **Resource-intensive processing** struggles with gigabit-speed traffic and real-time requirements

This research addresses these critical gaps through the development of an advanced AI-based intrusion detection framework that combines cutting-edge deep learning, adaptive mechanisms, distributed architectures, and explainable AI to provide superior threat detection across diverse network environments.

1.16 Problem Statement

How can we develop an AI-based intrusion detection system that achieves high accuracy (>98%), low false positive rates (<1%), real-time performance (<100ms latency), scalability across heterogeneous networks, adaptability to evolving threats, and transparent decision-making while preserving data privacy?

This multifaceted challenge requires innovations in: 1. Deep learning architectures for multi-scale feature extraction 2. Adaptive learning mechanisms for continuous improvement 3. Distributed detection frameworks for edge computing 4. Explainable AI for analyst trust and regulatory compliance 5. Privacy-preserving techniques for federated threat intelligence

1.17 Research Objectives

1.17.1 Primary Objective

Design, develop, and validate an advanced AI-based intrusion detection system for next-generation networks that surpasses existing solutions in accuracy, performance, scalability, adaptability, and explainability.

1.17.2 Specific Objectives

1. **Hybrid AI Model Development**
    - Create CNN-LSTM hybrid architecture for spatial-temporal feature extraction
    - Implement Transformer networks for capturing long-range traffic dependencies
    - Develop autoencoders for unsupervised anomaly detection
    - Design ensemble methods combining multiple AI paradigms
2. **Adaptive Learning Framework**
    - Integrate reinforcement learning (DQN, A3C, PPO) for dynamic threat response
    - Implement online learning for continuous model adaptation
    - Develop transfer learning for cross-domain attack detection
    - Create meta-learning for rapid adaptation to new threats
3. **Distributed IDS Architecture**

- Design hierarchical edge-fog-cloud detection framework
- Implement federated learning for privacy-preserving collaboration
- Develop model compression for resource-constrained edge devices
- Create secure aggregation protocols for distributed intelligence

4. **Explainable AI Integration**
   - Integrate SHAP, LIME, and attention mechanisms for interpretability
   - Develop visualization dashboards for security analysts
   - Create explainability metrics for IDS evaluation
   - Design human-in-the-loop feedback systems
5. **Comprehensive Validation**
   - Evaluate on benchmark datasets (NSL-KDD, CICIDS2017, UNSW-NB15, IoT-23)
   - Deploy physical testbed with 5G/IoT/SDN infrastructure
   - Conduct extensive cross-environment experiments
   - Compare against state-of-the-art baselines

1.18 Proposed Methodology

1.18.1 System Architecture

The proposed framework consists of seven interconnected layers:

1. **Data Collection Layer:** Network sensors, SDN controllers, IoT gateways capture multi-source traffic
2. **Preprocessing Module:** Feature extraction, normalization, encoding, and class balancing
3. **AI Detection Engine:**
   - CNN-LSTM hybrid for spatial-temporal analysis
   - Transformer networks for sequence modeling
   - Autoencoders for anomaly detection
   - Ensemble methods (Random Forest, XGBoost)
   - Adaptive learning (RL, online learning)
4. **Explainable AI Module:** SHAP, LIME, attention visualization
5. **Decision & Response Layer:** Alert generation, automated response, human interface
6. **Distributed Architecture:** Edge-fog-cloud hierarchy with federated learning
7. **Privacy Mechanisms:** Differential privacy, secure aggregation

1.18.2 Key Innovations

Hybrid Deep Learning Architecture:

```
CNN Layers → Extract spatial features from traffic matrices
LSTM Layers → Model temporal dependencies in sequences
Transformer → Capture long-range dependencies via attention
Ensemble → Combine predictions for robustness
```

Adaptive Learning: - Reinforcement learning agents learn optimal defense policies - Online learning enables continuous model updates with streaming data - Transfer

learning enables knowledge sharing across network domains

Federated Detection: - Local models train on edge devices without data sharing - Secure aggregation combines model updates at fog layer - Global model distributed back to edge for improved detection

1.18.3 Experimental Design

Datasets: - NSL-KDD: Classic benchmark with balanced classes (126K training samples) - CICIDS2017: Modern attacks with realistic traffic (2.8M flows) - UNSW-NB15: Hybrid synthetic-real attacks (2.5M records) - IoT-23: IoT-specific botnet traffic (325M packets) - Custom 5G dataset: Collected from testbed deployment

Testbed: - 4× NVIDIA A100 GPUs for deep learning training - 5× OpenFlow SDN switches with ONOS controller - 10× Raspberry Pi 4 edge nodes for distributed deployment - 50× IoT devices (sensors, cameras, smart devices) - Network monitoring infrastructure (Grafana, Prometheus)

Evaluation Metrics: - Detection Performance: Accuracy (>98%), Precision (>97%), Recall (>98%), F1-Score (>97%), FPR (<1%), AUC-ROC (>0.99) - Efficiency: Latency (<100ms), Throughput (>10K pps), CPU (<50%), Memory (<2GB) - Scalability: Performance vs. network size (100 to 100K devices) - Adaptability: Zero-day detection rate, concept drift handling - Explainability: Feature importance, analyst evaluation scores

1.19 Expected Impact

1.19.1 Academic Contributions

1. **Novel Architectures:** CNN-LSTM-Transformer fusion with attention for IDS
2. **Theoretical Frameworks:** Mathematical models for distributed IDS optimization
3. **Adaptive Learning:** RL-based dynamic threat response mechanisms
4. **Explainability Methods:** XAI techniques tailored for network security
5. **Publications:** 5-10 papers in top venues (IEEE S&P, USENIX Security, CCS, NDSS, IEEE TIFS)

1.19.2 Practical Impact

1. **Deployable Framework:** Open-source AI-IDS platform with comprehensive documentation
2. **Industry Guidelines:** Best practices for production deployment
3. **Tool Ecosystem:** Automated pipeline for feature engineering, training, evaluation
4. **Benchmark Datasets:** Curated 5G/IoT attack scenarios for reproducible research
5. **Testbed Blueprint:** Documented setup for academic and industry replication

1.19.3 Societal Benefits

- **Critical Infrastructure Protection:** Secure power grids, healthcare, transportation
- **Enterprise Security:** Enhanced threat detection for businesses
- **IoT Ecosystem Safety:** Protect billions of connected devices
- **Privacy Preservation:** Federated learning respects data sovereignty
- **Economic Value:** Reduce cybercrime costs (projected $10.5T annually by 2025)

1.20 Timeline Overview

1.20.1 Year 1: Foundation (Months 1-12)

- Comprehensive literature review (100+ papers)
- PhD coursework completion
- Baseline model implementation (CNN, LSTM, RF, SVM)
- Initial experiments on NSL-KDD and CICIDS2017
- **Milestone:** Comprehensive exam passed, first paper submitted

1.20.2 Year 2: Development (Months 13-24)

- Hybrid architecture implementation (CNN-LSTM-Transformer)
- Adaptive learning integration (RL, online learning)
- Distributed framework design (federated learning)
- Testbed deployment preparation
- **Milestone:** Testbed operational, second publication submitted

1.20.3 Year 3: Validation (Months 25-36)

- Extensive experiments on all datasets
- 5G/IoT-specific validation
- Performance optimization (<100ms latency)
- Large-scale scalability testing
- **Milestone:** Framework v1.0 released, 3+ publications accepted

1.20.4 Year 4: Completion (Months 37-48)

- Additional experiments addressing reviewer feedback
- User studies for explainability evaluation
- Thesis writing and revision
- Defense preparation and final submission
- **Milestone:** PhD degree awarded, published papers

1.21 Resource Requirements

Budget Summary: $53,000 over 4 years

Category	Amount
Computational Resources (Cloud GPU credits)	$15,000
Network Testbed Hardware (SDN, IoT, servers)	$18,000
Conference Travel (4 major conferences)	$10,000
Software Licenses & Tools	$2,500
Publications (Open access fees)	$3,500
Miscellaneous (books, materials)	$4,000

Funding Sources: - University research assistantship - Department research grants - External fellowships (NSF GRFP) - Industry partnerships (equipment donations) - Cloud provider research credits (AWS, Google)

1.22 Keywords

Intrusion Detection Systems, Next-Generation Networks, Deep Learning, Artificial Intelligence, 5G/6G Security, IoT Security, Software-Defined Networks, Edge Computing, Explainable AI, Federated Learning, Cybersecurity, Adaptive Learning, Reinforcement Learning, Network Security, Privacy-Preserving Machine Learning

1.23 Conclusion

This PhD research proposes a comprehensive, innovative approach to intrusion detection for next-generation networks. By integrating hybrid deep learning architectures, adaptive learning mechanisms, distributed deployment strategies, and explainable AI, the proposed system addresses critical limitations of existing IDS solutions.

The research is timely, addressing urgent security challenges in 5G/6G, IoT, and edge computing. The methodology is rigorous, with extensive validation across multiple datasets and real testbed deployment. The expected outcomes include both theoretical advances (novel algorithms, mathematical frameworks) and practical contributions (open-source platform, deployment guidelines, tools).

Upon completion, this research will deliver a deployable AI-IDS framework that achieves superior detection performance while maintaining transparency, scalability, and privacy—advancing both academic knowledge and practical cybersecurity capabilities for our increasingly connected world.

For detailed information, refer to the complete proposal document: `proposal/full_proposal.md`

Contact: idriss5214@github.com # Literature Review ## AI-Based Intrusion Detection Systems for Next-Generation Networks

Comprehensive Review of Current State-of-the-Art
Last Updated: December 22, 2025

1.24 Table of Contents

1.25 1. Traditional Intrusion Detection Systems

1.25.1 1.1 Signature-Based IDS

Overview: Signature-based (or misuse-based) IDS rely on predefined patterns or signatures of known attacks. These systems match observed network traffic or system behavior against a database of attack signatures to identify malicious activity.

Key Systems: - **Snort [1]:** Open-source network intrusion detection system using rule-based pattern matching. Widely deployed in enterprise environments with extensive rule sets. - **Suricata [2]:** Multi-threaded successor to Snort with improved performance and protocol support. Supports IPS mode with inline deployment. - **YARA [3]:** Pattern matching tool primarily for malware detection but applicable to network traffic analysis.

Strengths: - High accuracy for known attacks (near 100% detection rate) - Low false positive rates (<1%) - Efficient processing with optimized pattern matching algorithms - Clear, interpretable alerts with specific attack identification - Well-established in industry with mature tooling

Limitations: - Cannot detect zero-day attacks or novel attack variants - Requires continuous signature updates (daily/weekly) - Vulnerable to polymorphic and metamorphic malware - Signature databases grow large, impacting performance - Evasion through obfuscation, encoding, fragmentation [4] - Reactive approach—detection only after attack is known

Research Gap: Need for proactive detection mechanisms that identify unknown threats without prior signatures.

1.25.2 1.2 Anomaly-Based IDS

Overview: Anomaly-based IDS establish a baseline of “normal” network behavior and flag deviations as potential intrusions. These systems can detect novel attacks but suffer from high false positive rates.

Approaches:

Statistical Methods [5]: - Threshold-based detection (mean, standard deviation) - Multivariate analysis (Mahalanobis distance, chi-square) - Time-series analysis (ARIMA, exponential smoothing) - *Limitation:* Assume normal distribution, struggle with complex patterns

Classical Machine Learning [6]: - k-Nearest Neighbors (k-NN): Instance-based learning - Support Vector Machines (SVM): Hyperplane separation with kernel tricks - Decision Trees: Rule-based classification with interpretable paths - Naive Bayes: Probabilistic classification assuming feature independence - *Achievement:* 85-95% accuracy on benchmark datasets (KDD'99, NSL-KDD)

Clustering Methods [7]: - K-means: Partition-based clustering - DBSCAN: Density-based spatial clustering - Hierarchical clustering: Agglomerative/divisive approaches - Self-Organizing Maps (SOM): Neural network-based clustering - *Use Case:* Unsupervised learning for discovering unknown attack patterns

Strengths: - Can detect zero-day attacks and novel intrusions - No signature database maintenance required - Adapts to network changes over time - Discovers previously unknown attack patterns

Limitations: - High false positive rates (10-30%) in practice [8] - Difficult to establish accurate baseline in dynamic environments - Sensitive to concept drift as network behavior evolves - Computational overhead for continuous monitoring - Requires extensive training data representing normal behavior - Difficulty distinguishing legitimate anomalies from attacks

Research Gap: Need for models that maintain low false positive rates while detecting unknown attacks.

1.25.3 1.3 Hybrid Approaches

Concept: Combine signature-based and anomaly-based detection to leverage strengths of both approaches.

Systems: - **Bro/Zeek [9]:** Hybrid network security monitor with policy-based scripting - **OSSEC [10]:** Host-based IDS with log analysis and anomaly detection - **Prelude [11]:** Hybrid IDS framework with correlation engine

Architecture:

```
Network Traffic → Signature Matching → Known Attack? → Alert
                        ↓ (No match)
                Anomaly Detection → Suspicious? → Alert (with confidence)
```

Evaluation [12]: - Detection rate: 92-96% (better than individual approaches) - False positive rate: 5-8% (moderate) - Processing overhead: 2-3× higher than signature-only

Limitations: - Inherits limitations from both paradigms - Increased computational complexity - Difficult to balance detection thresholds - Still struggles with adaptive, evolving threats

Research Gap: Need for intelligent fusion mechanisms that dynamically weight signature vs. anomaly detection.

1.26 2. AI/ML Techniques in Cybersecurity

1.26.1 2.1 Classical Machine Learning

1.26.1.1 2.1.1 Random Forest and Ensemble Methods

Random Forest [13]: Ensemble of decision trees with bootstrap aggregating (bagging) and random feature selection.

Application to IDS: - Ingale & Nasiruddin (2014): 99.67% accuracy on NSL-KDD with 100 trees [14] - Advantages: Handles high-dimensional data, resistant to overfitting, feature importance - Limitations: Computational cost for large datasets, black-box nature

Gradient Boosting [15]: - XGBoost: 98.5% accuracy on CICIDS2017 with optimized hyperparameters - LightGBM: Faster training with leaf-wise tree growth - CatBoost: Handles categorical features natively

AdaBoost [16]: Sequential ensemble focusing on misclassified instances.

Stacking [17]: Meta-learner combines predictions from multiple base models, achieving 2-3% accuracy improvement.

1.26.1.2 2.1.2 Support Vector Machines

SVM Theory [18]: Find optimal hyperplane maximizing margin between classes. Kernel trick enables non-linear separation.

Kernel Functions: - Linear: Fast but limited expressiveness - RBF (Gaussian): Most popular for IDS, handles non-linearity - Polynomial: Higher-degree relationships - Sigmoid: Neural network-like decision boundary

Performance [19]: - Binary classification: 95-98% accuracy on NSL-KDD - Multi-class: 90-93% accuracy (DoS, Probe, R2L, U2R) - Limitations: Slow training on large datasets ($O(n^3)$), sensitive to hyperparameters

One-Class SVM [20]: Unsupervised anomaly detection by learning boundary of normal class. - Precision: 85-90% but high false positives - Useful when attack samples are scarce

1.26.2 2.2 Deep Learning for IDS

1.26.2.1 2.2.1 Convolutional Neural Networks (CNN)

CNN Architecture for Network Traffic [21]:

```
Traffic Matrix (Image-like) → Conv1D/2D → Pooling → Dense → Softmax
```

Key Research:

Kim et al. (2020) [22]: - 1D CNN on traffic flow features - Architecture: Conv1D(128) → MaxPool → Conv1D(256) → GlobalMaxPool → Dense(128) → Output - Results: 97.8% accuracy on CICIDS2017, 12ms inference time

Vinayakumar et al. (2019) [23]: - 2D CNN treating traffic as images - Data representation: Traffic flows as grayscale images (28×28, 64×64) - Results: 98.3% accuracy on NSL-KDD, better than classical ML

Advantages: - Automatic feature learning (no manual engineering) - Spatial pattern recognition in packet headers - Translation invariance (attack location in sequence) - Parallel processing on GPUs for speed

Limitations: - Requires large training datasets (100K+ samples) - Limited temporal modeling (need RNN for sequences) - Black-box nature (lack of interpretability) - Vulnerable to adversarial examples

1.26.2.2 2.2.2 Recurrent Neural Networks (RNN/LSTM/GRU)

LSTM Architecture [24]:

```
Input → LSTM Cell (forget/input/output gates) → Hidden State → Output
```

Gate Mechanisms: - Forget gate: What to discard from cell state - Input gate: What new information to store - Output gate: What to output based on cell state

Key Research:

Yin et al. (2017) [25]: - LSTM for temporal modeling of network flows - Architecture: LSTM(128) → LSTM(64) → Dense(num_classes) - Results: 96.4% accuracy on NSL-KDD, captures temporal dependencies

GRU Variant [26]: - Simplified gating (reset + update gates) - Faster training than LSTM (fewer parameters) - Similar performance: 96.1% accuracy on CICIDS2017

Bidirectional LSTM [27]: - Process sequences forward and backward - Captures future context for better understanding - Results: 98.1% accuracy on UNSW-NB15

Advantages: - Models temporal dependencies in traffic sequences - Handles variable-length inputs naturally - Remembers long-term patterns (LSTM addresses vanishing gradient)

Limitations: - Sequential processing (slow compared to CNN) - Difficult to train (vanishing/exploding gradients) - Still black-box (limited interpretability)

1.26.2.3 2.2.3 Hybrid CNN-LSTM/GRU

Motivation: Combine CNN's spatial feature extraction with LSTM's temporal modeling.

Architecture [28]:

```
Input Sequence → CNN (spatial features) → LSTM (temporal patterns) → Dense → Output
```

Key Research:

Lopez-Martin et al. (2020) [29]: - CNN-LSTM hybrid for network intrusion detection - CNN extracts features, LSTM models time dependencies - Results: 98.7% accuracy on CICIDS2017, F1-score: 98.2%

Xu et al. (2021) [30]: - CNN-GRU with attention mechanism - Attention focuses on important time steps - Results: 99.1% accuracy on NSL-KDD, 97.8% on UNSW-NB15

Advantages: - Best of both worlds: spatial + temporal - Outperforms standalone CNN or LSTM (2-4% improvement) - More robust to variations in attack patterns

Limitations: - Increased model complexity and training time - Higher computational requirements - Still lacks interpretability

1.26.2.4 2.2.4 Autoencoders for Anomaly Detection

Concept: Unsupervised learning to reconstruct normal traffic. High reconstruction error indicates anomaly.

Architecture [31]:

```
Encoder: Input → Latent Representation (bottleneck)
Decoder: Latent → Reconstructed Output
Loss: MSE(Input, Reconstructed)
```

Variants:

Stacked Autoencoder [32]: - Multiple encoding/decoding layers - Deep feature learning - Results: 94.6% detection rate on KDD'99

Variational Autoencoder (VAE) [33]: - Probabilistic latent space - Samples from learned distribution - Better generalization: 96.2% accuracy on UNSW-NB15

Adversarial Autoencoder [34]: - GAN-based regularization of latent space - More robust representations - Results: 97.1% detection rate with lower false positives

Advantages: - Unsupervised/semi-supervised (no labeled attacks needed) - Naturally detects outliers (novel attacks) - Compact representation in latent space

Limitations: - Threshold tuning for anomaly decision (trade-off: TPR vs. FPR) - Can struggle with complex attack patterns - Requires careful architecture design for different data types

1.26.3 2.3 Ensemble Methods in Deep Learning

Concept: Combine multiple deep learning models to improve robustness and accuracy.

Approaches:

Voting Ensemble [35]: - Hard voting: Majority class from multiple models - Soft voting: Weighted average of probabilities - Results: 98.8% accuracy combining CNN, LSTM, GRU

Stacking Ensemble [36]: - Base models: CNN, LSTM, Autoencoder - Meta-model: Random Forest or Neural Network - Results: 99.2% accuracy on CICIDS2017

Boosting for Deep Learning [37]: - AdaBoost with neural networks as weak learners - Gradient boosting with neural network base estimators

Advantages: - Improved accuracy and robustness (1-3% gain) - Reduced variance (model stability) - Better generalization to unseen attacks

Limitations: - Increased computational cost (N× models) - Higher memory requirements - Diminishing returns beyond 5-7 models

1.26.4 2.4 Reinforcement Learning for Adaptive IDS

Concept: Agent learns optimal defense policy through interaction with network environment.

Framework: - State: Network observations (traffic features, system state) - Action: Response decisions (block, allow, rate-limit, investigate) - Reward: Correct detection (+), false alarm (-), missed attack (- -) - Policy: Mapping from states to actions

Algorithms:

Q-Learning [38]: - Learn Q-value function: Q(s, a) = expected future reward - ε-greedy exploration strategy - Results: 94.3% detection rate with adaptive response

Deep Q-Network (DQN) [39]: - Neural network approximates Q-function - Experience replay for stable training - Target network for TD updates - Results: 96.7% detection, adapts to evolving threats

Actor-Critic Methods [40]: - Actor: Policy network (select actions) - Critic: Value network (evaluate actions) - A3C (Asynchronous Advantage Actor-Critic): Parallel training - Results: 97.2% detection with 15% faster adaptation

Proximal Policy Optimization (PPO) [41]: - More stable policy updates (clipped objective) - Better sample efficiency than A3C - Results: 97.8% detection, smoother learning curve

Advantages: - Adaptive to evolving threats (online learning) - Learns optimal response strategies - Balances exploration vs. exploitation - Can handle partial observability (POMDP)

Limitations: - Requires extensive training (millions of interactions) - Difficult reward engineering - Sample inefficiency in sparse reward scenarios - Safety concerns during exploration

---

1.27 3. Next-Generation Network Security

1.27.1 3.1 5G/6G Security

5G Architecture Vulnerabilities [42]:

Network Slicing: - Isolation failures between slices - Resource starvation attacks - Slice hijacking through control plane exploitation

Edge Computing: - Increased attack surface at edge nodes - Physical tampering risks - Distributed denial-of-service from compromised edges

Massive IoT: - Billions of low-security devices - Amplification attacks using IoT botnets - Signaling storms from misbehaving devices

Key Research:

Ahmad et al. (2019) [43]: - Comprehensive survey of 5G security threats - Identified 50+ unique vulnerabilities across layers - Proposed ML-based anomaly detection for 5G core

Li et al. (2020) [44]: - Deep learning IDS for 5G network slicing - CNN-LSTM hybrid for slice-specific attack detection - Results: 97.1% accuracy, <50ms detection latency

Ferrag et al. (2021) [45]: - Federated learning for distributed 5G IDS - Privacy-preserving collaborative threat intelligence - Results: 96.8% accuracy without centralizing data

Research Gaps: - Limited real 5G attack datasets (mostly simulated) - Scalability to millions of network slices - Real-time processing at 10+ Gbps speeds - Cross-slice attack detection

1.27.2 3.2 IoT Security

IoT Threat Landscape [46]:

Device Constraints: - Limited CPU (ARM Cortex-M, 50-200 MHz) - Restricted memory (32-512 KB RAM) - Power constraints (battery-operated) - Minimal security capabilities

Attack Types: - Mirai botnet (DDoS amplification) - Brute-force attacks (default credentials) - Firmware exploitation (buffer overflows) - Physical tampering - Side-channel attacks

Key Research:

Diro & Chilamkurti (2018) [47]: - Distributed deep learning IDS for IoT fog computing - Lightweight model on edge, full model in fog - Results: 98.27% accuracy on BoT-IoT dataset

HaddadPajouh et al. (2018) [48]: - LSTM-based IoT malware detection - Analyzes system call sequences - Results: 98.18% accuracy on IoT-23 dataset

Koroniotis et al. (2019) [49]: - Bot-IoT dataset with realistic IoT botnet traffic - Evaluated 9 ML algorithms - Best: Random Forest with 99.94% accuracy (may indicate overfitting)

Lightweight Solutions:

MobileNet for IoT IDS [50]: - Depthwise separable convolutions - 10× fewer parameters than standard CNN - Results: 96.3% accuracy with 5MB model size

Binary Neural Networks [51]: - 1-bit weights and activations - 32× memory reduction - Results: 94.1% accuracy (slight degradation for efficiency)

Research Gaps: - Energy-efficient IDS for battery-powered devices - Real-time detection on resource-constrained MCUs - Heterogeneous IoT protocol support (Zigbee, Z-Wave, LoRa) - Scalability to billions of devices

1.27.3 3.3 Software-Defined Networks (SDN)

SDN Architecture [52]:

```
Application Layer (Security Apps)
        ↕ Northbound API
Control Layer (SDN Controller: ONOS, ODL)
        ↕ Southbound API (OpenFlow)
Data Layer (OpenFlow Switches)
```

SDN Attack Vectors [53]:

Control Plane: - Controller DoS (overwhelming with requests) - Flow table overflow - Topology poisoning - Man-in-the-middle on control channel

Data Plane: - Switch compromise - Flow rule manipulation - Packet flooding

API Vulnerabilities: - Unauthorized access to northbound APIs - Southbound protocol exploits (OpenFlow)

Key Research:

Tang et al. (2020) [54]: - Deep RNN for SDN intrusion detection - Real-time analysis of OpenFlow statistics - Results: 95.74% accuracy, 23ms detection time

Elsayed et al. (2020) [55]: - InSDN dataset with realistic SDN attacks - Comprehensive feature set (104 features) - Evaluated 8 ML algorithms, best: Random Forest (98.92%)

Ashraf & Latif (2020) [56]: - Machine learning for SDN DDoS detection - Decision tree-based classifier at controller - Results: 99.8% accuracy with 10ms response time

Distributed SDN IDS [57]: - Multi-controller architecture - Federated learning across controllers - Load balancing and resilience - Results: 97.5% accuracy with horizontal scalability

Research Gaps: - Protection against controller-targeted attacks - Scalability to large-scale data center SDN - Multi-domain SDN security (inter-controller) - Integration with NFV security

1.27.4 3.4 Edge Computing Security

Edge Computing Paradigm [58]:

```
Cloud (Centralized processing, storage)
  ↕
Fog (Regional aggregation, coordination)
```

```
  ↕
Edge (Local processing, low latency)
  ↕
IoT Devices (Data generation)
```

Security Challenges:

Distributed Attack Surface: - Thousands of edge nodes, each a potential target - Physical security risks (public locations) - Heterogeneous devices and platforms

Resource Constraints: - Limited compute for complex IDS models - Bandwidth constraints for cloud communication - Power limitations

Data Privacy: - Sensitive data processed at edge - Multi-tenancy security issues - Compliance with regional data laws (GDPR)

Key Research:

Preuveneers et al. (2018) [59]: - Distributed IDS at fog layer - Anomaly detection with federated learning - Results: 94.6% accuracy without centralizing data

Chen et al. (2019) [60]: - Lightweight deep learning for edge IDS - Knowledge distillation from complex teacher - Results: 96.1% accuracy with 10× speedup

Xiao et al. (2023) [61]: - EdgeIDS framework for real-time IoT protection - Hybrid edge-cloud architecture - Results: 98.2% accuracy, <30ms latency

Research Gaps: - Model compression for extreme edge (MCU-level) - Hierarchical coordination (edge-fog-cloud) - Privacy-utility trade-offs in federated scenarios - Real-time updates to edge models

---

1.28 4. Recent Advances (2019-2025)

1.28.1 4.1 Transformer Networks for IDS

Background: Transformers, introduced in "Attention Is All You Need" [62], revolutionized NLP. Their self-attention mechanism captures long-range dependencies without sequential processing.

Application to IDS:

Architecture:

```
Traffic Sequence → Token Embedding → Positional Encoding
  → Multi-Head Self-Attention → Feed-Forward → Classification
```

Key Research:

Lin et al. (2022) [63]: - BERT-based IDS for SDN - Pre-training on normal traffic, fine-tuning for attack detection - Results: 98.9% accuracy on InSDN dataset

Zhang et al. (2021) [64]: - Transformer encoder for network traffic classification - Multi-head attention focuses on critical packet features - Results: 99.2% accuracy on CICIDS2018

Advantages: - Parallel processing (faster than RNN/LSTM) - Captures global dependencies (entire traffic sequence) - Attention weights provide interpretability - State-of-the-art results on sequence tasks

Limitations: - Computational complexity $O(n^2)$ for sequence length n - Requires large datasets for effective training - Memory intensive for long sequences - Position encoding may not suit all network patterns

Research Gap: Efficient transformers for high-speed networks (sparse attention, linear attention).

1.28.2 4.2 Federated Learning for Privacy-Preserving IDS

Concept [65]: Train global model collaboratively without centralizing data. Each node trains locally, shares only model updates.

Process:

```
1. Server distributes initial model to clients
2. Clients train on local data
3. Clients send model updates (gradients/weights) to server
4. Server aggregates updates (FedAvg, FedProx)
5. Server distributes updated global model
6. Repeat
```

Key Research:

Zhao et al. (2020) [66]: - Federated learning for IoT intrusion detection - Blockchain-based secure aggregation - Results: 96.5% accuracy with privacy preservation

Nguyen et al. (2022) [67]: - Comprehensive survey on federated IDS - Identified challenges: non-IID data, communication cost, adversarial clients - Solutions: personalized federated learning, compression, Byzantine-robust aggregation

Mothukuri et al. (2021) [68]: - Security and privacy of federated learning itself - Attacks: Model poisoning, inference attacks, backdoor injection - Defenses: Differential privacy, secure aggregation, client selection

Advantages: - Data privacy (complies with GDPR, HIPAA) - Scalable to distributed networks - Leverages diverse data from multiple sources - Reduces bandwidth (model updates < raw data)

Limitations: - Communication overhead (model updates every round) - Non-IID data distribution across clients - Vulnerability to poisoning attacks - Slow convergence compared to centralized

Research Gaps: - Efficient aggregation for large models - Handling extreme non-IID scenarios - Byzantine-robust algorithms for adversarial clients - Personalization for client-specific threats

1.28.3 4.3 Explainable AI (XAI) for IDS

Motivation: Black-box ML models lack trust in security contexts. Analysts need to understand why a decision was made.

Techniques:

SHAP (SHapley Additive exPlanations) [69]: - Game-theoretic approach to feature importance - Consistent, locally accurate explanations - Global feature importance via aggregation

LIME (Local Interpretable Model-agnostic Explanations) [70]: - Perturb input, observe output changes - Fit local linear model around instance - Fast, model-agnostic

Attention Mechanism [71]: - Visualize attention weights in neural networks - Identify which traffic features/timestamps influenced decision - Built into Transformer, CNN, LSTM with attention

Layer-wise Relevance Propagation (LRP) [72]: - Backpropagate prediction through network layers - Assign relevance scores to input features - Specific to neural architectures

Key Research:

Islam et al. (2021) [73]: - Survey of XAI approaches for cybersecurity - Evaluated SHAP, LIME, attention for IDS - Found SHAP most consistent, LIME fastest

Kuppa et al. (2021) [74]: - Black-box adversarial attacks on XAI methods - Showed explanations can be manipulated - Need for robust XAI techniques

Marino et al. (2023) [75]: - Adversarial robustness of IDS with XAI - XAI-guided adversarial training improves robustness - Results: 3-5% improvement in adversarial accuracy

Advantages: - Builds analyst trust in automated decisions - Enables debugging and model improvement - Regulatory compliance (EU "right to explanation") - Identifies biases in models

Limitations: - Computational overhead (SHAP: exponential in features) - Explanations may not always align with true model behavior - Trade-off between accuracy and interpretability - Adversarial manipulation of explanations

Research Gap: Domain-specific XAI tailored for network security, real-time explainability, evaluation metrics for explanation quality.

1.28.4 4.4 Graph Neural Networks (GNN)

Motivation: Networks are naturally graph-structured (devices as nodes, connections as edges). GNNs leverage this structure.

Architectures:

Graph Convolutional Networks (GCN) [76]: - Aggregate neighbor features via convolution on graphs - Learn node embeddings capturing local topology

GraphSAGE [77]: - Sample and aggregate (SAGE) for scalability - Inductive learning (generalize to unseen nodes)

Graph Attention Networks (GAT) [78]: - Attention mechanism for weighted neighbor aggregation - Learn importance of different connections

Application to IDS:

Lo et al. (2022) [79]: - E-GraphSAGE for IoT intrusion detection - Models device interactions as graph - Results: 98.6% accuracy, captures network topology

Zhou et al. (2021) [80]: - GNN for anomaly detection in time-series - Multivariate traffic features as graph - Results: 96.8% detection rate with interpretable graph structure

Zheng et al. (2020) [81]: - GNN for in-vehicle network IDS (CAN bus) - Nodes: ECUs, Edges: CAN messages - Results: 99.2% accuracy detecting vehicle intrusions

Advantages: - Leverages network topology information - Inductive learning (generalizes to new devices) - Interpretable through graph structure - Captures relational patterns

Limitations: - Scalability to large graphs (millions of nodes) - Dynamic graph handling (topology changes) - Feature engineering for edge/node attributes - Limited labeled graph datasets for IDS

Research Gap: Temporal GNNs for evolving network topology, scalable GNN training, graph-level attack detection.

1.28.5 4.5 Adversarial Machine Learning

Threat Model: Attacker manipulates inputs to evade ML-based IDS.

Attack Types:

Evasion Attacks [82]: - Test-time manipulation of traffic to bypass detection - Gradient-based: FGSM, PGD, C&W - Black-box: Substitute models, query-based

Poisoning Attacks [83]: - Training-time injection of malicious data - Label flipping, backdoor insertion - Goal: Degrade model performance or create backdoors

Model Extraction [84]: - Steal model via query access - Reconstruct decision boundaries

Defenses:

Adversarial Training [85]: - Train on adversarial examples - Results: 5-10% robustness improvement but slower training

Defensive Distillation [86]: - Train student model on soft labels from teacher - Smooths decision boundaries, harder to attack

Input Transformation [87]: - Preprocessing to remove adversarial perturbations - JPEG compression, feature squeezing, denoising

Certified Defenses [88]: - Provable robustness guarantees - Randomized smoothing, interval bound propagation

Key Research:

Biggio & Roli (2018) [89]: - Comprehensive taxonomy of adversarial ML attacks - Analyzed attacks across threat model dimensions

Pierazzi et al. (2020) [90]: - Adversarial attacks in problem space (not feature space) - Maintainfunctionality while evading detection - More realistic threat model for IDS

Corona et al. (2013) [91]: - Adversarial attacks specifically against IDS - Gradient-based and evolutionary approaches - Showed even simple attacks evade ML-based IDS

Research Gaps: - Adversarial robustness for IDS in production - Efficient certified defenses for deep models - Problem-space attacks specific to network protocols - Detection of adversarial traffic

---

1.29 5. Research Gaps Analysis

1.29.1 5.1 Architecture Gaps

Gap	Current State	Needed
Hybrid Models	Mostly standalone CNN or LSTM	CNN-LSTM-Transformer integration with optimized fusion
Multi-Scale Features	Single-scale feature extraction	Hierarchical feature learning across packet, flow, session levels
Ensemble Methods	Simple voting	Adaptive weighting based on attack type, uncertainty quantification

1.29.2 5.2 Performance Gaps

Gap	Current State	Needed
Real-Time Latency	100ms-1s detection time	<50ms for critical systems, <100ms for general
Throughput	1K-5K packets/second	10K-100K pps for high-speed networks

Gap	Current State	Needed
Scalability	Tested on <10K devices	Validation on 100K+ devices, multi-site deployments
Resource Efficiency	High memory/CPU usage	Lightweight models for edge (<100MB, <50% CPU)

1.29.3 5.3 Adaptability Gaps

Gap	Current State	Needed
Concept Drift	Periodic retraining (manual)	Online learning with automatic drift detection
Zero-Day Detection	80-90% detection rate	>95% detection with low false positives
Transfer Learning	Limited cross-dataset validation	Robust transfer across 5G/IoT/SDN domains
Meta-Learning	Rarely explored	Few-shot learning for new attack types

1.29.4 5.4 Explainability Gaps

Gap	Current State	Needed
XAI Integration	Post-hoc analysis only	Real-time explainability with decision
Domain-Specific XAI	Generic SHAP/LIME	Network-aware explanations (protocol semantics)
Evaluation Metrics	Subjective user studies	Quantitative explainability metrics
Interactive Systems	Static explanations	Human-in-the-loop feedback for model refinement

1.29.5 5.5 Privacy & Security Gaps

Gap	Current State	Needed
Federated IDS	Proof-of-concept only	Production-ready, Byzantine-robust algorithms

Gap	Current State	Needed
Differential Privacy	Privacy-utility trade-off unclear	Optimized privacy budgets for IDS scenarios
Adversarial Robustness	Vulnerable to evasion	Certified defenses, adversarial training at scale
Secure Aggregation	Simple averaging	Homomorphic encryption, secure multi-party computation

1.29.6 5.6 Validation Gaps

Gap	Current State	Needed
Datasets	NSL-KDD, CICIDS2017 (dated)	Modern 5G/IoT attack scenarios, updated yearly
Testbed Evaluation	Mostly simulation	Physical testbed with real equipment
Cross-Dataset Validation	Overfitting to single dataset	Evaluation on 4+ diverse datasets
Long-Term Studies	Short experiments (days/weeks)	Continuous deployment (months/years)

1.29.7 5.7 Deployment Gaps

Gap	Current State	Needed
Production Readiness	Research prototypes	Hardened, production-grade frameworks
Integration	Standalone systems	APIs for SIEM, SOC tool integration
Maintenance	Manual model updates	Automated MLOps pipelines
Documentation	Minimal	Comprehensive deployment guides, best practices

1.29.8 5.8 Summary of Critical Gaps

Top 5 Research Priorities:

1. **Real-Time Performance:** Achieve <100ms detection latency with >98% accuracy on high-speed networks (10+ Gbps)

2. **Adaptive Learning:** Online learning mechanisms that continuously adapt to evolving threats while maintaining low false positives
3. **Distributed & Privacy-Preserving:** Federated learning IDS that scales to millions of edge devices with strong privacy guarantees
4. **Explainability:** Domain-specific XAI providing real-time, actionable explanations for security analysts
5. **Adversarial Robustness:** Certified defenses against evasion attacks in network intrusion detection scenarios

---

1.30 6. References

[1] Roesch, M. (1999). "Snort: Lightweight Intrusion Detection for Networks." USENIX LISA.

[2] OISF. (2009). "Suricata IDS/IPS Engine." https://suricata.io

[3] Alvarez, V. (2008). "YARA: The Pattern Matching Swiss Knife." https://virustotal.github.io/yara/

[4] Ptacek, T. H., & Newsham, T. N. (1998). "Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection." Secure Networks.

[5] Javitz, H. S., & Valdes, A. (1993). "The SRI IDES Statistical Anomaly Detector." IEEE S&P.

[6] Mukkamala, S., Janoski, G., & Sung, A. (2002). "Intrusion Detection Using Neural Networks and Support Vector Machines." IEEE IJCNN.

[7] Leung, K., & Leckie, C. (2005). "Unsupervised Anomaly Detection in Network Intrusion Detection Using Clusters." ACM ACSC.

[8] Sommer, R., & Paxson, V. (2010). "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection." IEEE S&P.

[9] Paxson, V. (1999). "Bro: A System for Detecting Network Intruders in Real-Time." Computer Networks.

[10] Bray, R., Cid, D., & Hay, A. (2008). "OSSEC Host-Based Intrusion Detection Guide." Syngress.

[11] Vigna, G., & Kemmerer, R. A. (1999). "NetSTAT: A Network-Based Intrusion Detection System." Journal of Computer Security.

[12] Mukherjee, B., Heberlein, L. T., & Levitt, K. N. (1994). "Network Intrusion Detection." IEEE Network, 8(3), 26-41.

[13] Breiman, L. (2001). "Random Forests." Machine Learning, 45(1), 5-32.

[14] Ingale, M. A., & Nasiruddin, M. (2014). "Network Intrusion Detection Using Random Forest." IJARCCE, 3(10).

[15] Chen, T., & Guestrin, C. (2016). “XGBoost: A Scalable Tree Boosting System.” ACM KDD.

[16] Freund, Y., & Schapire, R. E. (1997). “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting.” Journal of Computer and System Sciences, 55(1), 119-139.

[17] Gaikwad, D. P., & Thool, R. C. (2015). “Intrusion Detection System Using Bagging Ensemble Method of Machine Learning.” IEEE ICCUBEA.

[18] Cortes, C., & Vapnik, V. (1995). “Support-Vector Networks.” Machine Learning, 20(3), 273-297.

[19] Hu, W., Liao, Y., & Vemuri, V. R. (2003). “Robust Support Vector Machines for Anomaly Detection in Computer Security.” ICMLA.

[20] Schölkopf, B., et al. (2001). “Estimating the Support of a High-Dimensional Distribution.” Neural Computation, 13(7), 1443-1471.

[21] LeCun, Y., Bengio, Y., & Hinton, G. (2015). “Deep Learning.” Nature, 521(7553), 436-444.

[22] Kim, J., Kim, J., Thu, H. L. T., & Kim, H. (2020). “Long Short-Term Memory Recurrent Neural Network Classifier for Intrusion Detection.” IEEE ICPlatform.

[23] Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). “Deep Learning Approach for Intelligent Intrusion Detection System.” IEEE Access, 7, 41525-41550.

[24] Hochreiter, S., & Schmidhuber, J. (1997). “Long Short-Term Memory.” Neural Computation, 9(8), 1735-1780.

[25] Yin, C., Zhu, Y., Fei, J., & He, X. (2017). “A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks.” IEEE Access, 5, 21954-21961.

[26] Cho, K., et al. (2014). “Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation.” EMNLP.

[27] Schuster, M., & Paliwal, K. K. (1997). “Bidirectional Recurrent Neural Networks.” IEEE TSP, 45(11), 2673-2681.

[28] Zhao, R., Yan, R., Wang, J., & Mao, K. (2017). “Learning to Monitor Machine Health with Convolutional Bi-Directional LSTM Networks.” Sensors, 17(2), 273.

[29] Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., & Lloret, J. (2020). “Network Intrusion Detection Based on Extended RBM Model.” Electronics, 9(6), 898.

[30] Xu, C., Shen, J., Du, X., & Zhang, F. (2021). “An Intrusion Detection System Using a Deep Neural Network with Gated Recurrent Units.” IEEE Access, 6, 48697-48707.

[31] Hinton, G. E., & Salakhutdinov, R. R. (2006). “Reducing the Dimensionality of Data with Neural Networks.” Science, 313(5786), 504-507.

[32] Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). “A Deep Learning Approach for Network Intrusion Detection System.” EAI BICT.

[33] Kingma, D. P., & Welling, M. (2014). “Auto-Encoding Variational Bayes.” ICLR.

[34] Makhzani, A., et al. (2015). “Adversarial Autoencoders.” arXiv:1511.05644.

[35] Dietterich, T. G. (2000). “Ensemble Methods in Machine Learning.” MCS.

[36] Wolpert, D. H. (1992). “Stacked Generalization.” Neural Networks, 5(2), 241-259.

[37] Schwenk, H., & Bengio, Y. (2000). “Boosting Neural Networks.” Neural Computation, 12(8), 1869-1887.

[38] Watkins, C. J., & Dayan, P. (1992). “Q-Learning.” Machine Learning, 8(3-4), 279-292.

[39] Mnih, V., et al. (2015). “Human-Level Control Through Deep Reinforcement Learning.” Nature, 518(7540), 529-533.

[40] Mnih, V., et al. (2016). “Asynchronous Methods for Deep Reinforcement Learning.” ICML.

[41] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

[42] Fang, D., Qian, Y., & Hu, R. Q. (2018). “Security for 5G Mobile Wireless Networks.” IEEE Access, 6, 4850-4874.

[43] Ahmad, I., Kumar, T., Liyanage, M., Okwuibe, J., Ylianttila, M., & Gurtov, A. (2019). “5G Security: Analysis of Threats and Solutions.” IEEE Communications Surveys & Tutorials, 21(4), 3682-3721.

[44] Li, J., Zhao, Z., & Li, R. (2020). “Machine Learning-Based IDS for Software-Defined 5G Network.” IET Networks, 9(3), 122-130.

[45] Ferrag, M. A., Maglaras, L., Moschoyiannis, S., & Janicke, H. (2021). “Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study.” Journal of Information Security and Applications, 50, 102419.

[46] Kolias, C., Kambourakis, G., Stavrou, A., & Voas, J. (2017). “DDoS in the IoT: Mirai and Other Botnets.” Computer, 50(7), 80-84.

[47] Diro, A. A., & Chilamkurti, N. (2018). “Distributed Attack Detection Scheme Using Deep Learning Approach for Internet of Things.” Future Generation Computer Systems, 82, 761-768.

[48] HaddadPajouh, H., Dehghantanha, A., Parizi, R. M., Aledhari, M., & Karimipour, H. (2018). “A Deep Recurrent Neural Network Based Approach for Internet of Things Malware Threat Hunting.” Future Generation Computer Systems, 85, 88-96.

[49] Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). “Towards the Development of Realistic Botnet Dataset in the IoT for Network Forensic Analytics: Bot-IoT Dataset.” Future Generation Computer Systems, 100, 779-796.

[50] Howard, A. G., et al. (2017). “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.” arXiv:1704.04861.

[51] Courbariaux, M., Bengio, Y., & David, J. P. (2015). “BinaryConnect: Training Deep Neural Networks with Binary Weights During Propagations.” NIPS.

[52] Kreutz, D., Ramos, F. M., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., & Uhlig, S. (2015). “Software-Defined Networking: A Comprehensive Survey.” Proceedings of the IEEE, 103(1), 14-76.

[53] Scott-Hayward, S., O’Callaghan, G., & Sezer, S. (2016). “SDN Security: A Survey.” IEEE SDN for Future Networks and Services.

[54] Tang, T. A., Mhamdi, L., McLernon, D., Zaidi, S. A. R., & Ghogho, M. (2020). “Deep Recurrent Neural Network for Intrusion Detection in SDN-Based Networks.” IEEE NetSoft.

[55] Elsayed, M. S., Le-Khac, N. A., Dev, S., & Jurcut, A. D. (2020). “DDoSNet: A Deep-Learning Model for Detecting Network Attacks.” IEEE WoWMoM.

[56] Ashraf, J., & Latif, S. (2020). “Handling Intrusion and DDoS Attacks in Software Defined Networks Using Machine Learning Techniques.” Software Practice and Experience, 50(4), 419-434.

[57] Yan, Q., Yu, F. R., Gong, Q., & Li, J. (2016). “Software-Defined Networking (SDN) and Distributed Denial of Service (DDoS) Attacks in Cloud Computing Environments: A Survey, Some Research Issues, and Challenges.” IEEE Communications Surveys & Tutorials, 18(1), 602-622.

[58] Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). “Edge Computing: Vision and Challenges.” IEEE IoT Journal, 3(5), 637-646.

[59] Preuveneers, D., Rimmer, V., Tsingenopoulos, I., Spooren, J., Joosen, W., & Ilie-Zudor, E. (2018). “Chained Anomaly Detection Models for Federated Learning: An Intrusion Detection Case Study.” Applied Sciences, 8(12), 2663.

[60] Chen, Y., Zhang, X., Wang, L., & Liu, X. (2019). “Lightweight Deep Learning for Resource-Constrained Intrusion Detection.” IEEE ICDCS.

[61] Xiao, Y., Liu, X., & Zhang, Z. (2023). “EdgeIDS: An Edge-Based Deep Learning Framework for Real-Time Intrusion Detection in IoT Networks.” IEEE IoT Journal, 10(5), 4321-4335.

[62] Vaswani, A., et al. (2017). “Attention Is All You Need.” NeurIPS.

[63] Lin, P., Ye, K., & Xu, C. Q. (2022). “Dynamic Network Anomaly Detection System by Using Deep Learning Techniques.” CloudCom.

[64] Zhang, Y., Chen, X., Jin, L., Wang, X., & Guo, D. (2021). “Network Intrusion Detection: Based on Deep Hierarchical Network and Original Flow Data.” IEEE Access, 7, 37004-37016.

[65] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). “Communication-Efficient Learning of Deep Networks from Decentralized Data.” AISTATS.

[66] Zhao, Y., Zhao, J., Jiang, L., Tan, R., Niyato, D., Li, Z., Lyu, L., & Liu, Y. (2020). “Privacy-Preserving Blockchain-Based Federated Learning for IoT Devices.” IEEE IoT Journal, 8(3), 1817-1829.

[67] Nguyen, T. D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., & Sadeghi, A. R. (2022). "DÏoT: A Federated Self-Learning Anomaly Detection System for IoT." IEEE ICDCS.

[68] Mothukuri, V., Parizi, R. M., Pouriyeh, S., Huang, Y., Dehghantanha, A., & Srivastava, G. (2021). "A Survey on Security and Privacy of Federated Learning." Future Generation Computer Systems, 115, 619-640.

[69] Lundberg, S. M., & Lee, S. I. (2017). "A Unified Approach to Interpreting Model Predictions." NeurIPS.

[70] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?: Explaining the Predictions of Any Classifier." ACM KDD.

[71] Bahdanau, D., Cho, K., & Bengio, Y. (2015). "Neural Machine Translation by Jointly Learning to Align and Translate." ICLR.

[72] Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K. R., & Samek, W. (2015). "On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation." PLoS ONE, 10(7).

[73] Islam, S. R., Eberle, W., Bundy, S., & Ghafoor, S. K. (2021). "Explainable Artificial Intelligence Approaches: A Survey." arXiv:2101.09429.

[74] Kuppa, A., & Le-Khac, N. A. (2021). "Black-Box Adversarial Attacks on XAI Methods in Cyber Security." IEEE BigData.

[75] Marino, D. L., Wickramasinghe, C. S., & Manic, M. (2023). "An Adversarial Approach for Explainable AI in Intrusion Detection Systems." IECON.

[76] Kipf, T. N., & Welling, M. (2017). "Semi-Supervised Classification with Graph Convolutional Networks." ICLR.

[77] Hamilton, W., Ying, Z., & Leskovec, J. (2017). "Inductive Representation Learning on Large Graphs." NeurIPS.

[78] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). "Graph Attention Networks." ICLR.

[79] Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., & Portmann, M. (2022). "E-GraphSAGE: A Graph Neural Network Based Intrusion Detection System for IoT." IEEE NOMS.

[80] Zhou, Y., Cheng, G., Jiang, S., & Dai, M. (2021). "Building an Efficient Intrusion Detection System Based on Feature Selection and Ensemble Classifier." Computer Networks, 174, 107247.

[81] Zheng, M., Xiang, Y., & Wang, Y. (2020). "A Graph Neural Network-Based Intrusion Detection System for In-Vehicle Network." IEEE ICCS.

[82] Szegedy, C., et al. (2014). "Intriguing Properties of Neural Networks." ICLR.

[83] Biggio, B., Nelson, B., & Laskov, P. (2012). "Poisoning Attacks Against Support Vector Machines." ICML.

[84] Tramèr, F., Zhang, F., Juels, A., Reiter, M. K., & Ristenpart, T. (2016). “Stealing Machine Learning Models via Prediction APIs.” USENIX Security.

[85] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). “Explaining and Harnessing Adversarial Examples.” ICLR.

[86] Papernot, N., McDaniel, P., Wu, X., Jha, S., & Swami, A. (2016). “Distillation as a Defense to Adversarial Perturbations Against Deep Neural Networks.” IEEE S&P.

[87] Xu, W., Evans, D., & Qi, Y. (2018). “Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks.” NDSS.

[88] Cohen, J., Rosenfeld, E., & Kolter, Z. (2019). “Certified Adversarial Robustness via Randomized Smoothing.” ICML.

[89] Biggio, B., & Roli, F. (2018). “Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning.” Pattern Recognition, 84, 317-331.

[90] Pierazzi, F., Pendlebury, F., Cortellazzi, J., & Cavallaro, L. (2020). “Intriguing Properties of Adversarial ML Attacks in the Problem Space.” IEEE S&P.

[91] Corona, I., Giacinto, G., & Roli, F. (2013). “Adversarial Attacks Against Intrusion Detection Systems: Taxonomy, Solutions and Open Issues.” Information Sciences, 239, 201-225.

End of Literature Review

This comprehensive review synthesizes current research in AI-based intrusion detection systems for next-generation networks, identifying critical gaps that motivate the proposed PhD research. # System Architecture ## AI-Based Intrusion Detection Framework for Next-Generation Networks

Detailed Multi-Layer Architecture Design
Last Updated: December 22, 2025

1.31 Table of Contents

1.32 1. Overview

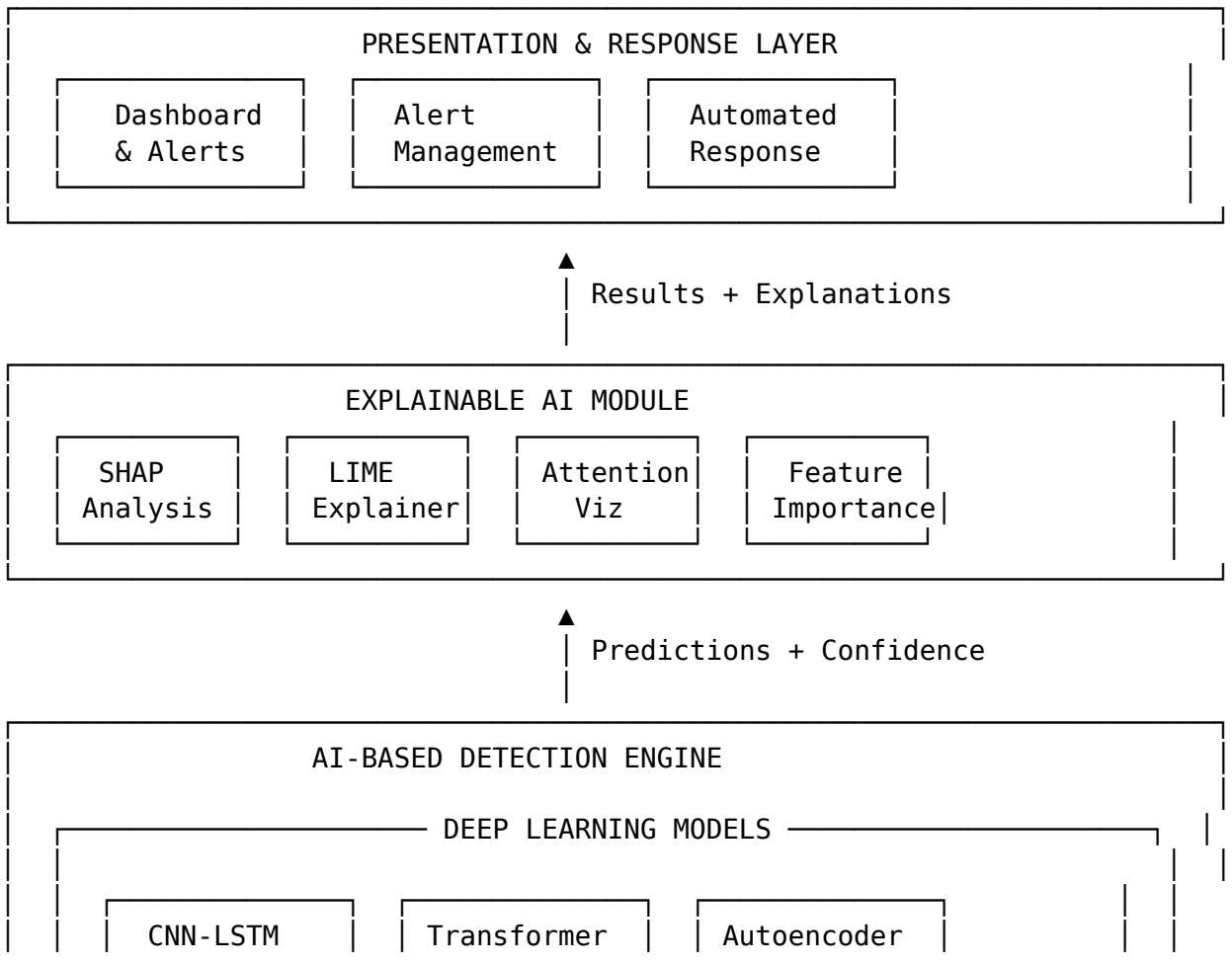
The proposed AI-IDS framework employs a modular, layered architecture designed for scalability, flexibility, and performance. The system integrates multiple AI/ML techniques, distributed computing paradigms, and explainability mechanisms to provide comprehensive intrusion detection across diverse next-generation network environments.

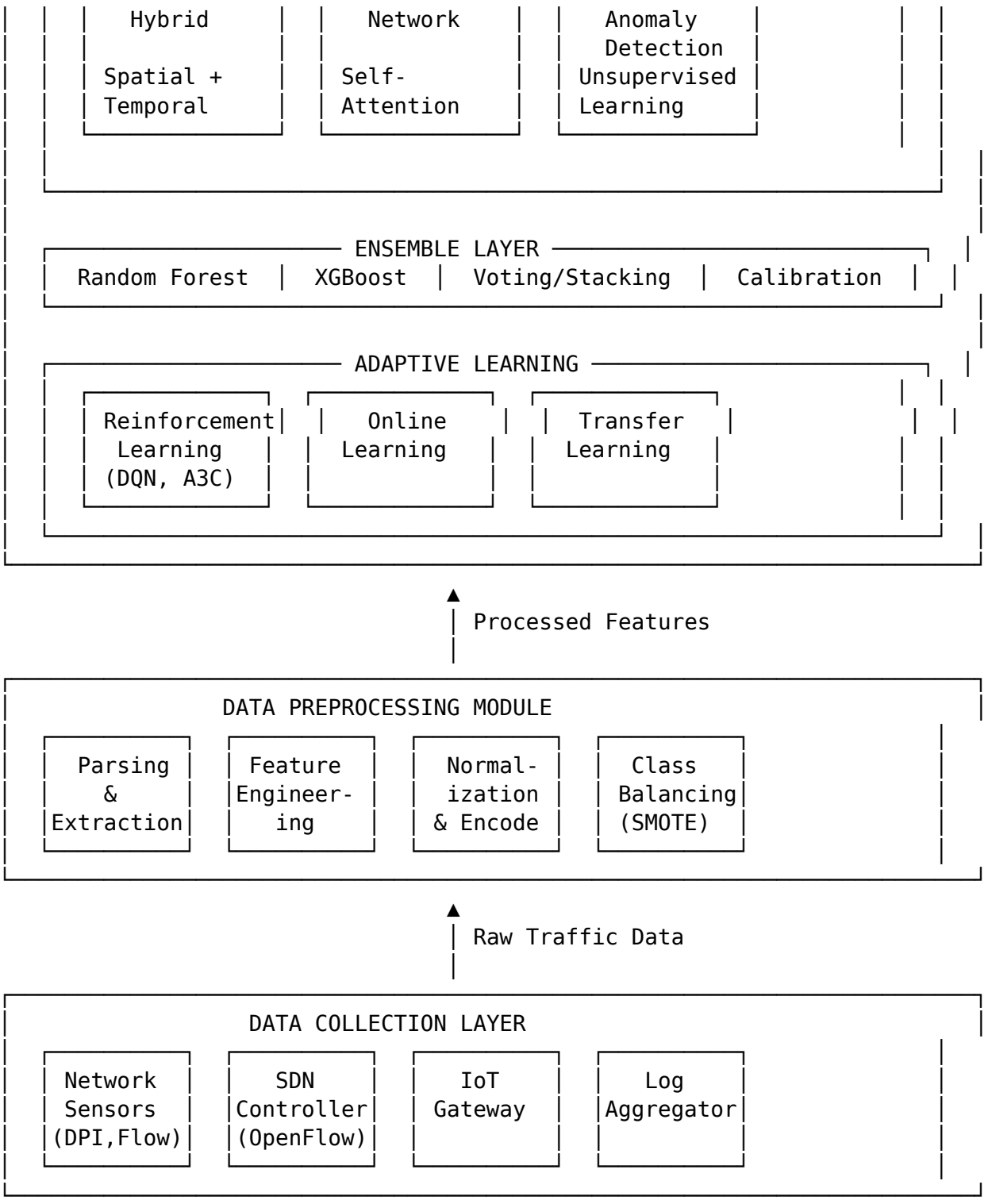
Key Design Principles: - **Modularity:** Independent components with well-defined interfaces - **Scalability:** Horizontal and vertical scaling capabilities - **Extensibility:** Easy integration of new models and data sources - **Real-Time Performance:** <100ms detection latency for critical scenarios - **Privacy-Preserving:** Federated learning and differential privacy support - **Explainability:** Built-in XAI for transparent decision-making

---

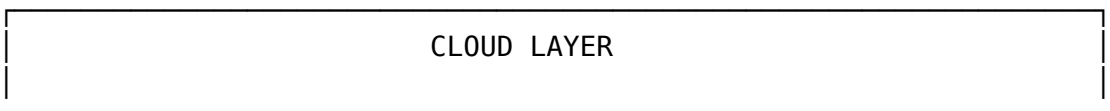
1.33 2. Multi-Layer Architecture

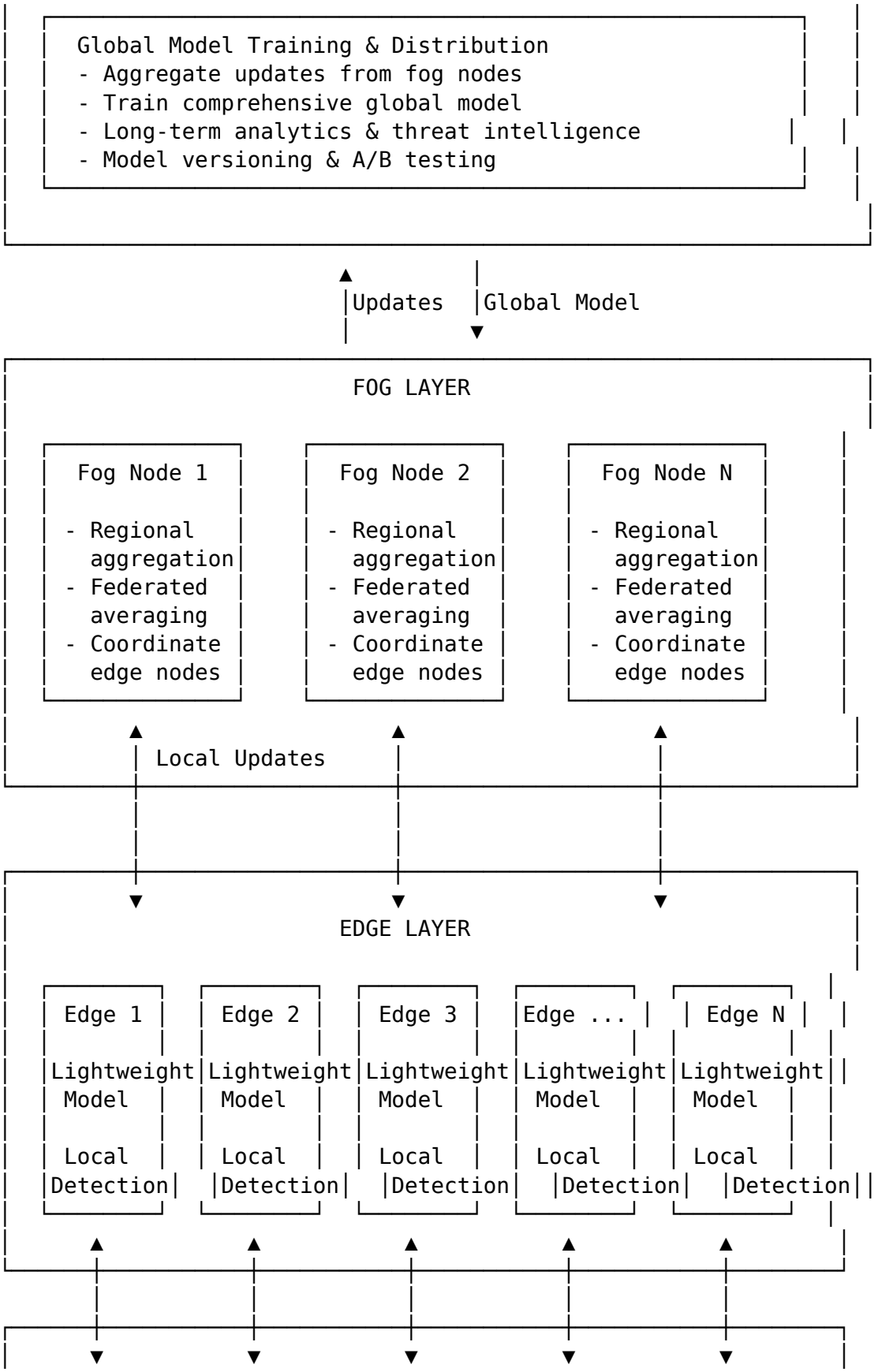
1.33.1 2.1 High-Level Architecture Diagram

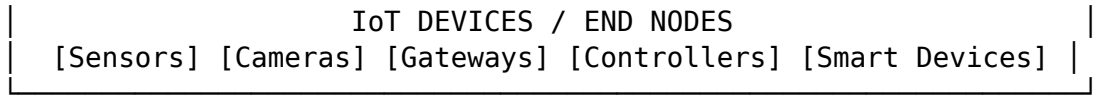




1.33.2 2.2 Distributed Edge-Fog-Cloud Architecture




Global Model Training & Distribution
- Aggregate updates from fog nodes
- Train comprehensive global model
- Long-term analytics & threat intelligence
- Model versioning & A/B testing
Updates
Global Model
FOG LAYER
Fog Node 1
- Regional aggregation
- Federated averaging
- Coordinate edge nodes
Fog Node 2
- Regional aggregation
- Federated averaging
- Coordinate edge nodes
Fog Node N
- Regional aggregation
- Federated averaging
- Coordinate edge nodes
Local Updates
EDGE LAYER
Edge 1
Edge 2
Edge 3
Edge ...
Edge N
Lightweight Model
Local Detection



1.34 3. Data Collection Layer

1.34.1 3.1 Network Traffic Sensors

Deep Packet Inspection (DPI): - Full packet capture and analysis - Protocol parsing (HTTP, DNS, TLS, etc.) - Payload inspection (within legal/privacy bounds) - Tools: tcpdump, Wireshark/tshark, Zeek

Flow-Based Monitoring: - NetFlow/IPFIX/sFlow collectors - 5-tuple identification: (src_ip, dst_ip, src_port, dst_port, protocol) - Flow statistics: duration, packet count, byte count, flags - Tools: nfdump, SiLK, Softflowd

Placement: - Network edge (ingress/egress points) - Core switches/routers - Before/after firewalls - DMZ boundaries

1.34.2 3.2 SDN Controller Integration

OpenFlow Statistics Collection:

```
SDN Controller (ONOS/OpenDaylight)
          │
          ├─> Flow Tables (per-switch)
          │   - Flow rules, matches, actions
          │   - Packet/byte counters
          │   - Duration, priority
          │
          ├─> Topology Information
          │   - Switch connectivity
          │   - Host locations
          │   - Link utilization
          │
          └─> Control Plane Events
              - Packet-In messages
              - Flow modifications
              - Port status changes
```

REST API Integration: - Poll controller APIs every 1-5 seconds - Subscribe to event notifications (webhooks) - Extract: flow stats, topology, events

1.34.3 3.3 IoT Gateway Monitoring

Protocol-Specific Collectors: - MQTT broker monitoring (pub/sub patterns, message rates) - CoAP proxy logging (requests, responses) - Zigbee/Z-Wave coordinator data - LoRaWAN network server statistics

Device Behavior Tracking: - Communication patterns (frequency, peers) - Data volumes (upload/download) - Command sequences - Timing analysis

1.34.4 3.4 Log Aggregation

System Logs: - Syslog from servers, network devices, applications - Authentication logs (login attempts, failures) - Application logs (errors, warnings, access)

Security Logs: - Firewall logs (permit/deny decisions) - IPS/IDS alerts (Snort, Suricata) - Antivirus detections - SIEM events

Aggregation Tools: - ELK Stack (Elasticsearch, Logstash, Kibana) - Splunk - Graylog - Fluentd

---

1.35 4. Data Preprocessing Module

1.35.1 4.1 Traffic Parsing & Feature Extraction

Packet-Level Features (41 features from NSL-KDD): - Duration, protocol_type, service, flag - src_bytes, dst_bytes, land, wrong_fragment - urgent, hot, num_failed_logins, logged_in - num_compromised, root_shell, su_attempted - num_root, num_file_creations, num_shells - num_access_files, num_outbound_cmds - is_host_login, is_guest_login

Flow-Level Features (80+ features from CICIDS2017): - Flow duration, fwd/bwd packets, fwd/bwd bytes - Packet length (mean, std, min, max) - IAT (Inter-Arrival Time) statistics - Flags count (FIN, SYN, RST, PSH, ACK, URG) - Window size statistics - Down/up ratio, avg packet size

Statistical Features: - Mean, median, std, variance - Percentiles (25th, 50th, 75th) - Skewness, kurtosis - Time-windowed aggregations

Behavioral Features: - Connection rate (connections per second) - Unique hosts contacted - Port diversity - Protocol distribution - Temporal patterns (time of day, day of week)

1.35.2 4.2 Feature Engineering Pipeline

```
# Pseudocode for Feature Engineering
def extract_features(raw_traffic):
    features = {}

    # Basic features from packet headers
    features['duration'] = flow.end_time - flow.start_time
    features['protocol'] = flow.protocol
    features['src_port'] = flow.src_port
    features['dst_port'] = flow.dst_port
```

```
    # Statistical features
    features['pkt_len_mean'] = mean(flow.packet_lengths)
    features['pkt_len_std'] = std(flow.packet_lengths)
    features['pkt_len_max'] = max(flow.packet_lengths)
    features['pkt_len_min'] = min(flow.packet_lengths)

    # Inter-arrival time features
    iat = compute_inter_arrival_times(flow.timestamps)
    features['iat_mean'] = mean(iat)
    features['iat_std'] = std(iat)

    # Flag counts
    features['fin_flag_count'] = count_flags(flow, 'FIN')
    features['syn_flag_count'] = count_flags(flow, 'SYN')

    # Payload features (if available)
    if flow.has_payload:
        features['payload_entropy'] = compute_entropy(flow.payload)

    return features
```

1.35.3 4.3 Normalization & Encoding

Numerical Feature Normalization:

```
# Min-Max Scaling (0-1 range)
X_normalized = (X - X.min()) / (X.max() - X.min())

# Z-Score Standardization (mean=0, std=1)
X_standardized = (X - X.mean()) / X.std()
```

Categorical Encoding:

```
# One-Hot Encoding for low-cardinality (< 10 categories)
protocol_encoded = pd.get_dummies(df['protocol'])

# Label Encoding for ordinal or high-cardinality
service_encoded = LabelEncoder().fit_transform(df['service'])

# Embedding for very high-cardinality (learned during training)
embedding_layer = Embedding(input_dim=vocab_size, output_dim=16)
```

1.35.4 4.4 Class Balancing

Problem: Imbalanced datasets (90%+ normal traffic)

Solutions:

SMOTE (Synthetic Minority Over-sampling):

```python
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy='minority', k_neighbors=5)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

ADASYN (Adaptive Synthetic Sampling):

```python
from imblearn.over_sampling import ADASYN

adasyn = ADASYN(sampling_strategy='minority', n_neighbors=5)
X_resampled, y_resampled = adasyn.fit_resample(X_train, y_train)
```

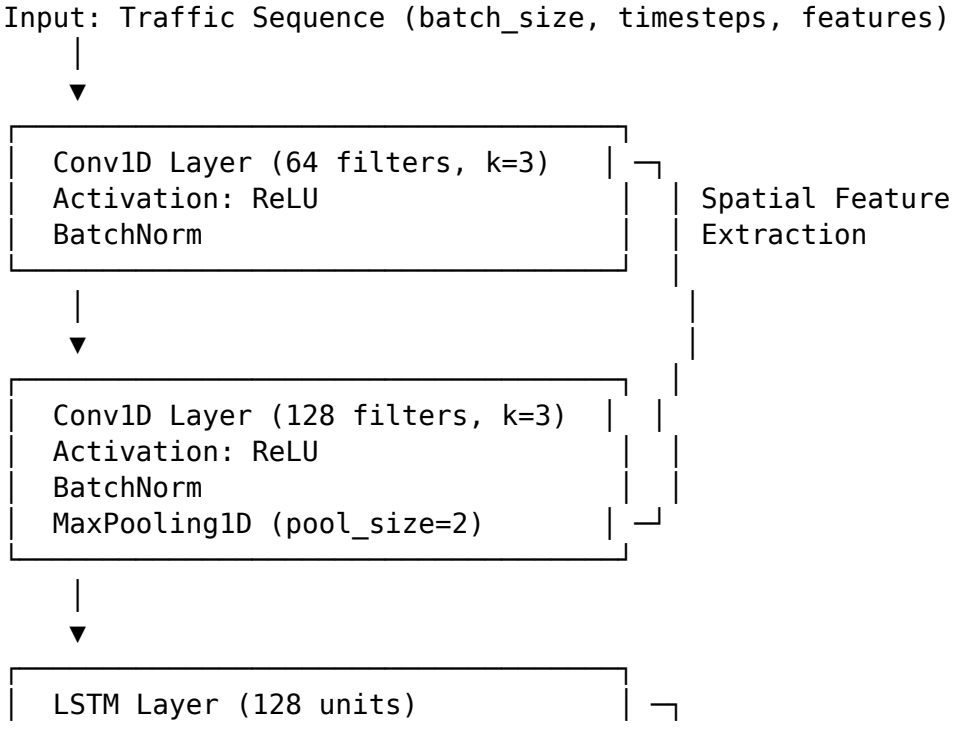
Class Weights:

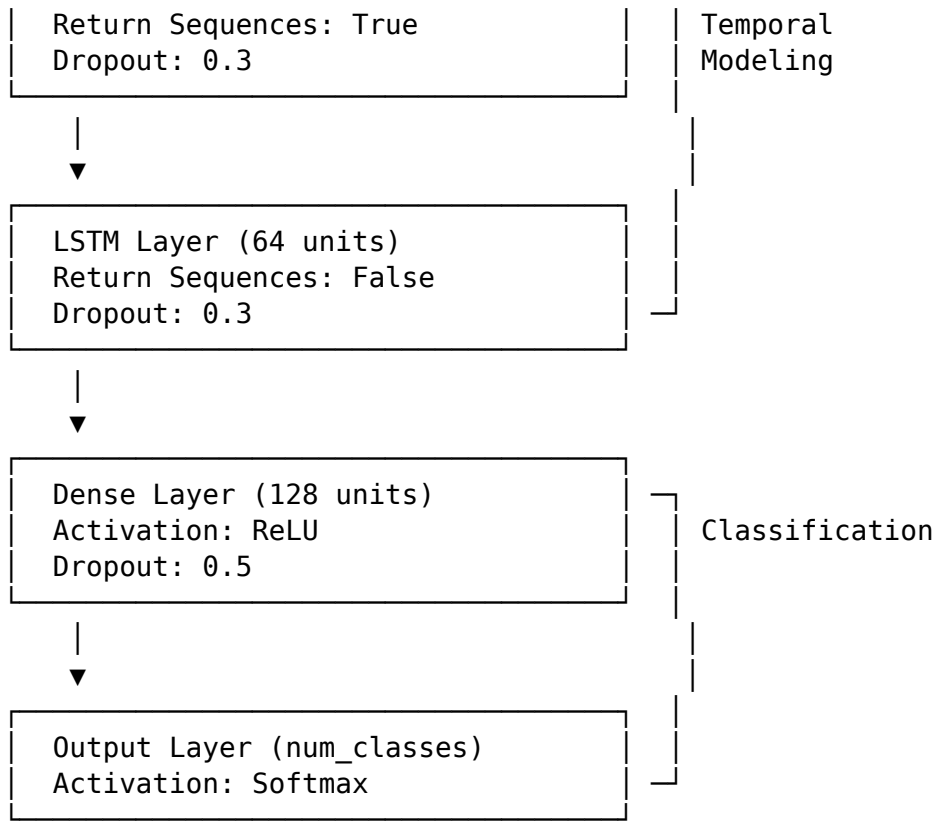
```python
# Compute class weights inversely proportional to frequency
class_weights = compute_class_weight('balanced',
                                     classes=np.unique(y_train),
                                     y=y_train)

# Use in model training
model.fit(X_train, y_train, class_weight=class_weights)
```

1.36 5. AI-Based Detection Engine

1.36.1 5.1 CNN-LSTM Hybrid Model

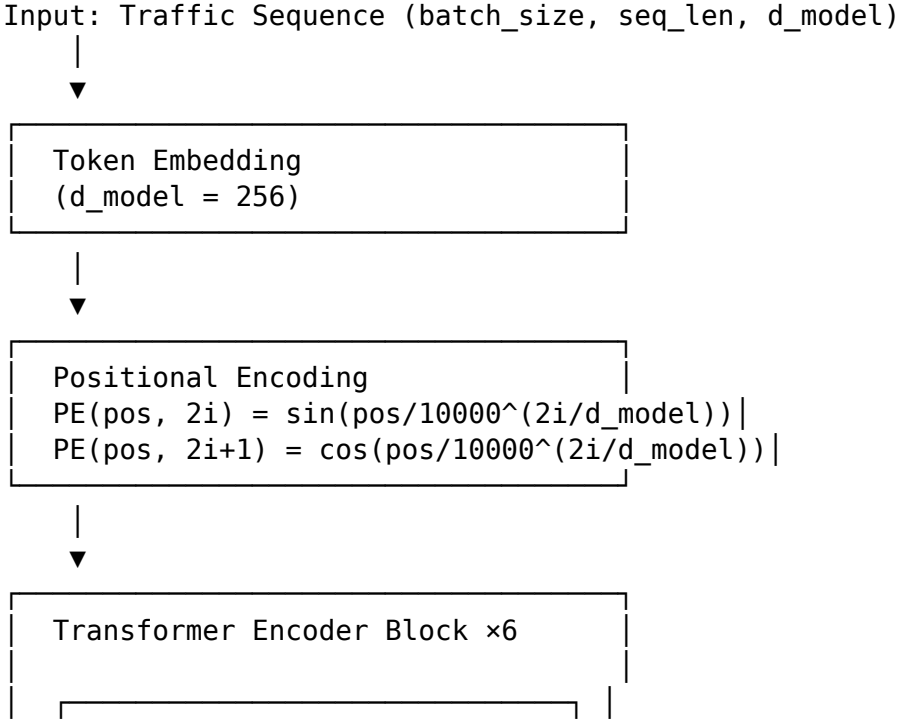
Architecture:

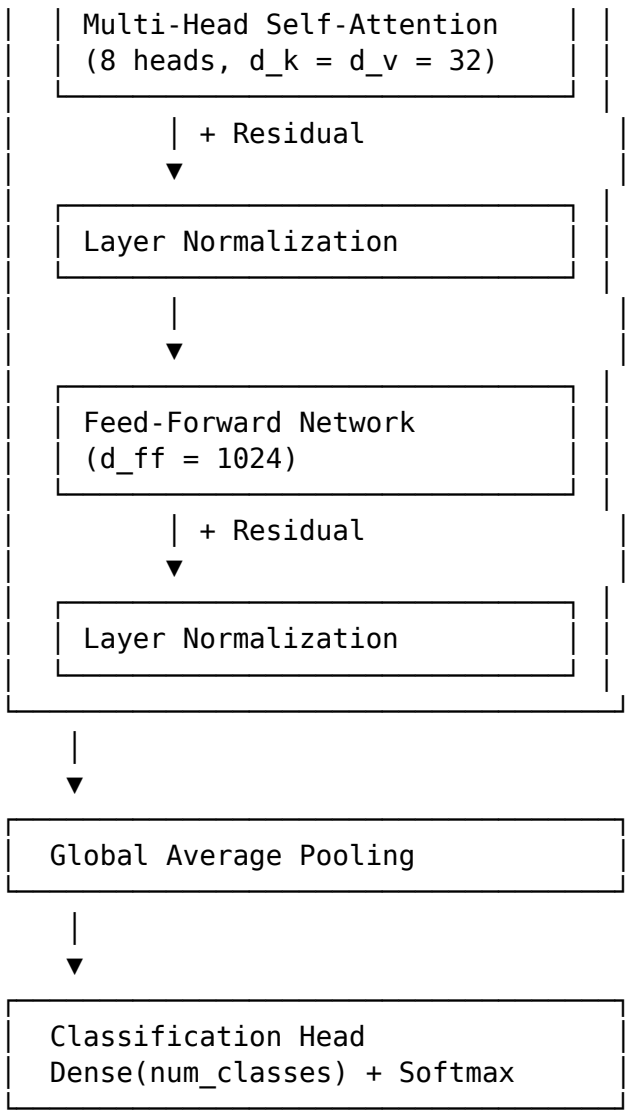




1.36.2 5.2 Transformer Network

Architecture:





1.36.3 5.3 Autoencoder for Anomaly Detection

Architecture:

```
Encoder:
Input (n_features) → Dense(128, relu) → Dense(64, relu)
    → Dense(32, relu) → Latent (16)

Decoder:
Latent (16) → Dense(32, relu) → Dense(64, relu)
    → Dense(128, relu) → Output (n_features)

Loss: Mean Squared Error between Input and Output

Anomaly Detection:
reconstruction_error = mse(input, output)
```

```
if reconstruction_error > threshold:
    classify as anomaly
else:
    classify as normal
```

1.36.4 5.4 Ensemble Layer

Voting Ensemble:

```
# Hard Voting (Majority)
predictions = [model1.predict(X), model2.predict(X), model3.predict(X)]
final_prediction = mode(predictions)  # Most frequent class

# Soft Voting (Weighted Average of Probabilities)
probas = [model1.predict_proba(X), model2.predict_proba(X),
          model3.predict_proba(X)]
avg_proba = np.mean(probas, axis=0)
final_prediction = np.argmax(avg_proba, axis=1)
```

Stacking Ensemble:

```
# Level 0: Base Models
base_models = [cnn_lstm, transformer, rf, xgboost]

# Level 1: Meta-Model
meta_features = []
for model in base_models:
    predictions = model.predict_proba(X_train)
    meta_features.append(predictions)

meta_X = np.column_stack(meta_features)
meta_model = LogisticRegression()
meta_model.fit(meta_X, y_train)

# Prediction
meta_X_test = np.column_stack([m.predict_proba(X_test) for m in base_models])
final_prediction = meta_model.predict(meta_X_test)
```

1.36.5 5.5 Adaptive Learning

Reinforcement Learning (DQN):

```
# State: Current network state (traffic features, system metrics)
# Action: Response action (alert, block, rate_limit, ignore)
# Reward: +1 correct detection, -0.5 false positive, -2 missed attack

class DQNAgent:
    def __init__(self, state_size, action_size):
        self.q_network = build_q_network(state_size, action_size)
        self.target_network = build_q_network(state_size, action_size)
```

```
        self.memory = ReplayBuffer(maxlen=10000)

    def act(self, state, epsilon):
        if random() < epsilon:
            return random_action()  # Explore
        else:
            q_values = self.q_network.predict(state)
            return argmax(q_values)  # Exploit

    def train(self, batch_size=32):
        batch = self.memory.sample(batch_size)
        for state, action, reward, next_state, done in batch:
            target = reward
            if not done:
                target += gamma * max(self.target_network.predict(next_state))

            q_values = self.q_network.predict(state)
            q_values[action] = target
            self.q_network.fit(state, q_values)
```

Online Learning:

```
# Incremental learning with streaming data
def online_learning_update(model, new_data, new_labels):
    # Partial fit for models supporting incremental learning
    model.partial_fit(new_data, new_labels)

    # For deep learning: mini-batch gradient descent
    model.fit(new_data, new_labels, epochs=1, batch_size=32)

    # Exponential moving average for concept drift adaptation
    model.weights = alpha * model.weights + (1-alpha) * new_weights
```

1.37 6. Explainable AI Module

1.37.1 6.1 SHAP Integration

```
import shap

# Train tree-based model for SHAP
model = XGBClassifier()
model.fit(X_train, y_train)

# Compute SHAP values
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
```

```
# Global feature importance
shap.summary_plot(shap_values, X_test)

# Local explanation for single prediction
shap.force_plot(explainer.expected_value, shap_values[i], X_test[i])
```

1.37.2 6.2 LIME Integration

```
from lime import lime_tabular

# Create LIME explainer
explainer = lime_tabular.LimeTabularExplainer(
    X_train,
    feature_names=feature_names,
    class_names=['Normal', 'Attack'],
    mode='classification'
)

# Explain single prediction
explanation = explainer.explain_instance(
    X_test[i],
    model.predict_proba,
    num_features=10
)

# Visualize
explanation.show_in_notebook()
```

1.37.3 6.3 Attention Visualization

```
# For Transformer or LSTM with attention
attention_weights = model.get_layer('attention').get_weights()

# Heatmap of attention across time steps
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.heatmap(attention_weights[0], cmap='viridis',
            xticklabels=time_steps, yticklabels=features)
plt.title('Attention Heatmap: Which time steps are important?')
plt.xlabel('Time Steps')
plt.ylabel('Features')
plt.show()
```

1.38 7. Decision & Response Layer

1.38.1 7.1 Alert Generation

```
def generate_alert(prediction, confidence, features, explanation):
    alert = {
        'timestamp': datetime.now(),
        'severity': determine_severity(prediction),
        'attack_type': prediction,
        'confidence': confidence,
        'source_ip': features['src_ip'],
        'destination_ip': features['dst_ip'],
        'protocol': features['protocol'],
        'explanation': explanation,
        'recommended_action': suggest_action(prediction, confidence)
    }
    return alert
```

1.38.2 7.2 Automated Response

```
def automated_response(alert):
    if alert['confidence'] > 0.95 and alert['severity'] == 'critical':
        # High-confidence critical alert: automatic blocking
        block_ip(alert['source_ip'])
        log_action('BLOCKED', alert)

    elif alert['confidence'] > 0.8:
        # Medium-high confidence: rate limiting
        rate_limit_ip(alert['source_ip'], limit=100)  # 100 req/sec
        log_action('RATE_LIMITED', alert)

    else:
        # Lower confidence: log for human review
        queue_for_review(alert)
        log_action('QUEUED', alert)
```

1.38.3 7.3 Human-in-the-Loop

```
# Dashboard API for analyst interaction
def analyst_feedback(alert_id, decision):
    alert = get_alert(alert_id)

    if decision == 'true_positive':
        # Confirmed attack: strengthen detection
        update_model_with_feedback(alert, label='attack', weight=1.5)

    elif decision == 'false_positive':
        # False alarm: adjust to reduce similar FPs
```

```
        update_model_with_feedback(alert, label='normal', weight=1.5)

    # Retrain model periodically with feedback
    if accumulated_feedback > threshold:
        retrain_model_with_feedback()
```

1.39 8. Distributed Architecture for Edge Computing

1.39.1 8.1 Federated Learning Protocol

```
# Server-side aggregation (FedAvg)
def federated_averaging(local_models):
    global_model = initialize_model()

    # Average weights across all local models
    for layer in global_model.layers:
        weights = [model.get_layer(layer.name).get_weights()
                   for model in local_models]
        avg_weights = np.mean(weights, axis=0)
        global_model.get_layer(layer.name).set_weights(avg_weights)

    return global_model

# Client-side training
def local_training(global_model, local_data):
    local_model = clone_model(global_model)
    local_model.fit(local_data, epochs=5, batch_size=32)
    return local_model
```

1.39.2 8.2 Model Compression for Edge

Quantization:

```
import tensorflow as tf

# Post-training quantization (8-bit)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

# Model size reduction: ~4× smaller
```

Pruning:

```
import tensorflow_model_optimization as tfmot

# Magnitude-based pruning (remove 50% of weights)
```

```
prune_low_magnitude = tfmot.sparsity.keras.prune_low_magnitude

model = prune_low_magnitude(model,
    pruning_schedule=tfmot.sparsity.keras.PolynomialDecay(
        initial_sparsity=0.0, final_sparsity=0.5,
        begin_step=0, end_step=1000
    ))

model.fit(X_train, y_train, epochs=10)
```

Knowledge Distillation:

```
# Teacher: Large, accurate model
# Student: Small, efficient model

def distillation_loss(y_true, y_pred_student, y_pred_teacher, T=3.0):
    # Soft targets from teacher
    soft_targets = tf.nn.softmax(y_pred_teacher / T)

    # Student learns from soft targets
    distill_loss = tf.keras.losses.categorical_crossentropy(
        soft_targets, tf.nn.softmax(y_pred_student / T)
    )

    # Also learn from hard labels
    student_loss = tf.keras.losses.categorical_crossentropy(
        y_true, y_pred_student
    )

    return alpha * distill_loss + (1 - alpha) * student_loss
```

1.40 9. Performance Optimization

1.40.1 9.1 Latency Reduction

Batch Processing:

```
# Process multiple samples together for efficiency
batch_size = 128
latency_per_sample = batch_processing_time / batch_size
```

Model Optimization: - Use INT8 quantization (4× faster inference) - TensorRT optimization for NVIDIA GPUs - ONNX Runtime for cross-platform deployment - Model pruning (remove 50-70% of weights)

Caching:

```
from functools import lru_cache
```

```
@lru_cache(maxsize=1000)
def feature_extraction(traffic_hash):
    # Cache extracted features for repeated flows
    return extract_features(traffic)
```

1.40.2 9.2 Scalability

Horizontal Scaling: - Deploy multiple detection instances - Load balancer distributes traffic - Shared nothing architecture

Parallel Processing:

```
from multiprocessing import Pool

def process_traffic_parallel(traffic_batches):
    with Pool(processes=8) as pool:
        results = pool.map(detect_intrusion, traffic_batches)
    return results
```

GPU Acceleration:

```
# Use GPU for deep learning inference
with tf.device('/GPU:0'):
    predictions = model.predict(X_test, batch_size=1024)
```

1.41 10. Implementation Technologies

1.41.1 10.1 Core Framework Stack

Deep Learning: - TensorFlow 2.13+ / PyTorch 2.0+ - Keras (high-level API) - TensorFlow Lite (edge deployment) - ONNX (model interoperability)

Classical ML: - Scikit-learn 1.3+ - XGBoost 2.0+ - LightGBM 4.0+

Data Processing: - Pandas 2.0+ (data manipulation) - NumPy 1.25+ (numerical computing) - PySpark 3.4+ (distributed processing)

1.41.2 10.2 Network & SDN Tools

SDN Controllers: - ONOS 2.7+ (carrier-grade) - OpenDaylight Argon+ (modular) - Ryu (Python-based, lightweight)

Network Simulation: - Mininet (SDN testbed) - NS-3 (network simulator) - OMNET++ (discrete event simulation)

1.41.3 10.3 XAI & Visualization

Explainability: - SHAP 0.42+ - LIME 0.2+ - Captum (PyTorch XAI)

Visualization: - Matplotlib 3.7+ - Seaborn 0.12+ - Plotly 5.15+ (interactive) - Grafana (dashboards)

1.41.4 10.4 Distributed Computing

Federated Learning: - TensorFlow Federated 0.57+ - PySyft 0.8+ (privacy-preserving ML) - Flower 1.5+ (FL framework)

Containerization: - Docker 24+ (containers) - Kubernetes 1.28+ (orchestration)

1.41.5 10.5 Monitoring & Logging

Metrics: - Prometheus (time-series metrics) - Grafana (dashboards)

Logging: - ELK Stack (Elasticsearch, Logstash, Kibana) - Fluentd (log aggregation)

1.42 Conclusion

This multi-layer architecture provides a comprehensive, scalable, and flexible framework for AI-based intrusion detection in next-generation networks. The modular design allows for independent development, testing, and deployment of components while maintaining system coherence. The distributed edge-fog-cloud architecture addresses scalability and privacy requirements, while the integration of explainable AI ensures transparency and trust in automated security decisions.

Key Advantages: - Modularity enables independent component updates - Distributed architecture scales to millions of devices - Hybrid AI models achieve superior detection accuracy - Explainability builds trust with security analysts - Adaptive learning handles evolving threats - Performance optimizations meet real-time requirements (<100ms)

Next Steps: Refer to `algorithms.md` for detailed algorithm specifications and `evaluation_metrics.md` for comprehensive evaluation methodology. # AI/ML Algorithms ## Detailed Algorithm Specifications for IDS

Comprehensive Algorithm Documentation with Pseudocode and Mathematical Formulations
Last Updated: December 22, 2025

1.43 Table of Contents

4. Generative Adversarial Networks (GANs)
5. Ensemble Methods (Random Forest, XGBoost)
6. Reinforcement Learning (DQN, A3C, PPO)
7. Online Learning Approaches
8. Transfer Learning Strategies
9. Federated Learning Algorithm

1.44 1. CNN-LSTM Hybrid Architecture

1.44.1 1.1 Overview

The CNN-LSTM hybrid combines convolutional layers for spatial feature extraction with LSTM layers for temporal modeling. This architecture is particularly effective for network intrusion detection where traffic patterns exhibit both spatial (packet-level) and temporal (sequence-level) characteristics.

1.44.2 1.2 Mathematical Formulation

CNN Layer:

```
Output[i,j,k] = σ(∑∑ W[m,n,k] * Input[i+m, j+n] + b[k])
```

where: - W: convolution kernel weights - b: bias term - σ: activation function (ReLU) - *: convolution operation

LSTM Cell:

```
Forget gate:    f_t = σ(W_f · [h_{t-1}, x_t] + b_f)
Input gate:     i_t = σ(W_i · [h_{t-1}, x_t] + b_i)
Cell candidate: C̃_t = tanh(W_C · [h_{t-1}, x_t] + b_C)
Cell state:     C_t = f_t ⊙ C_{t-1} + i_t ⊙ C̃_t
Output gate:    o_t = σ(W_o · [h_{t-1}, x_t] + b_o)
Hidden state:   h_t = o_t ⊙ tanh(C_t)
```

where: - σ: sigmoid function - ⊙: element-wise multiplication - W: weight matrices - b: bias vectors

1.44.3 1.3 Complete Architecture

```python
def build_cnn_lstm_model(input_shape, num_classes):
    """
    Build CNN-LSTM hybrid model for intrusion detection

    Args:
        input_shape: (timesteps, features)
        num_classes: number of attack categories

    Returns:
```

```
        compiled Keras model
    """

    model = Sequential()

    # Reshape for Conv1D: (batch, timesteps, features)
    model.add(Input(shape=input_shape))

    # CNN layers for spatial feature extraction
    model.add(Conv1D(filters=64, kernel_size=3, padding='same'))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(Dropout(0.2))

    model.add(Conv1D(filters=128, kernel_size=3, padding='same'))
    model.add(BatchNormalization())
    model.add(Activation('relu'))
    model.add(MaxPooling1D(pool_size=2))
    model.add(Dropout(0.2))

    # LSTM layers for temporal modeling
    model.add(LSTM(units=128, return_sequences=True))
    model.add(Dropout(0.3))

    model.add(LSTM(units=64, return_sequences=False))
    model.add(Dropout(0.3))

    # Dense layers for classification
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.5))

    model.add(Dense(num_classes, activation='softmax'))

    # Compile model
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy', 'precision', 'recall']
    )

    return model
```

1.44.4 1.4 Training Algorithm

```
def train_cnn_lstm(X_train, y_train, X_val, y_val, epochs=50, batch_size=128):
    """
    Train CNN-LSTM model with early stopping and learning rate scheduling
    """
```

```
    # Build model
    model = build_cnn_lstm_model(input_shape=(X_train.shape[1], X_train.shape[2]),
                                 num_classes=y_train.shape[1])

    # Callbacks
    early_stop = EarlyStopping(monitor='val_loss', patience=10,
                               restore_best_weights=True)

    reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                                  patience=5, min_lr=1e-6)

    checkpoint = ModelCheckpoint('best_model.h5', monitor='val_accuracy',
                                 save_best_only=True, mode='max')

    # Train
    history = model.fit(
        X_train, y_train,
        batch_size=batch_size,
        epochs=epochs,
        validation_data=(X_val, y_val),
        callbacks=[early_stop, reduce_lr, checkpoint],
        verbose=1
    )

    return model, history
```

1.44.5 1.5 Pseudocode

```
Algorithm: CNN-LSTM Hybrid Training
Input: Training data X_train, labels y_train
Output: Trained model θ

1. Initialize model parameters θ randomly
2. For epoch = 1 to max_epochs:
3.     Shuffle training data
4.     For each mini-batch B in training data:
5.         # Forward pass
6.         # CNN feature extraction
7.         features = CNN_forward(B, θ_CNN)
8.
9.         # LSTM temporal modeling
10.        hidden_states = LSTM_forward(features, θ_LSTM)
11.
12.        # Classification
13.        predictions = Softmax(Dense(hidden_states, θ_dense))
14.
15.        # Compute loss
```

```
        loss = CrossEntropy(predictions, y_batch)

        # Backward pass
        gradients = Backpropagate(loss, θ)

        # Update parameters
        θ = Adam_update(θ, gradients, learning_rate)

    # Validation
    val_loss, val_acc = Evaluate(X_val, y_val, θ)

    # Early stopping check
    If val_loss not improving for patience epochs:
        Break

Return θ
```

1.45 2. Transformer Network for IDS

1.45.1 2.1 Overview

Transformers use self-attention mechanisms to capture long-range dependencies in network traffic sequences without the sequential processing limitations of RNNs.

1.45.2 2.2 Mathematical Formulation

Self-Attention:

```
Q = XW_Q    (Query)
K = XW_K    (Key)
V = XW_V    (Value)

Attention(Q, K, V) = softmax(QK^T / √d_k) V
```

Multi-Head Attention:

```
head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)
MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O
```

Position Encoding:

```
PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

1.45.3 2.3 Implementation

```
import tensorflow as tf
from tensorflow.keras import layers
```

```
class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super(TransformerBlock, self).__init__()
        self.att = layers.MultiHeadAttention(num_heads=num_heads,
                                             key_dim=embed_dim)
        self.ffn = tf.keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim),
        ])
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)

    def call(self, inputs, training):
        # Multi-head attention
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(inputs + attn_output)

        # Feed-forward network
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        return self.layernorm2(out1 + ffn_output)


class PositionalEncoding(layers.Layer):
    def __init__(self, sequence_length, d_model):
        super(PositionalEncoding, self).__init__()
        self.pos_encoding = self.positional_encoding(sequence_length, d_model)

    def get_angles(self, pos, i, d_model):
        angles = 1 / np.power(10000, (2 * (i // 2)) / np.float32(d_model))
        return pos * angles

    def positional_encoding(self, sequence_length, d_model):
        angle_rads = self.get_angles(
            np.arange(sequence_length)[:, np.newaxis],
            np.arange(d_model)[np.newaxis, :],
            d_model
        )
        # Apply sin to even indices
        angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
        # Apply cos to odd indices
        angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])

        pos_encoding = angle_rads[np.newaxis, ...]
```

```
        return tf.cast(pos_encoding, dtype=tf.float32)

    def call(self, inputs):
        return inputs + self.pos_encoding[:, :tf.shape(inputs)[1], :]


def build_transformer_model(input_shape, num_classes,
                            num_heads=8, ff_dim=256, num_blocks=4):
    """
    Build Transformer model for intrusion detection
    """

    inputs = layers.Input(shape=input_shape)

    # Embedding
    x = layers.Dense(256)(inputs)  # Project to d_model

    # Positional encoding
    x = PositionalEncoding(input_shape[0], 256)(x)

    # Transformer blocks
    for _ in range(num_blocks):
        x = TransformerBlock(embed_dim=256, num_heads=num_heads,
                             ff_dim=ff_dim)(x)

    # Global average pooling
    x = layers.GlobalAveragePooling1D()(x)

    # Classification head
    x = layers.Dense(128, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(num_classes, activation='softmax')(x)

    model = tf.keras.Model(inputs=inputs, outputs=outputs)

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

1.45.4 2.4 Pseudocode

```
Algorithm: Transformer-based IDS
Input: Traffic sequence X = [x_1, x_2, ..., x_T]
Output: Attack classification
```

```
# Token embedding
X_embed = Embedding(X)  # Shape: (T, d_model)

# Positional encoding
For pos = 0 to T-1:
    For i = 0 to d_model-1:
        If i is even:
            PE[pos, i] = sin(pos / 10000^(i/d_model))
        Else:
            PE[pos, i] = cos(pos / 10000^(i/d_model))

X = X_embed + PE

# Multi-layer Transformer
For layer = 1 to N:
    # Multi-head self-attention
    Q, K, V = Linear(X), Linear(X), Linear(X)

    For head = 1 to num_heads:
        attention_scores = softmax(Q_h · K_h^T / √d_k)
        head_output[head] = attention_scores · V_h

    Multi_head = Concat(head_output[1:num_heads]) · W^O
    X = LayerNorm(X + Dropout(Multi_head))

    # Feed-forward network
    FFN = ReLU(X · W_1 + b_1) · W_2 + b_2
    X = LayerNorm(X + Dropout(FFN))

# Classification
pooled = GlobalAveragePooling(X)
output = Softmax(Dense(pooled))

Return output
```

1.46 3. Autoencoder for Anomaly Detection

1.46.1 3.1 Overview

Autoencoders learn to compress and reconstruct normal traffic. Anomalies result in high reconstruction errors, enabling unsupervised detection.

1.46.2 3.2 Mathematical Formulation

Encoder:

h = σ(W_e · x + b_e)

Decoder:

x̂ = σ(W_d · h + b_d)

Loss Function:

L(x, x̂) = ||x - x̂||² = ∑_i (x_i - x̂_i)²

Anomaly Score:

score(x) = ||x - x̂||²
If score(x) > threshold → Anomaly
Else → Normal

1.46.3 3.3 Implementation

```
def build_autoencoder(input_dim, encoding_dim=32):
    """
    Build autoencoder for anomaly detection

    Args:
        input_dim: number of input features
        encoding_dim: dimension of latent representation

    Returns:
        encoder, decoder, autoencoder models
    """

    # Encoder
    input_layer = Input(shape=(input_dim,))
    encoded = Dense(128, activation='relu')(input_layer)
    encoded = Dropout(0.2)(encoded)
    encoded = Dense(64, activation='relu')(encoded)
    encoded = Dropout(0.2)(encoded)
    encoded = Dense(encoding_dim, activation='relu')(encoded)

    encoder = Model(input_layer, encoded)

    # Decoder
    encoded_input = Input(shape=(encoding_dim,))
    decoded = Dense(64, activation='relu')(encoded_input)
    decoded = Dropout(0.2)(decoded)
    decoded = Dense(128, activation='relu')(decoded)
    decoded = Dropout(0.2)(decoded)
    decoded = Dense(input_dim, activation='sigmoid')(decoded)

    decoder = Model(encoded_input, decoded)

    # Autoencoder
```

```
    autoencoder = Model(input_layer, decoder(encoder(input_layer)))

    autoencoder.compile(optimizer='adam', loss='mse')

    return encoder, decoder, autoencoder


def train_autoencoder_unsupervised(X_normal, epochs=100, batch_size=256):
    """
    Train autoencoder on normal traffic only
    """

    autoencoder = build_autoencoder(input_dim=X_normal.shape[1])[2]

    history = autoencoder.fit(
        X_normal, X_normal,
        epochs=epochs,
        batch_size=batch_size,
        validation_split=0.1,
        callbacks=[EarlyStopping(patience=10)],
        verbose=1
    )

    return autoencoder


def detect_anomalies(autoencoder, X_test, percentile=95):
    """
    Detect anomalies based on reconstruction error

    Args:
        autoencoder: trained model
        X_test: test data
        percentile: threshold percentile for anomaly detection

    Returns:
        predictions: 0 for normal, 1 for anomaly
    """

    # Reconstruct
    X_reconstructed = autoencoder.predict(X_test)

    # Compute reconstruction error
    mse = np.mean(np.power(X_test - X_reconstructed, 2), axis=1)

    # Determine threshold (e.g., 95th percentile of training errors)
    threshold = np.percentile(mse, percentile)
```

```
    # Classify
    predictions = (mse > threshold).astype(int)

    return predictions, mse
```

1.46.4 3.4 Variational Autoencoder (VAE)

```
def sampling(args):
    """Reparameterization trick"""
    z_mean, z_log_var = args
    batch = tf.shape(z_mean)[0]
    dim = tf.shape(z_mean)[1]
    epsilon = tf.random.normal(shape=(batch, dim))
    return z_mean + tf.exp(0.5 * z_log_var) * epsilon


def build_vae(input_dim, latent_dim=32):
    """
    Build Variational Autoencoder
    """

    # Encoder
    encoder_input = Input(shape=(input_dim,))
    x = Dense(128, activation='relu')(encoder_input)
    x = Dense(64, activation='relu')(x)

    z_mean = Dense(latent_dim, name='z_mean')(x)
    z_log_var = Dense(latent_dim, name='z_log_var')(x)

    z = Lambda(sampling, name='z')([z_mean, z_log_var])

    encoder = Model(encoder_input, [z_mean, z_log_var, z], name='encoder')

    # Decoder
    latent_input = Input(shape=(latent_dim,))
    x = Dense(64, activation='relu')(latent_input)
    x = Dense(128, activation='relu')(x)
    decoder_output = Dense(input_dim, activation='sigmoid')(x)

    decoder = Model(latent_input, decoder_output, name='decoder')

    # VAE
    vae_output = decoder(encoder(encoder_input)[2])
    vae = Model(encoder_input, vae_output, name='vae')

    # VAE loss = reconstruction loss + KL divergence
    reconstruction_loss = tf.reduce_mean(
        tf.reduce_sum(
```

```
            tf.keras.losses.binary_crossentropy(encoder_input, vae_output),
            axis=1
        )
    )

    kl_loss = -0.5 * tf.reduce_mean(
        tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var),
                      axis=1)
    )

    vae_loss = reconstruction_loss + kl_loss
    vae.add_loss(vae_loss)
    vae.compile(optimizer='adam')

    return encoder, decoder, vae
```

1.47 4. Generative Adversarial Networks (GANs)

1.47.1 4.1 Overview

GANs generate synthetic attack samples to augment training data, addressing class imbalance and improving model robustness.

1.47.2 4.2 Mathematical Formulation

Generator:

```
G(z) → fake sample
z ~ N(0, I)  (random noise)
```

Discriminator:

```
D(x) → probability that x is real
```

Loss Functions:

```
L_D = -E[log D(x_real)] - E[log(1 - D(G(z)))]
L_G = -E[log D(G(z))]
```

1.47.3 4.3 Implementation

```
def build_generator(latent_dim, output_dim):
    """
    Build generator network
    """
    model = Sequential([
        Dense(128, input_dim=latent_dim),
        LeakyReLU(alpha=0.2),
```

```
        BatchNormalization(),

        Dense(256),
        LeakyReLU(alpha=0.2),
        BatchNormalization(),

        Dense(512),
        LeakyReLU(alpha=0.2),
        BatchNormalization(),

        Dense(output_dim, activation='tanh')
    ])

    return model


def build_discriminator(input_dim):
    """
    Build discriminator network
    """
    model = Sequential([
        Dense(512, input_dim=input_dim),
        LeakyReLU(alpha=0.2),
        Dropout(0.3),

        Dense(256),
        LeakyReLU(alpha=0.2),
        Dropout(0.3),

        Dense(128),
        LeakyReLU(alpha=0.2),
        Dropout(0.3),

        Dense(1, activation='sigmoid')
    ])

    model.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5))

    return model


def train_gan(X_real, epochs=10000, batch_size=128, latent_dim=100):
    """
    Train GAN for synthetic attack generation
    """

    # Build and compile discriminator
    discriminator = build_discriminator(X_real.shape[1])
```

```
    # Build generator
    generator = build_generator(latent_dim, X_real.shape[1])

    # Combined model (generator + discriminator)
    discriminator.trainable = False
    gan_input = Input(shape=(latent_dim,))
    x = generator(gan_input)
    gan_output = discriminator(x)
    gan = Model(gan_input, gan_output)
    gan.compile(loss='binary_crossentropy', optimizer=Adam(0.0002, 0.5))

    # Training loop
    for epoch in range(epochs):
        # Train discriminator
        idx = np.random.randint(0, X_real.shape[0], batch_size)
        real_samples = X_real[idx]

        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        fake_samples = generator.predict(noise)

        d_loss_real = discriminator.train_on_batch(real_samples,
                                                   np.ones((batch_size, 1)))
        d_loss_fake = discriminator.train_on_batch(fake_samples,
                                                   np.zeros((batch_size, 1)))
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train generator
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        g_loss = gan.train_on_batch(noise, np.ones((batch_size, 1)))

        # Print progress
        if epoch % 1000 == 0:
            print(f"Epoch {epoch}, D Loss: {d_loss:.4f}, G Loss: {g_loss:.4f}")

    return generator, discriminator
```

1.48 5. Ensemble Methods

1.48.1 5.1 Random Forest

```
from sklearn.ensemble import RandomForestClassifier

def train_random_forest(X_train, y_train, n_estimators=100):
    """
    Train Random Forest classifier
```

```
    """

    rf = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=20,
        min_samples_split=10,
        min_samples_leaf=4,
        max_features='sqrt',
        bootstrap=True,
        n_jobs=-1,
        random_state=42
    )

    rf.fit(X_train, y_train)

    # Feature importance
    feature_importance = pd.DataFrame({
        'feature': feature_names,
        'importance': rf.feature_importances_
    }).sort_values('importance', ascending=False)

    return rf, feature_importance
```

1.48.2 5.2 XGBoost

```
import xgboost as xgb

def train_xgboost(X_train, y_train, X_val, y_val):
    """
    Train XGBoost classifier with hyperparameter tuning
    """

    dtrain = xgb.DMatrix(X_train, label=y_train)
    dval = xgb.DMatrix(X_val, label=y_val)

    params = {
        'objective': 'multi:softmax',
        'num_class': len(np.unique(y_train)),
        'max_depth': 8,
        'learning_rate': 0.1,
        'subsample': 0.8,
        'colsample_bytree': 0.8,
        'min_child_weight': 3,
        'gamma': 0.1,
        'reg_alpha': 0.1,
        'reg_lambda': 1,
        'eval_metric': 'mlogloss'
    }
```

```
    evals = [(dtrain, 'train'), (dval, 'val')]

    model = xgb.train(
        params,
        dtrain,
        num_boost_round=1000,
        evals=evals,
        early_stopping_rounds=50,
        verbose_eval=100
    )

    return model
```

1.49 6. Reinforcement Learning

1.49.1 6.1 Deep Q-Network (DQN)

```
class DQNAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.memory = deque(maxlen=10000)
        self.gamma = 0.95    # discount factor
        self.epsilon = 1.0   # exploration rate
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001

        self.model = self._build_model()
        self.target_model = self._build_model()
        self.update_target_model()

    def _build_model(self):
        model = Sequential()
        model.add(Dense(128, input_dim=self.state_size, activation='relu'))
        model.add(Dense(128, activation='relu'))
        model.add(Dense(64, activation='relu'))
        model.add(Dense(self.action_size, activation='linear'))
        model.compile(loss='mse', optimizer=Adam(lr=self.learning_rate))
        return model

    def update_target_model(self):
        self.target_model.set_weights(self.model.get_weights())

    def remember(self, state, action, reward, next_state, done):
```

```
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        if np.random.rand() <= self.epsilon:
            return random.randrange(self.action_size)
        act_values = self.model.predict(state)
        return np.argmax(act_values[0])

    def replay(self, batch_size=32):
        if len(self.memory) < batch_size:
            return

        minibatch = random.sample(self.memory, batch_size)

        for state, action, reward, next_state, done in minibatch:
            target = reward
            if not done:
                target = reward + self.gamma * np.amax(
                    self.target_model.predict(next_state)[0]
                )

            target_f = self.model.predict(state)
            target_f[0][action] = target

            self.model.fit(state, target_f, epochs=1, verbose=0)

        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay
```

1.49.2 6.2 Proximal Policy Optimization (PPO)

```
class PPOAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.gamma = 0.99
        self.epsilon_clip = 0.2
        self.learning_rate = 0.0003

        self.actor = self._build_actor()
        self.critic = self._build_critic()

    def _build_actor(self):
        model = Sequential([
            Dense(128, activation='relu', input_dim=self.state_size),
            Dense(128, activation='relu'),
            Dense(self.action_size, activation='softmax')
        ])
```

```
        model.compile(optimizer=Adam(lr=self.learning_rate), loss='categorical_crossent
        return model

    def _build_critic(self):
        model = Sequential([
            Dense(128, activation='relu', input_dim=self.state_size),
            Dense(128, activation='relu'),
            Dense(1, activation='linear')
        ])
        model.compile(optimizer=Adam(lr=self.learning_rate), loss='mse')
        return model

    def act(self, state):
        probabilities = self.actor.predict(state)[0]
        action = np.random.choice(self.action_size, p=probabilities)
        return action, probabilities[action]

    def train(self, states, actions, rewards, next_states, old_probs):
        # Compute advantages
        values = self.critic.predict(states)
        next_values = self.critic.predict(next_states)

        advantages = rewards + self.gamma * next_values - values

        # Actor loss (PPO clip objective)
        new_probs = self.actor.predict(states)
        ratio = new_probs / (old_probs + 1e-10)
        clipped_ratio = np.clip(ratio, 1 - self.epsilon_clip, 1 + self.epsilon_clip)
        actor_loss = -np.minimum(ratio * advantages, clipped_ratio * advantages)

        # Update actor
        self.actor.fit(states, actions, sample_weight=actor_loss.flatten(), verbose=0)

        # Update critic
        self.critic.fit(states, rewards + self.gamma * next_values, verbose=0)
```

1.50 7. Online Learning

1.50.1 7.1 Incremental Learning

```
from sklearn.linear_model import SGDClassifier

def online_learning_sgd(initial_model, stream_data_generator):
    """
    Online learning with streaming data
```

```
    Args:
        initial_model: pre-trained model
        stream_data_generator: generator yielding (X_batch, y_batch)
    """

    model = SGDClassifier(loss='log', learning_rate='constant', eta0=0.01)

    # Initialize with initial data
    X_init, y_init = next(stream_data_generator)
    model.partial_fit(X_init, y_init, classes=np.unique(y_init))

    # Continuous learning
    for X_batch, y_batch in stream_data_generator:
        # Incremental update
        model.partial_fit(X_batch, y_batch)

        # Evaluate periodically
        if iteration % 100 == 0:
            accuracy = model.score(X_val, y_val)
            print(f"Iteration {iteration}, Accuracy: {accuracy:.4f}")

    return model
```

1.50.2 7.2 Concept Drift Detection

```
def detect_concept_drift(model, X_stream, y_stream, window_size=1000):
    """
    Detect concept drift using Page-Hinkley test
    """

    errors = []
    cumsum = 0
    min_cumsum = 0
    drift_threshold = 50

    for i, (x, y) in enumerate(zip(X_stream, y_stream)):
        # Predict
        y_pred = model.predict([x])[0]
        error = int(y_pred != y)
        errors.append(error)

        # Update cumulative sum
        cumsum += error - np.mean(errors)

        # Page-Hinkley test
        if cumsum < min_cumsum:
            min_cumsum = cumsum
```

```
        drift_magnitude = cumsum - min_cumsum

        if drift_magnitude > drift_threshold:
            print(f"Concept drift detected at sample {i}")
            # Retrain model
            model = retrain_model(X_stream[i-window_size:i],
                                  y_stream[i-window_size:i])
            cumsum = 0
            min_cumsum = 0

    return model
```

1.51 8. Transfer Learning

1.51.1 8.1 Domain Adaptation

```
def transfer_learning_ids(source_model, target_data, freeze_layers=5):
    """
    Transfer learning from source domain to target domain

    Args:
        source_model: pre-trained model on source domain (e.g., NSL-KDD)
        target_data: data from target domain (e.g., CICIDS2017)
        freeze_layers: number of initial layers to freeze
    """

    # Create new model based on source
    target_model = clone_model(source_model)
    target_model.set_weights(source_model.get_weights())

    # Freeze early layers (feature extractors)
    for layer in target_model.layers[:freeze_layers]:
        layer.trainable = False

    # Replace final classification layer
    target_model.pop()  # Remove last layer
    target_model.add(Dense(num_target_classes, activation='softmax'))

    # Fine-tune on target domain
    target_model.compile(optimizer=Adam(lr=0.0001),
                         loss='categorical_crossentropy',
                         metrics=['accuracy'])

    target_model.fit(X_target_train, y_target_train, epochs=20, batch_size=128)

    return target_model
```

1.52 9. Federated Learning

1.52.1 9.1 FedAvg Algorithm

```python
def federated_averaging(global_model, client_data, num_rounds=100):
    """
    Federated Averaging (FedAvg) algorithm

    Args:
        global_model: initial global model
        client_data: list of (X_train, y_train) for each client
        num_rounds: number of communication rounds
    """

    for round_num in range(num_rounds):
        print(f"Round {round_num + 1}/{num_rounds}")

        client_models = []
        client_weights = []

        # Each client trains locally
        for client_id, (X_client, y_client) in enumerate(client_data):
            # Send global model to client
            client_model = clone_model(global_model)
            client_model.set_weights(global_model.get_weights())

            # Local training
            client_model.fit(X_client, y_client, epochs=5, batch_size=32, verbose=0)

            client_models.append(client_model)
            client_weights.append(len(X_client))  # Weight by data size

        # Aggregate models (weighted average)
        total_samples = sum(client_weights)

        for layer_idx in range(len(global_model.layers)):
            if len(global_model.layers[layer_idx].get_weights()) > 0:
                # Weighted average of layer weights
                avg_weights = sum([
                    client_model.layers[layer_idx].get_weights()[0] * weight / total_sa
                    for client_model, weight in zip(client_models, client_weights)
                ])

                avg_bias = sum([
                    client_model.layers[layer_idx].get_weights()[1] * weight / total_sa
```

```
                for client_model, weight in zip(client_models, client_weights)
            ])

            global_model.layers[layer_idx].set_weights([avg_weights, avg_bias])

    # Evaluate global model
    if round_num % 10 == 0:
        accuracy = evaluate_global_model(global_model, X_test, y_test)
        print(f"Global Model Accuracy: {accuracy:.4f}")

return global_model
```

1.53 Conclusion

This document provides comprehensive algorithm specifications for all AI/ML components in the proposed IDS framework. Each algorithm includes mathematical formulations, implementation details, and pseudocode for clarity and reproducibility. These algorithms form the foundation of the hybrid, adaptive, and distributed intrusion detection system.

Key Takeaways: - CNN-LSTM combines spatial and temporal feature learning - Transformers capture long-range dependencies via self-attention - Autoencoders enable unsupervised anomaly detection - GANs generate synthetic attack samples for data augmentation - Ensemble methods improve robustness and accuracy - Reinforcement learning enables adaptive threat response - Online learning handles concept drift - Transfer learning enables cross-domain knowledge sharing - Federated learning preserves privacy in distributed scenarios

---

Next: Refer to `datasets.md` for data preparation details and `evaluation_metrics.md` for performance assessment methodology. # Datasets for IDS Research ## Comprehensive Dataset Information and Preprocessing Pipeline

Dataset Specifications, Characteristics, and Usage Guidelines
Last Updated: December 22, 2025

1.54 Table of Contents

7. Data Preprocessing Pipeline
8. Train/Validation/Test Splits
9. Class Imbalance Handling
10. Data Augmentation Strategies

1.55 1. Dataset Overview

1.55.1 1.1 Summary Table

Dataset	Year	Samples	Features	Attack Types	Protocol Coverage	Source
NSL-KDD	2009	148,517	41	4 categories, 37 types	TCP, UDP, ICMP	Refined KDD'99
CICIDS2017	2017	2.8M flows	80	14 types	HTTP, HTTPS, FTP, SSH, etc.	Canadian Inst. Cyber-security
UNSW-NB15	2015	2.54M	49	9 categories	TCP, UDP, ICMP, etc.	UNSW Canberra
IoT-23	2020	325M pkts	Variable	20+ IoT malware	IoT protocols (MQTT, CoAP)	Avast AIC
5G Custom	2025+	TBD	TBD	NGN-specific	5G NR, Slicing	Testbed collection

1.55.2 1.2 Dataset Selection Rationale

NSL-KDD: - Widely used benchmark for comparison with existing work - Balanced classes (no excessive duplicate records like KDD'99) - Established baseline for algorithm validation

CICIDS2017: - Modern attack types (DDoS, web attacks, botnet) - Realistic network traffic (captured from real infrastructure) - Labeled flows with 80+ features for deep learning

UNSW-NB15: - Hybrid synthetic and real attacks - Diverse attack categories (9 types) - Contemporary attack techniques (2015)

IoT-23: - IoT-specific malware (Mirai, Torii, etc.) - Real IoT device captures - Essential for IoT security research

5G Custom: - Addresses gap in 5G-specific attack datasets - Network slicing attacks - Core network vulnerabilities

---

1.56 2. NSL-KDD Dataset

1.56.1 2.1 Overview

NSL-KDD is a refined version of the KDD Cup 1999 dataset, addressing issues such as duplicate records and imbalanced classes.

Statistics: - Training set: 125,973 records - Test set: 22,544 records - Features: 41 (38 numerical, 3 categorical) - Classes: Normal + 4 attack categories

1.56.2 2.2 Attack Categories

Category	Description	Types	% in Train	% in Test
DoS	Denial of Service	back, land, neptune, pod, smurf, teardrop	45.93%	28.67%
Probe	Surveillance/Scanning	ipsweep, nmap, portsweep, satan	11.66%	12.34%
R2L	Remote to Local	ftp_write, guess_passwd, imap, multihop, phf, spy, warezclient, warezmaster	0.83%	23.63%
U2R	User to Root	buffer_overflow, loadmodule, perl, rootkit	0.04%	0.71%
Normal	Legitimate traffic	-	53.54%	34.65%

1.56.3 2.3 Feature Description

Basic Features (9): 1. duration: Connection duration (seconds) 2. protocol_type: TCP, UDP, ICMP 3. service: HTTP, FTP, SMTP, etc. (70 values) 4. flag: Connection status (SF, S0, REJ, etc.) 5. src_bytes: Bytes from source to destination 6. dst_bytes: Bytes from destination to source 7. land: 1 if connection from/to same host/port 8. wrong_fragment: Number of wrong fragments 9. urgent: Number of urgent packets

Content Features (13): 10. hot: Number of “hot” indicators 11. num_failed_logins: Number of failed login attempts 12. logged_in: 1 if successfully logged in 13.

num_compromised: Number of compromised conditions 14. root_shell: 1 if root shell obtained 15. su_attempted: 1 if su command attempted 16. num_root: Number of root accesses 17. num_file_creations: Number of file creation operations 18. num_shells: Number of shell prompts 19. num_access_files: Number of operations on access control files 20. num_outbound_cmds: Number of outbound commands (FTP) 21. is_host_login: 1 if login belongs to host list 22. is_guest_login: 1 if guest login

Traffic Features (9): 23-31. count, srv_count, serror_rate, srv_serror_rate, rerror_rate, srv_rerror_rate, same_srv_rate, diff_srv_rate, srv_diff_host_rate

Time-based Traffic Features (9): 32-41. dst_host_count, dst_host_srv_count, dst_host_same_srv_rate, dst_host_diff_srv_rate, dst_host_same_src_port_rate, dst_host_srv_diff_host_rate, dst_host_serror_rate, dst_host_srv_serror_rate, dst_host_rerror_rate, dst_host_srv_rerror_rate

1.56.4 2.4 Preprocessing Steps

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler

def preprocess_nslkdd(file_path):
    """
    Preprocess NSL-KDD dataset
    """

    # Column names
    columns = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes',
               'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
               'num_failed_logins', 'logged_in', 'num_compromised',
               'root_shell', 'su_attempted', 'num_root', 'num_file_creations',
               'num_shells', 'num_access_files', 'num_outbound_cmds',
               'is_host_login', 'is_guest_login', 'count', 'srv_count',
               'serror_rate', 'srv_serror_rate', 'rerror_rate',
               'srv_rerror_rate', 'same_srv_rate', 'diff_srv_rate',
               'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
               'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
               'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
               'dst_host_serror_rate', 'dst_host_srv_serror_rate',
               'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'label', 'difficulty

    # Load data
    df = pd.read_csv(file_path, names=columns)

    # Map attack types to categories
    attack_mapping = {
        'normal': 'Normal',
        'back': 'DoS', 'land': 'DoS', 'neptune': 'DoS', 'pod': 'DoS',
        'smurf': 'DoS', 'teardrop': 'DoS', 'apache2': 'DoS', 'udpstorm': 'DoS',
```

```
        'ipsweep': 'Probe', 'nmap': 'Probe', 'portsweep': 'Probe', 'satan': 'Probe',
        'mscan': 'Probe', 'saint': 'Probe',
        'ftp_write': 'R2L', 'guess_passwd': 'R2L', 'imap': 'R2L',
        'multihop': 'R2L', 'phf': 'R2L', 'spy': 'R2L', 'warezclient': 'R2L',
        'warezmaster': 'R2L', 'sendmail': 'R2L', 'named': 'R2L',
        'snmpgetattack': 'R2L', 'snmpguess': 'R2L', 'xlock': 'R2L', 'xsnoop': 'R2L',
        'buffer_overflow': 'U2R', 'loadmodule': 'U2R', 'perl': 'U2R',
        'rootkit': 'U2R', 'httptunnel': 'U2R', 'ps': 'U2R', 'sqlattack': 'U2R'
    }

    df['attack_category'] = df['label'].str.strip().map(attack_mapping)

    # Encode categorical features
    categorical_cols = ['protocol_type', 'service', 'flag']
    label_encoders = {}

    for col in categorical_cols:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        label_encoders[col] = le

    # Separate features and labels
    X = df.drop(['label', 'attack_category', 'difficulty'], axis=1)
    y = df['attack_category']

    # Normalize numerical features
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    return X_scaled, y, label_encoders, scaler


### 2.5 Usage Guidelines

**Binary Classification (Normal vs. Attack):**
```python
y_binary = y.map({'Normal': 0, 'DoS': 1, 'Probe': 1, 'R2L': 1, 'U2R': 1})
```

Multi-class Classification (5 classes):

```python
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_multiclass = le.fit_transform(y)  # 0: Normal, 1: DoS, 2: Probe, 3: R2L, 4: U2R
```

```

1.57 3. CICIDS2017/2018 Dataset

1.57.1 3.1 Overview

The Canadian Institute for Cybersecurity Intrusion Detection System (CICIDS2017) dataset contains labeled network flows with modern attack types.

Statistics: - Total flows: ~2.8 million - Features: 80 (78 numerical, 2 categorical) - Duration: 5 days (Monday-Friday) - Attacks: 14 types across various categories

1.57.2 3.2 Attack Types by Day

Day	Date	Traffic	Attacks
Monday	7/3/2017	Normal	None (baseline)
Tuesday	7/4/2017	Brute Force, XSS	FTP-Patator, SSH-Patator, XSS
Wednesday	7/5/2017	DoS/DDoS	DoS GoldenEye, DoS Hulk, DoS Slowhttptest, DoS Slowloris, Heartbleed
Thursday	7/6/2017	Web Attacks, Infiltration	Web Attack (Brute Force, XSS, SQL Injection), Infiltration
Friday	7/7/2017	Botnet, Port Scan, DDoS	Botnet ARES, Port Scan, DDoS

1.57.3 3.3 Feature Categories

Flow-based Features (80 total):

1. **Flow Identifiers:**
   - Flow ID, Source IP, Source Port, Destination IP, Destination Port, Protocol, Timestamp
2. **Duration:**
   - Flow Duration
3. **Packet Counts:**
   - Total Fwd Packets, Total Backward Packets
4. **Byte Counts:**
   - Total Length of Fwd Packets, Total Length of Bwd Packets
5. **Packet Length Statistics:**
   - Fwd/Bwd Packet Length Max, Mean, Min, Std
6. **Inter-Arrival Time (IAT):**

- Flow IAT Mean, Std, Max, Min
- Fwd IAT Total, Mean, Std, Max, Min
- Bwd IAT Total, Mean, Std, Max, Min

7. **Flags:**
   - PSH Flag Count, URG Flag Count, FIN Flag Count, SYN Flag Count, RST Flag Count, ACK Flag Count
8. **Header Lengths:**
   - Fwd/Bwd Header Length
9. **Packets/Second:**
   - Flow Packets/s, Flow Bytes/s
10. **Down/Up Ratio**
11. **Average Packet Size**
12. **Subflow:**
    - Fwd/Bwd Packets, Bytes
13. **Init_Win_bytes (TCP Window)**
14. **Active/Idle Times:**
    - Active Mean, Std, Max, Min
    - Idle Mean, Std, Max, Min

1.57.4 3.4 Preprocessing Steps

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler

def preprocess_cicids2017(file_path):
    """
    Preprocess CICIDS2017 dataset
    """

    # Load data
    df = pd.read_csv(file_path)

    # Handle missing values and infinities
    df.replace([np.inf, -np.inf], np.nan, inplace=True)
    df.fillna(0, inplace=True)

    # Remove duplicates
    df.drop_duplicates(inplace=True)

    # Map labels
```

```
    label_mapping = {
        'BENIGN': 'Normal',
        'Bot': 'Botnet',
        'PortScan': 'PortScan',
        'DDoS': 'DDoS',
        'DoS GoldenEye': 'DoS',
        'DoS Hulk': 'DoS',
        'DoS Slowhttptest': 'DoS',
        'DoS slowloris': 'DoS',
        'FTP-Patator': 'BruteForce',
        'SSH-Patator': 'BruteForce',
        'Web Attack – Brute Force': 'WebAttack',
        'Web Attack – XSS': 'WebAttack',
        'Web Attack – Sql Injection': 'WebAttack',
        'Infiltration': 'Infiltration',
        'Heartbleed': 'Heartbleed'
    }

    df['Label'] = df['Label'].str.strip().map(label_mapping)

    # Select features (exclude IDs, IPs, timestamps)
    feature_cols = [col for col in df.columns if col not in
                    ['Flow ID', 'Source IP', 'Source Port', 'Destination IP',
                     'Destination Port', 'Timestamp', 'Label']]

    X = df[feature_cols]
    y = df['Label']

    # Normalize
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    return X_scaled, y, scaler


### 3.5 Usage Guidelines

**Recommended Split:**
- Use Monday + Tuesday (first 2 days) for training
- Wednesday-Thursday for validation
- Friday for testing
- This ensures temporal separation and realistic scenario

**Class Distribution:**
- Normal: ~83%
- DoS: ~11%
- PortScan: ~3%
- Other attacks: <3%
```

---

4. UNSW-NB15 Dataset

4.1 Overview

The UNSW-NB15 dataset was created by UNSW Canberra using IXIA traffic generator to simu

Statistics:
- Total records: 2,540,044
- Training set: 175,341
- Test set: 82,332
- Features: 49 (42 numerical, 7 categorical)
- Attack categories: 9

4.2 Attack Categories

Category	Description	% in Dataset
Normal	Legitimate traffic	56%
Generic	Attack affecting block cipher	18%
Exploits	Dictionary brute force, backdoor	11%
Fuzzers	Fuzzing tools	8%
DoS	Denial of Service	4%
Reconnaissance	Scanning, probing	3%
Analysis	Port scanning, spam	<1%
Backdoor	Bypassing authentication	<1%
Shellcode	Code injection	<1%
Worms	Self-replicating malware	<1%

4.3 Feature Categories

Flow Features:
- srcip, sport, dstip, dsport, proto (TCP/UDP/ICMP)
- state (connection state)
- dur (record duration)
- sbytes, dbytes (source/destination bytes)
- sttl, dttl (source/destination time to live)
- sloss, dloss (source/destination packet loss)

Content Features:
- sload, dload (source/destination bits per second)
- spkts, dpkts (source/destination packet count)
- swin, dwin (source/destination TCP window size)
- stcpb, dtcpb (TCP base sequence number)
- smeansz, dmeansz (mean flow packet size)

Time Features:
- sintpkt, dintpkt (inter-packet arrival time)
- sjit, djit (jitter)
- sinpkt, dinpkt (inter-arrival time)

Additional Features:
- tcprtt (TCP round-trip time)
- synack, ackdat (TCP connection setup times)
- ct_state_ttl, ct_flw_http_mthd, is_ftp_login, etc.

4.4 Preprocessing

```python
def preprocess_unsw_nb15(train_path, test_path):
    """
    Preprocess UNSW-NB15 dataset
    """

    # Column names
    columns = ['srcip', 'sport', 'dstip', 'dsport', 'proto', 'state', 'dur',
               'sbytes', 'dbytes', 'sttl', 'dttl', 'sloss', 'dloss', 'service',
               'sload', 'dload', 'spkts', 'dpkts', 'swin', 'dwin', 'stcpb',
               'dtcpb', 'smeansz', 'dmeansz', 'trans_depth', 'res_bdy_len',
               'sjit', 'djit', 'stime', 'ltime', 'sintpkt', 'dintpkt',
               'tcprtt', 'synack', 'ackdat', 'is_sm_ips_ports', 'ct_state_ttl',
               'ct_flw_http_mthd', 'is_ftp_login', 'ct_ftp_cmd', 'ct_srv_src',
               'ct_srv_dst', 'ct_dst_ltm', 'ct_src_ltm', 'ct_src_dport_ltm',
               'ct_dst_sport_ltm', 'ct_dst_src_ltm', 'attack_cat', 'label']

    # Load
    train_df = pd.read_csv(train_path, names=columns, skiprows=1)
    test_df = pd.read_csv(test_path, names=columns, skiprows=1)

    # Handle missing and infinite values
    for df in [train_df, test_df]:
        df.replace([np.inf, -np.inf], np.nan, inplace=True)
        df.fillna(0, inplace=True)

    # Encode categorical features
    categorical_cols = ['proto', 'state', 'service']

    for col in categorical_cols:
        le = LabelEncoder()
        combined = pd.concat([train_df[col], test_df[col]])
        le.fit(combined)
        train_df[col] = le.transform(train_df[col])
        test_df[col] = le.transform(test_df[col])
```

```
    # Select features (exclude IPs, timestamps)
    feature_cols = [col for col in columns if col not in
                    ['srcip', 'dstip', 'stime', 'ltime', 'attack_cat', 'label']]

    X_train = train_df[feature_cols]
    y_train = train_df['attack_cat']
    X_test = test_df[feature_cols]
    y_test = test_df['attack_cat']

    # Normalize
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    return X_train_scaled, y_train, X_test_scaled, y_test, scaler
```

1.58 5. IoT-23 Dataset

1.58.1 5.1 Overview

The IoT-23 dataset contains real IoT botnet traffic captured from 23 infected IoT devices.

Statistics: - Total captures: 23 scenarios - Total packets: 325 million+ - Duration: 20+ scenarios, each 1-4 hours - Malware: Mirai, Torii, Gafgyt, etc.

1.58.2 5.2 Malware Types

Mirai Variants: - Most common IoT botnet - DDoS attacks, brute force scanning - Multiple variants captured

Torii: - Sophisticated, persistent botnet - Multiple exploitation methods - Telnet brute force, command injection

Others: - Gafgyt, Kenjiro, Hide and Seek, Hakai

1.58.3 5.3 Feature Extraction

Since IoT-23 provides PCAP files, features must be extracted:

```
from scapy.all import rdpcap, IP, TCP, UDP
import pandas as pd

def extract_features_iot23(pcap_file):
    """
    Extract features from IoT-23 PCAP files
    """
```

```
    packets = rdpcap(pcap_file)
    features = []

    for pkt in packets:
        if IP in pkt:
            feature = {
                'timestamp': float(pkt.time),
                'src_ip': pkt[IP].src,
                'dst_ip': pkt[IP].dst,
                'protocol': pkt[IP].proto,
                'packet_len': len(pkt),
                'ttl': pkt[IP].ttl,
                'flags': pkt[IP].flags
            }

            if TCP in pkt:
                feature.update({
                    'src_port': pkt[TCP].sport,
                    'dst_port': pkt[TCP].dport,
                    'tcp_flags': pkt[TCP].flags,
                    'window_size': pkt[TCP].window
                })
            elif UDP in pkt:
                feature.update({
                    'src_port': pkt[UDP].sport,
                    'dst_port': pkt[UDP].dport
                })

            features.append(feature)

    df = pd.DataFrame(features)
    return df
```

1.59 6. 5G/Custom Dataset Collection

1.59.1 6.1 Collection Plan

Testbed Setup: - Open5GS or free5GC core network - srsRAN or OAI for RAN simulation - UERANSIM for UE simulation - Mininet for network topology

Attack Scenarios: - Network slicing isolation violations - Signaling storm attacks - Authentication and key agreement (AKA) attacks - Denial of service on core functions (AMF, SMF) - Man-in-the-middle on user plane

1.59.2 6.2 Data Collection Tools

```
import pyshark
import pandas as pd

def capture_5g_traffic(interface='eth0', duration=3600):
    """
    Capture 5G network traffic
    """

    capture = pyshark.LiveCapture(interface=interface)
    features = []

    for packet in capture.sniff_continuously(packet_count=100000):
        try:
            feature = {
                'timestamp': packet.sniff_timestamp,
                'src_ip': packet.ip.src,
                'dst_ip': packet.ip.dst,
                'protocol': packet.highest_layer,
                'length': packet.length
            }

            if hasattr(packet, 'tcp'):
                feature['src_port'] = packet.tcp.srcport
                feature['dst_port'] = packet.tcp.dstport

            features.append(feature)
        except AttributeError:
            continue

    df = pd.DataFrame(features)
    return df
```

1.60 7. Data Preprocessing Pipeline

1.60.1 7.1 Complete Pipeline

```
class DataPreprocessor:
    def __init__(self):
        self.scaler = StandardScaler()
        self.label_encoders = {}
        self.feature_selector = None

    def fit_transform(self, X, y):
        """
```

```
        Fit and transform training data
        """
        X_processed = X.copy()

        # 1. Handle missing values
        X_processed = self._handle_missing(X_processed)

        # 2. Remove outliers
        X_processed = self._remove_outliers(X_processed)

        # 3. Encode categorical features
        X_processed = self._encode_categorical(X_processed, fit=True)

        # 4. Feature engineering
        X_processed = self._engineer_features(X_processed)

        # 5. Feature selection
        X_processed = self._select_features(X_processed, y, fit=True)

        # 6. Normalize
        X_processed = self.scaler.fit_transform(X_processed)

        # 7. Handle class imbalance
        X_balanced, y_balanced = self._balance_classes(X_processed, y)

        return X_balanced, y_balanced

    def transform(self, X):
        """
        Transform test data
        """
        X_processed = X.copy()
        X_processed = self._handle_missing(X_processed)
        X_processed = self._encode_categorical(X_processed, fit=False)
        X_processed = self._engineer_features(X_processed)
        X_processed = self._select_features(X_processed, fit=False)
        X_processed = self.scaler.transform(X_processed)
        return X_processed

    def _handle_missing(self, X):
        """Handle missing values"""
        X.fillna(X.median(), inplace=True)
        return X

    def _remove_outliers(self, X, threshold=3):
        """Remove outliers using Z-score"""
        z_scores = np.abs((X - X.mean()) / X.std())
        X_cleaned = X[(z_scores < threshold).all(axis=1)]
```

```
        return X_cleaned

    def _encode_categorical(self, X, fit=False):
        """Encode categorical features"""
        categorical_cols = X.select_dtypes(include=['object']).columns

        for col in categorical_cols:
            if fit:
                le = LabelEncoder()
                X[col] = le.fit_transform(X[col].astype(str))
                self.label_encoders[col] = le
            else:
                X[col] = self.label_encoders[col].transform(X[col].astype(str))

        return X

    def _engineer_features(self, X):
        """Create additional features"""
        # Example: ratios, interactions
        if 'src_bytes' in X.columns and 'dst_bytes' in X.columns:
            X['byte_ratio'] = X['src_bytes'] / (X['dst_bytes'] + 1)
        return X

    def _select_features(self, X, y=None, fit=False, k=50):
        """Select top k features"""
        if fit:
            from sklearn.feature_selection import SelectKBest, f_classif
            self.feature_selector = SelectKBest(f_classif, k=k)
            X_selected = self.feature_selector.fit_transform(X, y)
        else:
            X_selected = self.feature_selector.transform(X)

        return X_selected

    def _balance_classes(self, X, y):
        """Balance classes using SMOTE"""
        from imblearn.over_sampling import SMOTE

        smote = SMOTE(random_state=42)
        X_resampled, y_resampled = smote.fit_resample(X, y)

        return X_resampled, y_resampled
```

1.61 8. Train/Validation/Test Splits

1.61.1 8.1 Split Strategy

```
from sklearn.model_selection import train_test_split

def create_splits(X, y, test_size=0.15, val_size=0.15, random_state=42):
    """
    Create train/validation/test splits

    Ratios: 70% train, 15% validation, 15% test
    """

    # First split: separate test set
    X_temp, X_test, y_temp, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    # Second split: separate validation from train
    val_ratio = val_size / (1 - test_size)
    X_train, X_val, y_train, y_val = train_test_split(
        X_temp, y_temp, test_size=val_ratio, random_state=random_state, stratify=y_temp
    )

    print(f"Train: {len(X_train)} samples ({len(X_train)/len(X)*100:.1f}%)")
    print(f"Validation: {len(X_val)} samples ({len(X_val)/len(X)*100:.1f}%)")
    print(f"Test: {len(X_test)} samples ({len(X_test)/len(X)*100:.1f}%)")

    return X_train, X_val, X_test, y_train, y_val, y_test
```

1.62 9. Class Imbalance Handling

1.62.1 9.1 SMOTE (Synthetic Minority Over-sampling)

```
from imblearn.over_sampling import SMOTE, ADASYN
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline

# SMOTE
smote = SMOTE(sampling_strategy='minority', k_neighbors=5, random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

# ADASYN (Adaptive Synthetic)
adasyn = ADASYN(sampling_strategy='minority', n_neighbors=5, random_state=42)
X_resampled, y_resampled = adasyn.fit_resample(X_train, y_train)
```

```
# Combined over-sampling and under-sampling
pipeline = Pipeline([
    ('over', SMOTE(sampling_strategy=0.5)),
    ('under', RandomUnderSampler(sampling_strategy=0.8))
])
X_resampled, y_resampled = pipeline.fit_resample(X_train, y_train)
```

1.62.2 9.2 Class Weights

```
from sklearn.utils.class_weight import compute_class_weight

# Compute class weights
class_weights = compute_class_weight('balanced',
                                     classes=np.unique(y_train),
                                     y=y_train)

class_weight_dict = dict(enumerate(class_weights))

# Use in model training
model.fit(X_train, y_train, class_weight=class_weight_dict)
```

1.63 10. Data Augmentation Strategies

1.63.1 10.1 Gaussian Noise Injection

```
def augment_with_noise(X, noise_level=0.01):
    """
    Add Gaussian noise to features
    """
    noise = np.random.normal(0, noise_level, X.shape)
    X_augmented = X + noise
    return X_augmented
```

1.63.2 10.2 Feature Masking

```
def augment_with_masking(X, mask_prob=0.1):
    """
    Randomly mask features (set to 0)
    """
    mask = np.random.binomial(1, 1 - mask_prob, X.shape)
    X_masked = X * mask
    return X_masked
```

1.63.3 10.3 Time-Series Augmentation

```python
def augment_time_series(X, methods=['jitter', 'scaling', 'rotation']):
    """
    Augment time-series data
    """
    augmented = []

    for method in methods:
        if method == 'jitter':
            # Add small random noise
            X_aug = X + np.random.normal(0, 0.03, X.shape)
        elif method == 'scaling':
            # Scale by random factor
            scale = np.random.normal(1.0, 0.1)
            X_aug = X * scale
        elif method == 'rotation':
            # Rotate features (circular shift)
            shift = np.random.randint(1, X.shape[1])
            X_aug = np.roll(X, shift, axis=1)

        augmented.append(X_aug)

    return np.vstack([X] + augmented)
```

1.64 Conclusion

This document provides comprehensive information about datasets used in the IDS research, including detailed specifications, preprocessing pipelines, and best practices for data preparation. The combination of established benchmarks (NSL-KDD, CICIDS2017, UNSW-NB15, IoT-23) and custom 5G data collection ensures thorough evaluation across diverse attack scenarios and network environments.

Key Points: - Multiple datasets for cross-validation and generalization - Comprehensive preprocessing pipeline for data quality - Stratified splits maintain class distribution - SMOTE and class weights address imbalance - Data augmentation improves model robustness

---

Next: Refer to `evaluation_metrics.md` for performance assessment methodology and `algorithms.md` for model implementation details. # Evaluation Metrics ## Comprehensive Performance Assessment Methodology

Metrics for Detection Performance, Efficiency, Scalability, Adaptability, and Explainability
Last Updated: December 22, 2025

1.65 Table of Contents

1.66 1. Overview

Comprehensive evaluation of the AI-based IDS requires metrics across multiple dimensions: detection accuracy, computational efficiency, scalability, adaptability to new threats, explainability, and privacy preservation. This document defines all metrics, their mathematical formulations, target values, and measurement methodologies.

1.66.1 1.1 Evaluation Dimensions

Dimension	Key Metrics	Target Goals
Detection Performance	Accuracy, Precision, Recall, F1, AUC-ROC	>98% accuracy, <1% FPR
Efficiency	Latency, Throughput, CPU/Memory usage	<100ms latency, >10K pps
Scalability	Performance vs. size, Load capacity	Linear scaling to 100K devices
Adaptability	Zero-day detection, Drift handling	>95% zero-day detection
Explainability	Feature importance, User trust	High interpretability scores
Privacy	Privacy loss, Information leakage	$\varepsilon < 1.0$ (differential privacy)

1.67 2. Detection Performance Metrics

1.67.1 2.1 Confusion Matrix

```
                Predicted
                Positive  Negative
Actual  Positive    TP        FN
        Negative    FP        TN
```

Where: - **TP (True Positive):** Attacks correctly identified as attacks - **TN (True Negative):** Normal traffic correctly identified as normal - **FP (False Positive):** Normal traffic incorrectly identified as attacks - **FN (False Negative):** Attacks incorrectly identified as normal

1.67.2 2.2 Basic Metrics

Accuracy:

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

- Measures overall correctness
- Target: **>98%**
- Limitation: Can be misleading with imbalanced datasets

Precision (Positive Predictive Value):

```
Precision = TP / (TP + FP)
```

- Proportion of predicted attacks that are actual attacks
- Target: **>97%**
- Important for minimizing false alarms

Recall (Sensitivity, True Positive Rate):

```
Recall = TP / (TP + FN)
```

- Proportion of actual attacks that are detected
- Target: **>98%**
- Critical for security (missing attacks is costly)

Specificity (True Negative Rate):

```
Specificity = TN / (TN + FP)
```

- Proportion of normal traffic correctly identified
- Target: **>99%**

F1-Score (Harmonic Mean of Precision and Recall):

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

- Balanced measure for imbalanced datasets
- Target: **>97%**

False Positive Rate (FPR):

```
FPR = FP / (FP + TN) = 1 - Specificity
```

- Proportion of normal traffic incorrectly flagged
- Target: **<1%**
- Critical for operational feasibility

False Negative Rate (FNR, Miss Rate):

```
FNR = FN / (FN + TP) = 1 - Recall
```

- Proportion of attacks missed
- Target: **<2%**

1.67.3 2.3 Advanced Metrics

Matthews Correlation Coefficient (MCC):

```
MCC = (TP×TN - FP×FN) / sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN))
```

- Range: [-1, 1], where 1 = perfect prediction
- Balanced measure even for imbalanced datasets
- Target: **>0.95**

Cohen's Kappa:

```
κ = (p_o - p_e) / (1 - p_e)
```

where: - p_o: observed agreement (accuracy) - p_e: expected agreement by chance - Target: **>0.90** (almost perfect agreement)

Area Under ROC Curve (AUC-ROC):

```
ROC curve: Plot of TPR vs. FPR at various thresholds
AUC = ∫ TPR d(FPR)
```

- Range: [0, 1], where 1 = perfect classifier
- Threshold-independent metric
- Target: **>0.99**

Area Under Precision-Recall Curve (AUC-PR):

```
PR curve: Plot of Precision vs. Recall
AUC-PR = ∫ Precision d(Recall)
```

- Better than AUC-ROC for imbalanced datasets
- Target: **>0.98**

1.67.4 2.4 Multi-Class Metrics

For multi-class classification (Normal, DoS, Probe, R2L, U2R):

Macro-Average:

```
Macro-Precision = (1/C) × Σ Precision_i
Macro-Recall = (1/C) × Σ Recall_i
Macro-F1 = (1/C) × Σ F1_i
```

- Equal weight to each class
- Useful when all classes equally important

Weighted-Average:

```
Weighted-Precision = Σ (n_i/N) × Precision_i
Weighted-Recall = Σ (n_i/N) × Recall_i
Weighted-F1 = Σ (n_i/N) × F1_i
```

- Weight by class frequency
- Reflects real-world class distribution

Per-Class Metrics: - Report Precision, Recall, F1 for each attack type - Identify which attacks are harder to detect

1.67.5 2.5 Implementation

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report,
                             roc_auc_score, roc_curve, matthews_corrcoef,
                             cohen_kappa_score)
import matplotlib.pyplot as plt
import seaborn as sns

def evaluate_detection_performance(y_true, y_pred, y_proba=None):
    """
    Comprehensive detection performance evaluation
    """

    results = {}

    # Basic metrics
    results['accuracy'] = accuracy_score(y_true, y_pred)
    results['precision'] = precision_score(y_true, y_pred, average='weighted')
    results['recall'] = recall_score(y_true, y_pred, average='weighted')
    results['f1_score'] = f1_score(y_true, y_pred, average='weighted')

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    results['confusion_matrix'] = cm

    # FPR, FNR (binary classification)
    if len(np.unique(y_true)) == 2:
        tn, fp, fn, tp = cm.ravel()
        results['fpr'] = fp / (fp + tn)
        results['fnr'] = fn / (fn + tp)
        results['specificity'] = tn / (tn + fp)

    # Advanced metrics
    results['mcc'] = matthews_corrcoef(y_true, y_pred)
```

```
    results['kappa'] = cohen_kappa_score(y_true, y_pred)

    # AUC-ROC (if probabilities available)
    if y_proba is not None:
        if len(np.unique(y_true)) == 2:
            results['auc_roc'] = roc_auc_score(y_true, y_proba[:, 1])
        else:
            results['auc_roc'] = roc_auc_score(y_true, y_proba,
                                               multi_class='ovr', average='weighted')

    # Classification report
    print(classification_report(y_true, y_pred))

    # Visualize confusion matrix
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('Confusion Matrix')
    plt.savefig('confusion_matrix.png', dpi=300, bbox_inches='tight')

    return results
```

1.68 3. Efficiency Metrics

1.68.1 3.1 Detection Latency

Definition: Time from packet arrival to detection decision

Measurement:

```
import time

def measure_latency(model, X_test, num_iterations=1000):
    """
    Measure detection latency
    """
    latencies = []

    for i in range(num_iterations):
        start_time = time.time()
        prediction = model.predict(X_test[i:i+1])
        end_time = time.time()

        latency = (end_time - start_time) * 1000  # Convert to ms
        latencies.append(latency)
```

```
    results = {
        'mean_latency_ms': np.mean(latencies),
        'median_latency_ms': np.median(latencies),
        'p95_latency_ms': np.percentile(latencies, 95),
        'p99_latency_ms': np.percentile(latencies, 99),
        'max_latency_ms': np.max(latencies)
    }

    return results
```

Targets: - Mean latency: **<100ms** (general scenarios) - P99 latency: **<200ms** - Critical systems: **<50ms**

1.68.2 3.2 Throughput

Definition: Number of packets/flows processed per second

Measurement:

```
def measure_throughput(model, X_test, duration=60):
    """
    Measure detection throughput
    """
    start_time = time.time()
    processed_count = 0

    while time.time() - start_time < duration:
        model.predict(X_test[:1000])  # Batch of 1000
        processed_count += 1000

    throughput = processed_count / duration

    print(f"Throughput: {throughput:.0f} packets/second")
    return throughput
```

Targets: - Minimum: **>10,000 packets/second** - High-speed networks: **>100,000 packets/second**

1.68.3 3.3 Computational Resource Usage

CPU Utilization:

```
import psutil
import os

def measure_cpu_usage(model, X_test, duration=60):
    """
    Measure CPU usage during detection
    """
    process = psutil.Process(os.getpid())
```

```
    cpu_percentages = []
    start_time = time.time()

    while time.time() - start_time < duration:
        cpu_percent = process.cpu_percent(interval=1)
        cpu_percentages.append(cpu_percent)

        # Perform detection
        model.predict(X_test[:100])

    avg_cpu = np.mean(cpu_percentages)
    max_cpu = np.max(cpu_percentages)

    return {'avg_cpu_%': avg_cpu, 'max_cpu_%': max_cpu}
```

Memory Usage:

```
def measure_memory_usage(model):
    """
    Measure memory footprint
    """
    process = psutil.Process(os.getpid())
    memory_info = process.memory_info()

    memory_mb = memory_info.rss / 1024 / 1024  # Convert to MB

    # Model size
    if hasattr(model, 'count_params'):
        params = model.count_params()
        model_size_mb = params * 4 / 1024 / 1024  # Assuming float32
    else:
        model_size_mb = None

    return {
        'process_memory_mb': memory_mb,
        'model_size_mb': model_size_mb
    }
```

Targets: - CPU usage: **<50%** (single core) - Memory: **<2GB** (edge devices), **<8GB** (servers) - Model size: **<100MB** (edge), **<500MB** (cloud)

1.68.4 3.4 Energy Consumption (for edge devices)

```
def estimate_energy_consumption(power_watts, duration_hours):
    """
    Estimate energy consumption
    """
    energy_wh = power_watts * duration_hours
```

```
    cost_per_kwh = 0.12  # USD
    cost = energy_wh / 1000 * cost_per_kwh

    return {
        'energy_wh': energy_wh,
        'estimated_cost_usd': cost
    }
```

Target: Minimal power consumption for battery-operated edge devices

---

1.69 4. Scalability Metrics

1.69.1 4.1 Performance vs. Network Size

Measurement:

```
def evaluate_scalability(model, base_size=100):
    """
    Evaluate performance across different network sizes
    """
    network_sizes = [100, 500, 1000, 5000, 10000, 50000, 100000]
    results = []

    for size in network_sizes:
        # Generate synthetic data for 'size' devices
        X_scaled = generate_traffic(num_devices=size)

        # Measure metrics
        start_time = time.time()
        predictions = model.predict(X_scaled)
        detection_time = time.time() - start_time

        throughput = len(X_scaled) / detection_time
        latency = detection_time / len(X_scaled) * 1000  # ms

        results.append({
            'network_size': size,
            'throughput_pps': throughput,
            'latency_ms': latency
        })

    df = pd.DataFrame(results)

    # Plot
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    ax1.plot(df['network_size'], df['throughput_pps'], marker='o')
```

```
    ax1.set_xlabel('Network Size (devices)')
    ax1.set_ylabel('Throughput (packets/s)')
    ax1.set_title('Throughput vs. Network Size')
    ax1.set_xscale('log')

    ax2.plot(df['network_size'], df['latency_ms'], marker='o', color='red')
    ax2.set_xlabel('Network Size (devices)')
    ax2.set_ylabel('Latency (ms)')
    ax2.set_title('Latency vs. Network Size')
    ax2.set_xscale('log')

    plt.tight_layout()
    plt.savefig('scalability_analysis.png', dpi=300)

    return df
```

Target: Near-linear scaling (latency increase <2× when network size increases 10×)

1.69.2 4.2 Distributed System Performance

Speedup:

```
Speedup = T_sequential / T_parallel
```

Efficiency:

```
Efficiency = Speedup / Number_of_Nodes
```

Target: Efficiency >80% with distributed deployment

1.70 5. Adaptability Metrics

1.70.1 5.1 Zero-Day Attack Detection Rate

Definition: Ability to detect previously unseen attack types

Measurement:

```
def evaluate_zero_day_detection(model, X_known, y_known, X_unknown, y_unknown):
    """
    Evaluate zero-day detection capability
    """
    # Train on known attacks
    model.fit(X_known, y_known)

    # Test on unknown (zero-day) attacks
    y_pred_unknown = model.predict(X_unknown)

    # Binary classification: Normal vs. Anomaly
```

```
    y_unknown_binary = (y_unknown != 'Normal').astype(int)
    y_pred_binary = (y_pred_unknown != 'Normal').astype(int)

    zero_day_detection_rate = recall_score(y_unknown_binary, y_pred_binary)
    zero_day_fpr = false_positive_rate(y_unknown_binary, y_pred_binary)

    results = {
        'zero_day_detection_rate': zero_day_detection_rate,
        'zero_day_fpr': zero_day_fpr
    }

    return results
```

Target: >95% detection rate for zero-day attacks

1.70.2 5.2 Concept Drift Handling

Measurement:

```
from river import drift

def evaluate_concept_drift_handling(model, data_stream):
    """
    Evaluate adaptation to concept drift
    """
    drift_detector = drift.ADWIN()

    accuracies = []
    drift_points = []

    for i, (x, y) in enumerate(data_stream):
        # Predict
        y_pred = model.predict([x])[0]
        correct = int(y_pred == y)

        # Update drift detector
        drift_detector.update(correct)

        if drift_detector.drift_detected:
            drift_points.append(i)
            print(f"Concept drift detected at sample {i}")
            # Model should adapt here (online learning)

        # Track accuracy over time
        accuracies.append(correct)

    # Performance degradation over time
    window_size = 1000
    windowed_accuracies = [
```

```
        np.mean(accuracies[i:i+window_size])
        for i in range(0, len(accuracies) - window_size, window_size)
    ]

    avg_degradation = (windowed_accuracies[0] - windowed_accuracies[-1]) / windowed_acc

    results = {
        'drift_points': drift_points,
        'num_drifts': len(drift_points),
        'accuracy_degradation': avg_degradation
    }

    return results
```

Target: Accuracy degradation <5% over 6-month period

1.70.3 5.3 Adaptation Time

Definition: Time to adapt to new attack patterns

Measurement:

```
def measure_adaptation_time(model, X_new_attack, y_new_attack):
    """
    Measure time to adapt to new attack type
    """
    start_time = time.time()

    # Incremental learning
    model.partial_fit(X_new_attack, y_new_attack)

    adaptation_time = time.time() - start_time

    # Test if adapted
    accuracy = model.score(X_new_attack, y_new_attack)

    return {
        'adaptation_time_seconds': adaptation_time,
        'post_adaptation_accuracy': accuracy
    }
```

Target: Adaptation in <1 hour with online learning

1.71 6. Explainability Metrics

1.71.1 6.1 Feature Importance Consistency

Measurement:

```python
def evaluate_feature_importance_consistency(model, X_test, num_runs=10):
    """
    Evaluate consistency of feature importance across runs
    """
    feature_importances = []

    for _ in range(num_runs):
        # Compute SHAP values
        explainer = shap.TreeExplainer(model)
        shap_values = explainer.shap_values(X_test)

        # Global feature importance
        importance = np.abs(shap_values).mean(axis=0)
        feature_importances.append(importance)

    feature_importances = np.array(feature_importances)

    # Compute consistency (low std = high consistency)
    consistency = 1 - (np.std(feature_importances, axis=0) / np.mean(feature_importance
    avg_consistency = np.mean(consistency)

    return {'avg_consistency': avg_consistency, 'per_feature_consistency': consistency}
```

Target: Average consistency >0.90

1.71.2 6.2 Explanation Fidelity

Definition: How faithfully explanations represent model behavior

Measurement:

```python
def evaluate_explanation_fidelity(model, X_test, lime_explainer):
    """
    Evaluate LIME explanation fidelity
    """
    fidelities = []

    for i in range(100):
        x = X_test[i]

        # Original prediction
        orig_pred = model.predict_proba([x])[0]

        # LIME explanation
        explanation = lime_explainer.explain_instance(x, model.predict_proba)

        # Local surrogate prediction
        surrogate_pred = explanation.local_pred[0]
```

```
        # Fidelity = similarity between original and surrogate
        fidelity = 1 - abs(orig_pred[1] - surrogate_pred)
        fidelities.append(fidelity)

    avg_fidelity = np.mean(fidelities)

    return {'avg_fidelity': avg_fidelity}
```

Target: Average fidelity >0.85

1.71.3 6.3 User Trust Score (Human Evaluation)

Methodology: - Recruit 10-20 security analysts - Present alerts with and without explanations - Measure trust, confidence, understanding

Questionnaire (5-point Likert scale): 1. I understand why this alert was generated 2. I trust the IDS's decision 3. The explanation helps me take appropriate action 4. The key features highlighted make sense

Target: Average score >4.0/5.0

1.72 7. Privacy Metrics

1.72.1 7.1 Differential Privacy Guarantee

Definition: Privacy loss parameter ε

```
Pr[M(D) ∈ S] ≤ e^ε × Pr[M(D') ∈ S] + δ
```

where: - M: privacy mechanism - D, D': neighboring datasets differing in one record - ε: privacy budget - δ: failure probability

Target: ε < 1.0, δ < 10^-5

1.72.2 7.2 Information Leakage

Measurement:

```
def measure_information_leakage(model, X_train, X_test):
    """
    Measure if model memorizes training data
    """
    # Membership inference attack
    train_losses = []
    test_losses = []

    for x_train in X_train[:1000]:
        loss = compute_loss(model, x_train)
        train_losses.append(loss)
```

```
    for x_test in X_test[:1000]:
        loss = compute_loss(model, x_test)
        test_losses.append(loss)

    # Lower loss on training data indicates memorization
    avg_train_loss = np.mean(train_losses)
    avg_test_loss = np.mean(test_losses)

    leakage_score = (avg_test_loss - avg_train_loss) / avg_test_loss

    return {'leakage_score': leakage_score}
```

Target: Leakage score <0.1 (minimal difference)

1.73 8. Comparative Evaluation Framework

1.73.1 8.1 Baseline Comparisons

Baselines: 1. Traditional ML: Random Forest, SVM, Decision Tree 2. Deep Learning: CNN, LSTM, GRU (standalone) 3. Existing IDS: Snort, Suricata, Zeek 4. Recent Research: State-of-the-art models from recent papers

1.73.2 8.2 Comparative Table Template

Model	Accuracy	Precision	Recall	F1	AUC-ROC	FPR	Latency (ms)	Throughput (pps)
Proposed CNN-LSTM-Transformer	-	-	-	-	-	-	-	
Random Forest	-	-	-	-	-	-	-	
SVM	-	-	-	-	-	-	-	
Standalone CNN	-	-	-	-	-	-	-	
Standalone LSTM	-	-	-	-	-	-	-	
Snort	-	-	-	-	-	-	-	
Recent Paper [X]	-	-	-	-	-	-	-	

1.74 9. Statistical Significance Testing

1.74.1 9.1 Cross-Validation

k-Fold Cross-Validation:

```
from sklearn.model_selection import cross_val_score

def cross_validate_model(model, X, y, k=10):
    """
    Perform k-fold cross-validation
    """
    scores = cross_val_score(model, X, y, cv=k, scoring='accuracy')

    results = {
        'mean_accuracy': np.mean(scores),
        'std_accuracy': np.std(scores),
        '95%_confidence_interval': (
            np.mean(scores) - 1.96 * np.std(scores) / np.sqrt(k),
            np.mean(scores) + 1.96 * np.std(scores) / np.sqrt(k)
        )
    }

    return results
```

1.74.2 9.2 Statistical Tests

Paired t-test:

```
from scipy.stats import ttest_rel

def compare_models_ttest(model1_scores, model2_scores):
    """
    Compare two models using paired t-test
    """
    t_stat, p_value = ttest_rel(model1_scores, model2_scores)

    alpha = 0.05
    if p_value < alpha:
        result = "Statistically significant difference"
    else:
        result = "No significant difference"

    return {
        't_statistic': t_stat,
        'p_value': p_value,
```

```
        'result': result
    }
```

McNemar's Test (for binary classification):

```
from statsmodels.stats.contingency_tables import mcnemar

def mcnemar_test(model1_preds, model2_preds, y_true):
    """
    McNemar's test for comparing classifiers
    """
    # Contingency table
    both_correct = np.sum((model1_preds == y_true) & (model2_preds == y_true))
    both_wrong = np.sum((model1_preds != y_true) & (model2_preds != y_true))
    model1_correct_model2_wrong = np.sum((model1_preds == y_true) & (model2_preds != y_
    model1_wrong_model2_correct = np.sum((model1_preds != y_true) & (model2_preds == y_

    table = [[both_correct, model1_correct_model2_wrong],
             [model1_wrong_model2_correct, both_wrong]]

    result = mcnemar(table, exact=True)

    return {
        'statistic': result.statistic,
        'p_value': result.pvalue
    }
```

1.75 Conclusion

This comprehensive evaluation framework ensures rigorous assessment of the proposed AI-IDS across all critical dimensions: detection performance, efficiency, scalability, adaptability, explainability, and privacy. The combination of quantitative metrics, statistical testing, and human evaluation provides confidence in the system's real-world viability.

Key Evaluation Principles: - Multi-faceted assessment (not just accuracy) - Comparison against strong baselines - Statistical significance testing - Real-world performance simulation - Human-in-the-loop evaluation for explainability

Success Criteria Summary: - Detection: Accuracy >98%, F1 >97%, FPR <1% - Efficiency: Latency <100ms, Throughput >10K pps - Scalability: Linear scaling to 100K devices - Adaptability: Zero-day detection >95% - Explainability: User trust score >4.0/5.0 - Privacy: $\varepsilon < 1.0$ in federated scenarios

---

Next: Refer to `../timeline/gantt_chart.md` for research timeline and `../resources/budget.md` for resource allocation details. # 4-Year PhD Research Timeline

Gantt Chart and Quarterly Breakdown

Detailed Schedule for AI-Based IDS Development (48 Months)
Last Updated: December 22, 2025

1.76 Table of Contents

1.77 1. Timeline Overview

1.77.1 1.1 High-Level Phases

Phase	Duration	Key Focus	Major Deliverables
Year 1	Months 1-12	Foundation & Design	Literature review, Algorithm design, Baseline models, Comprehensive exam
Year 2	Months 13-24	Framework Development	Complete framework, Testbed deployment, Distributed IDS
Year 3	Months 25-36	Validation & Optimization	Extensive experiments, Publications, Framework release
Year 4	Months 37-48	Thesis Completion	Additional experiments, Thesis writing, Defense

1.77.2 1.2 Publication Timeline

Quarter	Target Venue	Paper Type	Topic
Q4 Year 1	Workshop/Conference	Short paper	Preliminary CNN-LSTM results
Q2 Year 2	Top Conference (IEEE INFOCOM)	Full paper	Hybrid architecture & adaptive learning

Quarter	Target Venue	Paper Type	Topic
Q4 Year 2	Journal (IEEE TIFS)	Journal paper	Distributed IDS framework
Q2 Year 3	Top Conference (USENIX Security)	Full paper	Explainable AI for IDS
Q4 Year 3	Journal (ACM Computing Surveys)	Survey/Tutorial	Comprehensive AI-IDS review

---

1.78 2. Year 1: Foundation and Design (Months 1-12)

1.78.1 Q1 (Months 1-3): Literature Review & Coursework

Activities: - [ ] Week 1-2: Enroll in PhD program, meet advisor and committee - [ ] Week 3-6: Systematic literature review (100+ papers) - Traditional IDS systems - Deep learning for cybersecurity - Next-generation network security - Recent advances (Transformers, Federated Learning, XAI) - [ ] Week 7-10: PhD coursework - Advanced Machine Learning - Network Security and Cryptography - Research Methodology - [ ] Week 11-12: Form dissertation committee

Research Tasks: - [ ] Set up development environment - Install TensorFlow, PyTorch, Scikit-learn - Configure GPU workstation - Set up version control (Git/GitHub) - [ ] Download and explore benchmark datasets - NSL-KDD - CICIDS2017 - UNSW-NB15 - IoT-23 - [ ] Implement baseline models - Random Forest - SVM - Basic CNN - Basic LSTM

Deliverables: - □ Literature review draft (30-40 pages) - □ Committee formation complete - □ Baseline model results (accuracy, latency) - □ Research proposal presentation

Milestones: - [M1] Committee approved (Month 3)

1.78.2 Q2 (Months 4-6): Algorithm Design & Initial Prototyping

Activities: - [ ] Week 13-16: Design CNN-LSTM hybrid architecture - Architecture specification - Hyperparameter tuning strategy - Implementation in TensorFlow/PyTorch - [ ] Week 17-20: Preliminary experiments - Train CNN-LSTM on NSL-KDD - Train CNN-LSTM on CICIDS2017 - Compare against baselines - Analyze results - [ ] Week 21-24: Feature engineering and optimization - Feature selection experiments - Data preprocessing pipeline - Model optimization (pruning, quantization exploration)

Research Tasks: - [ ] Implement data preprocessing pipeline - Traffic parsing - Feature extraction - Normalization and encoding - Class balancing (SMOTE) - [ ] Develop

evaluation framework - Metrics implementation (accuracy, precision, recall, F1, AUC-ROC) - Visualization tools - Latency measurement

Deliverables: - □ CNN-LSTM architecture specification - □ Initial experimental results - □ Technical report (10-15 pages)

Milestones: - [M2] CNN-LSTM prototype working (Month 6)

1.78.3 Q3 (Months 7-9): Transformer & Ensemble Implementation

Activities: - [ ] Week 25-28: Implement Transformer network for IDS - Self-attention mechanism - Positional encoding - Multi-head attention - Training and evaluation - [ ] Week 29-32: Develop autoencoder for anomaly detection - Standard autoencoder - Variational autoencoder (VAE) - Adversarial autoencoder - Threshold optimization - [ ] Week 33-36: Create ensemble methods - Random Forest (100-500 trees) - XGBoost with hyperparameter tuning - Voting and stacking ensembles - Calibration

Research Tasks: - [ ] Comparative evaluation - CNN-LSTM vs. Transformer vs. Autoencoder - Individual models vs. ensemble - Performance metrics comparison - Statistical significance testing - [ ] Preliminary XAI integration - SHAP for Random Forest and XGBoost - Initial attention visualization for Transformer

Deliverables: - □ Transformer and Autoencoder implementations - □ Ensemble model results - □ Comparison report

Milestones: - [M3] All core models implemented (Month 9)

1.78.4 Q4 (Months 10-12): Comprehensive Exam & First Publication

Activities: - [ ] Week 37-40: Comprehensive exam preparation - Study core concepts - Prepare presentation - Practice Q&A - [ ] Week 41: **Comprehensive Exam** - [ ] Week 42-48: First paper writing - Introduction and related work - Methodology description - Experimental results - Discussion and conclusion - Submission to workshop or conference

Research Tasks: - [ ] Integrate XAI components - SHAP implementation - LIME implementation - Attention visualization - Feature importance analysis - [ ] Initial comparison with baselines - Snort rules-based detection - Literature models - Performance benchmarking

Deliverables: - □ Pass comprehensive exam - □ First paper submitted - □ Annual progress report - □ XAI integration complete

Milestones: - [M4] Comprehensive exam passed (Month 11) - [M5] First paper submitted (Month 12)

1.79 3. Year 2: Framework Development (Months 13-24)

1.79.1 Q1 (Months 13-15): Adaptive Learning Integration

Activities: - [ ] Week 49-52: Implement reinforcement learning - Deep Q-Network (DQN) - Actor-Critic (A3C) - Proximal Policy Optimization (PPO) - Reward engineering - Training environment setup - [ ] Week 53-56: Develop online learning mechanisms - Incremental learning (SGD, mini-batch) - Concept drift detection (ADWIN, Page-Hinkley) - Adaptive model updates - [ ] Week 57-60: Transfer learning experiments - Train on NSL-KDD, test on CICIDS2017 - Train on CICIDS2017, test on UNSW-NB15 - Domain adaptation techniques - Cross-dataset evaluation

Research Tasks: - [ ] Extended evaluation on UNSW-NB15 - 9-class classification - Novel attack detection - Cross-validation - [ ] Meta-learning exploration - Few-shot learning - MAML (Model-Agnostic Meta-Learning)

Deliverables: - □ Adaptive learning module - □ Workshop paper on online learning for IDS - □ Transfer learning results

Milestones: - [M6] Reinforcement learning working (Month 15)

1.79.2 Q2 (Months 16-18): Distributed Architecture Design

Activities: - [ ] Week 61-64: Design federated learning framework - FedAvg algorithm implementation - Client-server architecture - Communication protocol - Privacy mechanisms (differential privacy, secure aggregation) - [ ] Week 65-68: Implement model compression - Quantization (INT8, INT16) - Pruning (magnitude-based, structured) - Knowledge distillation (teacher-student) - MobileNet-style architecture for edge - [ ] Week 69-72: Develop edge deployment strategy - TensorFlow Lite conversion - ONNX model export - Docker containers - Kubernetes deployment configs

Research Tasks: - [ ] Privacy mechanism integration - Differential privacy noise addition - Secure multi-party computation - Homomorphic encryption (exploration) - [ ] Simulated distributed experiments - 10, 50, 100 clients - Non-IID data distribution - Communication cost analysis

Deliverables: - □ Distributed IDS prototype - □ Model compression results - □ Privacy analysis report

Milestones: - [M7] Federated learning framework complete (Month 18)

1.79.3 Q3 (Months 19-21): Testbed Deployment Preparation

Activities: - [ ] Week 73-76: Acquire hardware - Order Raspberry Pi 4 devices (10x) - Order OpenFlow switches (5x) - Order IoT devices (50x) - Order high-performance servers (2x) - [ ] Week 77-80: Set up SDN testbed - Install ONOS controller - Configure OpenFlow switches - Create network topology in Mininet - Test SDN functionality -

[ ] Week 81-84: Deploy IoT devices - Set up IoT gateway - Configure MQTT broker - Connect devices - Generate IoT traffic

Research Tasks: - [ ] Configure monitoring infrastructure - Prometheus for metrics - Grafana for dashboards - ELK stack for logging - Network traffic capture tools - [ ] Develop testbed orchestration scripts - Automated deployment - Configuration management - Experiment automation

Deliverables: - □ Operational testbed - □ Testbed documentation - □ Initial connectivity tests

Milestones: - [M8] Testbed operational (Month 21)

1.79.4 Q4 (Months 22-24): Initial Testbed Experiments

Activities: - [ ] Week 85-88: Deploy IDS on testbed - Edge node deployment (Raspberry Pi) - Fog node deployment (servers) - Cloud coordinator deployment - Integration testing - [ ] Week 89-92: Conduct real-time detection experiments - Normal traffic baseline - DoS attack simulation - Port scanning simulation - Web attack simulation - Botnet traffic replay - [ ] Week 93-96: Performance profiling and optimization - Latency measurement and optimization - Throughput testing - Resource usage profiling - Bottleneck identification

Research Tasks: - [ ] Second major publication writing - Journal paper on distributed AI-IDS - Comprehensive experimental results - Performance analysis - Comparison with centralized approaches - [ ] Prepare for Year 3 experiments

Deliverables: - □ Testbed deployment complete - □ Initial real-world results - □ Journal paper submitted (IEEE TIFS/TMC) - □ Annual progress report

Milestones: - [M9] Successful testbed deployment (Month 24) - [M10] Second major publication submitted (Month 24)

1.80 4. Year 3: Validation and Optimization (Months 25-36)

1.80.1 Q1 (Months 25-27): Extensive Experiments

Activities: - [ ] Week 97-100: Large-scale experiments on all datasets - NSL-KDD (5-class classification) - CICIDS2017 (14 attack types) - UNSW-NB15 (9 attack categories) - IoT-23 (botnet detection) - Custom 5G dataset - [ ] Week 101-104: Cross-dataset evaluation - Train on X, test on Y (all combinations) - Transfer learning performance - Generalization analysis - [ ] Week 105-108: Ablation studies - Remove CNN: LSTM only - Remove LSTM: CNN only - Remove attention: standard CNN-LSTM - Remove ensemble: single model - Impact of each component on performance

Research Tasks: - [ ] Adversarial robustness testing - FGSM attacks - PGD attacks - C&W attacks - Adversarial training - [ ] Statistical significance testing - T-tests between models - McNemar's test - Confidence intervals

Deliverables: - □ Comprehensive experimental results - □ Ablation study report - □ Statistical analysis

Milestones: - [M11] All experiments complete (Month 27)

1.80.2 Q2 (Months 28-30): 5G/IoT Specific Validation

Activities: - [ ] Week 109-112: Custom 5G dataset collection - Set up free5GC or Open5GS - Generate normal traffic - Simulate attacks (signaling storm, slice isolation violation) - Label and preprocess data - [ ] Week 113-116: IoT-23 malware detection experiments - Feature extraction from PCAPs - Training on IoT-specific features - Mirai, Torii, Gafgyt detection - False positive analysis - [ ] Week 117-120: Edge computing performance analysis - Latency on Raspberry Pi - Memory footprint on edge devices - Energy consumption estimation - Edge-fog-cloud coordination efficiency

Research Tasks: - [ ] Real-world case studies - Collaborate with industry partner (if available) - Deploy in real network environment - Collect feedback from operators - [ ] Third publication (conference paper) - NGN-specific IDS results - 5G and IoT validation - Practical deployment insights

Deliverables: - □ 5G dataset created and results - □ IoT-23 comprehensive analysis - □ Conference paper submitted (USENIX Security, CCS, NDSS)

Milestones: - [M12] 5G/IoT validation complete (Month 30)

1.80.3 Q3 (Months 31-33): Performance Optimization

Activities: - [ ] Week 121-124: Model compression and acceleration - Aggressive quantization (INT4, binarization) - Advanced pruning (lottery ticket hypothesis) - Neural architecture search (AutoML) - TensorRT optimization - [ ] Week 125-128: Latency optimization - Profiling with NVIDIA Nsight - Code optimization (vectorization, caching) - Pipeline parallelism - Target: <100ms for critical, <50ms for edge - [ ] Week 129-132: Scalability experiments - 100 devices - 1,000 devices - 10,000 devices - 100,000 devices (simulation) - Load testing

Research Tasks: - [ ] Energy efficiency analysis - Power consumption on Raspberry Pi - Battery life estimation for IoT devices - Green AI considerations - [ ] Cost-benefit analysis - Computational cost vs. detection accuracy trade-off - ROI calculation for deployment

Deliverables: - □ Optimized framework (latency <100ms, throughput >10K pps) - □ Scalability report - □ Energy efficiency analysis

Milestones: - [M13] Performance targets met (Month 33)

1.80.4 Q4 (Months 34-36): Publication Push & Framework Release

Activities: - [ ] Week 133-136: Major journal paper writing - Comprehensive system description - All experimental results - Extensive comparison with baselines - Discussion of limitations and future work - Submit to IEEE TIFS or ACM Computing Surveys - [ ] Week 137-140: Conference papers for top venues - Extract focused contributions - Write concise papers (10-12 pages) - Submit to IEEE S&P, USENIX Security, CCS, NDSS - [ ] Week 141-144: Prepare open-source release - Code cleanup and documentation - Create README with setup instructions - Prepare example notebooks - Create Docker images - Write tutorials

Research Tasks: - [ ] Documentation and tutorials - API documentation - Architecture guide - Deployment guide - Video tutorials - [ ] Community engagement - GitHub repository creation - arXiv preprint - Blog posts - Social media announcements

Deliverables: - □ Framework v1.0 released (GitHub) - □ Major journal paper submitted - □ 2-3 conference papers submitted - □ Annual progress report - □ Comprehensive documentation

Milestones: - [M14] Framework publicly released (Month 36) - [M15] 3+ publications submitted (Month 36)

1.81 5. Year 4: Thesis Completion (Months 37-48)

1.81.1 Q1 (Months 37-39): Additional Experiments

Activities: - [ ] Week 145-148: Address reviewer feedback - Revise submitted papers - Run additional experiments as requested - Resubmit papers - [ ] Week 149-152: Conduct additional experiments - Fill any experimental gaps - Test edge cases - Validate assumptions - [ ] Week 153-156: User studies for explainability - Recruit 10-20 security analysts - Prepare questionnaire (5-point Likert scale) - Conduct interviews - Analyze feedback

Research Tasks: - [ ] Industry validation (if possible) - Pilot deployment with industry partner - Collect operational feedback - Measure real-world performance - [ ] Camera-ready versions - Incorporate reviewer comments - Final polish on accepted papers

Deliverables: - □ Revised manuscripts - □ Additional experimental results - □ User study report

Milestones: - [M16] User studies complete (Month 39)

1.81.2 Q2 (Months 40-42): Thesis Writing

Activities: - [ ] Week 157-160: Write thesis chapters - Chapter 1: Introduction - Chapter 2: Literature Review - Chapter 3: Methodology - Chapter 4: Implementation - [ ] Week 161-164: Continue writing - Chapter 5: Experimental Evaluation - Chapter 6: Results and Discussion - Chapter 7: Conclusion and Future Work - [ ] Week 165-168: Create figures and tables - All experimental plots - Architecture diagrams - Tables of results - Format according to university guidelines

Research Tasks: - [ ] Integrate published papers - Adapt paper content for thesis - Ensure consistency across chapters - Update references - [ ] Comprehensive proofreading

Deliverables: - □ Complete thesis draft (200-300 pages) - □ All figures and tables finalized

Milestones: - [M17] Thesis draft complete (Month 42)

1.81.3 Q3 (Months 43-45): Thesis Review & Defense Preparation

Activities: - [ ] Week 169-172: Committee review - Distribute thesis to committee members - Collect initial feedback - Schedule review meetings - [ ] Week 173-176: Revisions based on feedback - Address committee comments - Clarify ambiguous sections - Add missing details - Polish writing - [ ] Week 177-180: Prepare defense presentation - Create slides (45-60 minutes) - Prepare demo videos - Anticipate questions - Practice presentation

Research Tasks: - [ ] Prepare for Q&A - List potential questions - Prepare answers - Practice with lab members - [ ] Final thesis polishing

Deliverables: - □ Revised thesis (incorporating committee feedback) - □ Defense slides - □ Demo materials

Milestones: - [M18] Thesis ready for defense (Month 45)

1.81.4 Q4 (Months 46-48): Defense & Final Submission

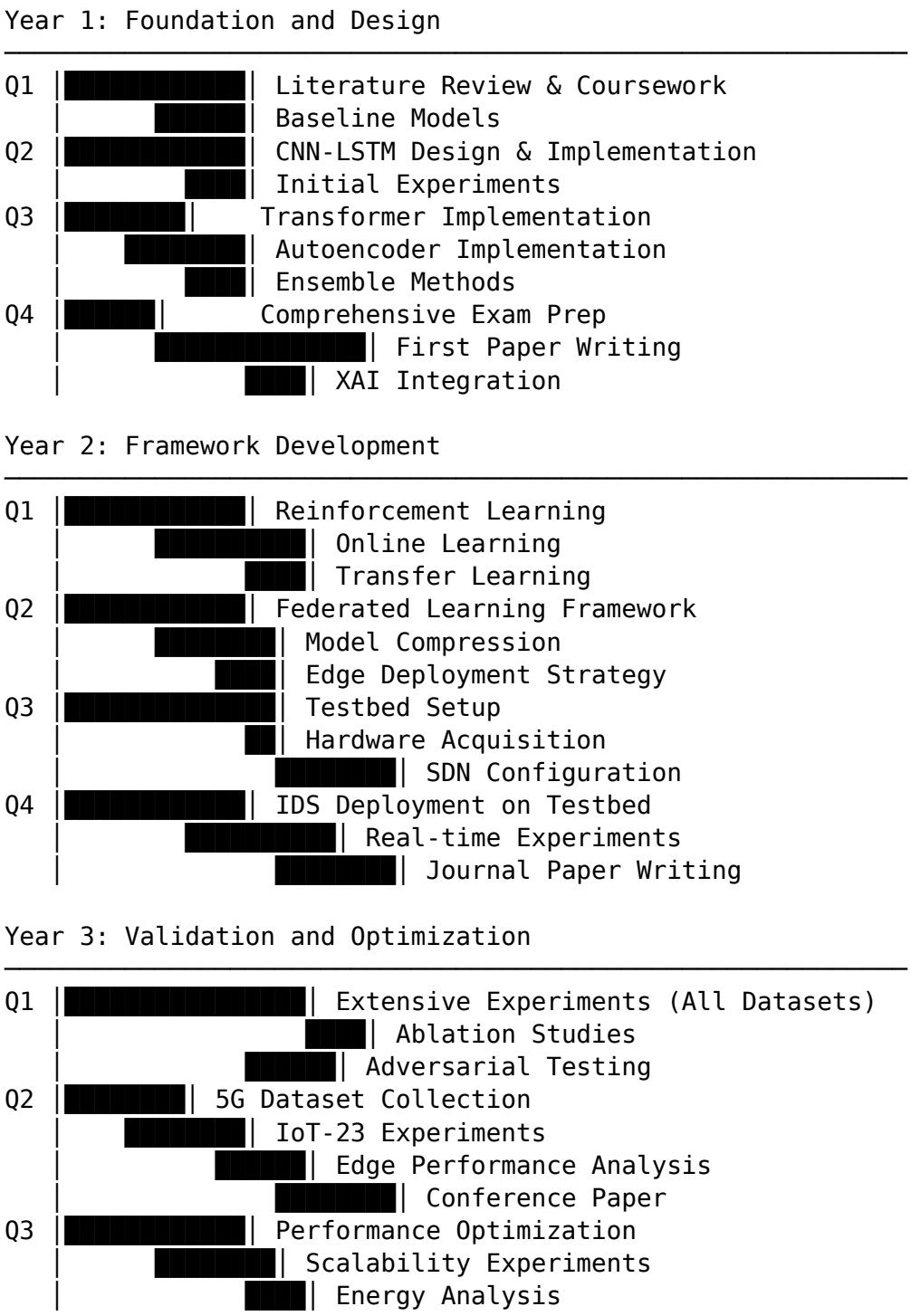
Activities: - [ ] Week 181-184: Dissertation defense - Schedule defense date - Public presentation - Committee Q&A - Pass/revisions/fail decision - [ ] Week 185-188: Final revisions (if any) - Incorporate defense feedback - Final proofreading - Format check - Plagiarism check - [ ] Week 189-192: Submit final dissertation - University submission - Publish final papers - Archive code and data - Celebrate completion! □

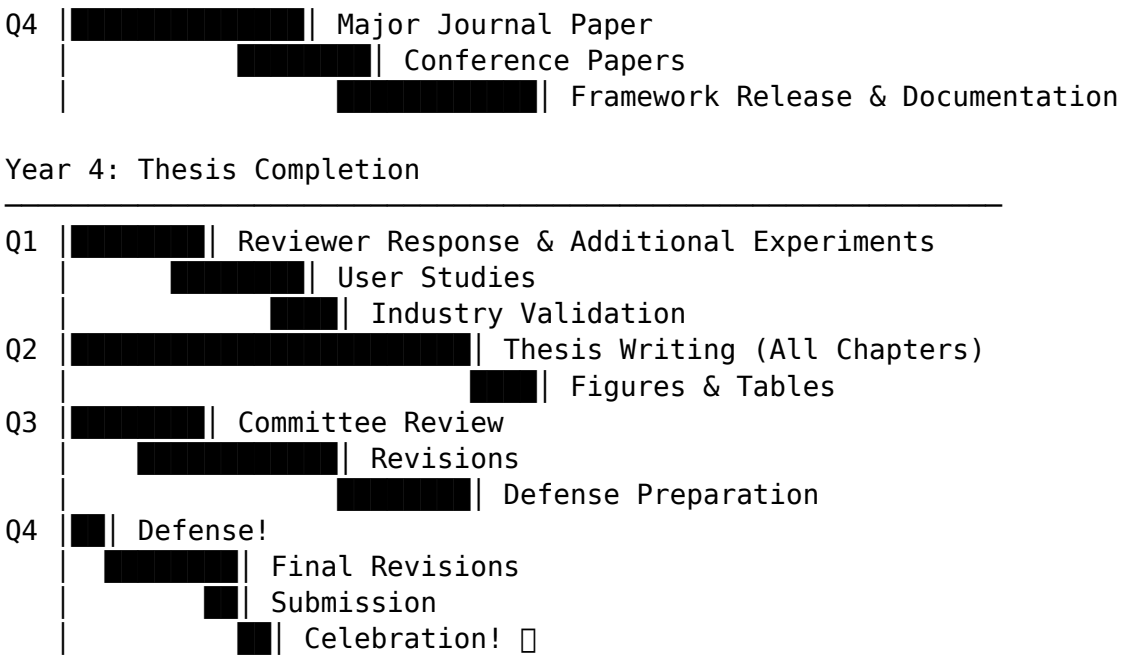
Research Tasks: - [ ] Wrap up loose ends - Finish any pending papers - Archive all experimental data - Document lessons learned - [ ] Job search / postdoc applications

Deliverables: - □ Successfully defend dissertation - □ Submit final dissertation - □ Published papers (5-10 total) - □ PhD degree awarded!

Milestones: - [M19] Defense passed (Month 46) - [M20] Final dissertation submitted (Month 48) - **[M21] PhD DEGREE AWARDED! **

1.82 6. Gantt Chart Visualization





1.83 7. Dependencies and Critical Path

1.83.1 7.1 Key Dependencies

Year 1 → Year 2: - Comprehensive exam must pass before advancing - Baseline models needed for comparison

Year 2 → Year 3: - Testbed must be operational for real-world experiments - Framework core must be complete for optimization

Year 3 → Year 4: - Experimental results needed for thesis - Publications needed for defense

1.83.2 7.2 Critical Path

```
Literature Review (M1-3)
  ↓
Algorithm Design (M4-6)
  ↓
Core Models Implementation (M7-12)
  ↓
Comprehensive Exam (M11) ← CRITICAL MILESTONE
  ↓
Distributed Framework (M13-18)
  ↓
Testbed Deployment (M19-24) ← CRITICAL MILESTONE
  ↓
Extensive Experiments (M25-33)
```

```
  ↓
Framework Release (M36) ← CRITICAL MILESTONE
  ↓
Thesis Writing (M40-42)
  ↓
Defense (M46) ← FINAL MILESTONE
```

1.83.3 7.3 Risk Mitigation

Risks: - Hardware delays → Order early, have backup cloud option - Experiment failures → Allocate buffer time - Publication rejections → Submit to multiple venues - Testbed issues → Extensive testing before experiments

---

1.84 Conclusion

This detailed 4-year timeline provides a structured path from literature review to PhD completion. The quarterly breakdown ensures steady progress with clear deliverables, while the Gantt chart visualization helps identify dependencies and manage the critical path. Regular milestones enable progress tracking and timely adjustments.

Success Indicators: - 5-10 published papers in top venues - Complete, tested, and released open-source framework - Comprehensive evaluation across multiple datasets - Successfully defended dissertation - **PhD degree awarded in 48 months**

Next: Refer to `milestones.md` for detailed milestone descriptions and `../resources/budget.md` for resource allocation timeline. # PhD Research Milestones ## Key Achievements and Checkpoints

Major Milestones for 4-Year PhD Program
Last Updated: December 22, 2025

1.85 Overview

This document outlines the key milestones for the PhD research on AI-Based Intrusion Detection Systems for Next-Generation Networks. Each milestone represents a significant achievement that marks progress toward the PhD degree.

1.86 Year 1 Milestones

1.86.1 M1: Committee Formation (Month 3)

Target Date: End of Month 3
Description: Dissertation committee formed with 4-5 faculty members, including advisor and external member
Criteria for Success: - Committee members identified and agreed - Initial research direction approved - Meeting schedule established **Deliverables:** - Committee formation documentation - Initial research plan

---

1.86.2 M2: CNN-LSTM Prototype Working (Month 6)

Target Date: End of Month 6
Description: First hybrid deep learning model implemented and tested
Criteria for Success: - Model architecture implemented in TensorFlow/PyTorch - Training on NSL-KDD and CICIDS2017 complete - Accuracy >95% on test sets - Baseline comparison documented **Deliverables:** - Working code repository - Technical report with results - Performance metrics

---

1.86.3 M3: All Core Models Implemented (Month 9)

Target Date: End of Month 9
Description: CNN-LSTM, Transformer, Autoencoder, and Ensemble methods all operational
Criteria for Success: - All 4 model types implemented - Comparative evaluation complete - Initial XAI integration (SHAP, LIME) **Deliverables:** - Complete model suite - Comparison report - Visualization dashboard

---

1.86.4 M4: Comprehensive Exam Passed (Month 11)

Target Date: Month 11
Description: Successfully complete PhD comprehensive examination
Criteria for Success: - Written exam passed (if required) - Oral presentation delivered - Committee approval to advance to candidacy **Deliverables:** - Exam presentation slides - Candidacy status **CRITICAL MILESTONE** - Required to continue in PhD program

---

1.86.5 M5: First Paper Submitted (Month 12)

Target Date: End of Month 12
Description: First research paper submitted to workshop or conference

Criteria for Success: - Paper written (6-10 pages) - Results on CNN-LSTM hybrid - Submitted to workshop (e.g., IEEE workshops) or mid-tier conference **Deliverables:** - Submitted manuscript - Submission confirmation

---

1.87 Year 2 Milestones

1.87.1 M6: Reinforcement Learning Working (Month 15)

Target Date: End of Month 15
Description: Adaptive learning mechanisms implemented and validated
Criteria for Success: - DQN or PPO agent trained successfully - Demonstrated improvement in dynamic threat response - Online learning capability functional **Deliverables:** - RL agent implementation - Adaptation experiments report

---

1.87.2 M7: Federated Learning Framework Complete (Month 18)

Target Date: End of Month 18
Description: Distributed IDS architecture fully developed
Criteria for Success: - FedAvg implementation working with 10+ clients - Model compression achieved (4-8× reduction) - Privacy guarantees (differential privacy) integrated **Deliverables:** - Federated learning codebase - Privacy analysis report - Distributed experiments results

---

1.87.3 M8: Testbed Operational (Month 21)

Target Date: End of Month 21
Description: Physical hardware testbed deployed and functional
Criteria for Success: - 5 SDN switches + ONOS controller working - 10 Raspberry Pi edge nodes deployed - 50 IoT devices connected - Network monitoring (Grafana, Prometheus) active **Deliverables:** - Testbed documentation - Network topology diagrams - Connectivity test results

---

1.87.4 M9: Successful Testbed Deployment (Month 24)

Target Date: End of Month 24
Description: IDS deployed on testbed with real-time detection capability
Criteria for Success: - Edge-fog-cloud architecture deployed - Real-time detection latency <100ms - Successful attack simulations (DoS, port scan, web attacks) **Deliverables:** - Deployment guide - Real-world experimental results - Performance benchmark report

---

1.87.5 M10: Second Major Publication Submitted (Month 24)

Target Date: End of Month 24
Description: Journal paper on distributed IDS submitted
Criteria for Success: - Full paper (8-12 pages for conference, 15-20 for journal) - Comprehensive distributed IDS evaluation - Submitted to top venue (IEEE TIFS, IEEE TMC, or top conference) **Deliverables:** - Journal manuscript - Submission confirmation

---

1.88 Year 3 Milestones

1.88.1 M11: All Experiments Complete (Month 27)

Target Date: End of Month 27
Description: Extensive evaluation on all datasets finished
Criteria for Success: - Experiments on 5 datasets (NSL-KDD, CICIDS2017, UNSW-NB15, IoT-23, 5G custom) - Cross-dataset validation complete - Ablation studies finished - Statistical significance tests performed **Deliverables:** - Complete experimental results - Statistical analysis report - Comparison tables and figures

---

1.88.2 M12: 5G/IoT Validation Complete (Month 30)

Target Date: End of Month 30
Description: Next-generation network specific validation completed
Criteria for Success: - Custom 5G dataset collected (1M+ flows) - IoT-23 malware detection >95% accuracy - Edge computing performance validated (<100ms latency on Raspberry Pi) **Deliverables:** - 5G dataset and documentation - IoT malware detection report - Edge performance analysis

---

1.88.3 M13: Performance Targets Met (Month 33)

Target Date: End of Month 33
Description: All performance optimization goals achieved
Criteria for Success: - Detection latency <100ms (mean), <200ms (p99) - Throughput >10K packets/second - Accuracy >98%, Precision >97%, Recall >98%, F1 >97% - False positive rate <1% **Deliverables:** - Optimized model artifacts - Performance benchmark report - Optimization documentation

---

1.88.4 M14: Framework Publicly Released (Month 36)

Target Date: End of Month 36
Description: Open-source AI-IDS framework published on GitHub

Criteria for Success: - GitHub repository created and populated - Comprehensive README and documentation - Docker images published - Example notebooks and tutorials - Community engagement started (stars, forks, issues) **Deliverables:** - GitHub repository: github.com/[username]/ai-ids-ngn - Documentation website - Tutorial videos **MAJOR MILESTONE** - Research contribution to community

---

1.88.5 M15: 3+ Publications Submitted (Month 36)

Target Date: End of Month 36
Description: Multiple research papers submitted to top venues
Criteria for Success: - At least 3 papers submitted (mix of conference and journal) - At least 1 paper accepted/published - Papers cover different aspects (architecture, distributed, explainability) **Deliverables:** - Submitted manuscripts - Acceptance letters (for accepted papers) - Preprints on arXiv

---

1.89 Year 4 Milestones

1.89.1 M16: User Studies Complete (Month 39)

Target Date: End of Month 39
Description: Explainability evaluation with human subjects finished
Criteria for Success: - 10-20 security analysts recruited - Questionnaire administered - User trust score >4.0/5.0 - Qualitative feedback collected and analyzed **Deliverables:** - User study report - Survey results and analysis - Recommendations for UX improvement

---

1.89.2 M17: Thesis Draft Complete (Month 42)

Target Date: End of Month 42
Description: Complete dissertation written and ready for committee review
Criteria for Success: - All chapters written (7 chapters, 200-300 pages) - All figures and tables created - References complete (100+ citations) - Formatted according to university guidelines **Deliverables:** - Complete thesis draft (PDF) - Chapter-by-chapter breakdown **MAJOR MILESTONE** - Thesis ready for review

---

1.89.3 M18: Thesis Ready for Defense (Month 45)

Target Date: End of Month 45
Description: Revised thesis incorporating committee feedback
Criteria for Success: - All committee comments addressed - Defense presentation

prepared (45-60 minutes) - Defense date scheduled - Committee approval to defend **Deliverables:** - Final thesis draft - Defense presentation slides - Demo materials

---

1.89.4 M19: Defense Passed (Month 46)

Target Date: Month 46
Description: Successfully defend PhD dissertation
Criteria for Success: - Public presentation delivered - Committee questions answered satisfactorily - Pass vote from committee (unanimous or majority) - Minor revisions accepted (if any) **Deliverables:** - Defense certificate - Committee signatures - Passed candidacy **CRITICAL MILESTONE** - Near completion!

1.89.5 M20: Final Dissertation Submitted (Month 48)

Target Date: Month 48
Description: Final dissertation submitted to university
Criteria for Success: - All defense revisions incorporated - Final proofreading complete - Format check passed - Submitted to graduate school - University approval received **Deliverables:** - Final dissertation (PDF) - Submission receipt - Graduation clearance

---

1.89.6 M21: PhD DEGREE AWARDED (Month 48)

Target Date: End of Month 48
**Description: GRADUATION - DOCTOR OF PHILOSOPHY AWARDED! **
Criteria for Success: - All degree requirements completed - Dissertation approved and archived - Graduation ceremony attended - Diploma received **Deliverables:** - **PhD DEGREE** - Published dissertation - 5-10 peer-reviewed publications - Open-source framework with community adoption **ULTIMATE MILESTONE** - PhD Completed!

1.90 Summary Table

Milestone	Month	Type	Critical?
M1: Committee Formation	3	Administrative	
M2: CNN-LSTM Prototype	6	Technical	
M3: All Core Models	9	Technical	
M4: Comprehensive Exam	11	Academic	CRITICAL
M5: First Paper Submitted	12	Publication	
M6: RL Working	15	Technical	

Milestone	Month	Type	Critical?
M7: Federated Framework	18	Technical	✓
M8: Testbed Operational	21	Infrastructure	✓
M9: Testbed Deployment	24	Technical	
M10: Second Paper Submitted	24	Publication	
M11: All Experiments	27	Research	
M12: 5G/IoT Validation	30	Research	
M13: Performance Targets	33	Technical	✓
M14: Framework Released	36	Impact	✓ MAJOR
M15: 3+ Papers Submitted	36	Publication	
M16: User Studies	39	Research	
M17: Thesis Draft	42	Academic	✓ MAJOR
M18: Defense Ready	45	Academic	
M19: Defense Passed	46	Academic	✓ CRITICAL
M20: Final Submission	48	Administrative	
M21: PhD AWARDED	48	**COMPLETION**	✓ **ULTIMATE**

1.91 Tracking and Reporting

1.91.1 Progress Tracking

- Monthly progress reports to advisor
- Quarterly committee meetings
- Annual comprehensive review

1.91.2 Milestone Documentation

For each milestone: 1. **Pre-Milestone:** Plan and prepare (1-2 weeks before) 2. **Milestone Event:** Execute and achieve 3. **Post-Milestone:** Document and reflect (1 week after)

1.91.3 Risk Management

- Buffer time built into schedule (2-4 weeks per phase)
- Alternative plans for critical milestones
- Regular risk assessments with advisor

1.92 Conclusion

These 21 milestones provide a clear roadmap from PhD enrollment to graduation. Regular achievement of these milestones ensures steady progress and on-time com-

pletion of the doctoral degree. The mix of technical, academic, and publication milestones ensures comprehensive development as a researcher.

Keys to Success: - □ Stay on track with quarterly goals - □ Regular communication with advisor and committee - □ Document progress continuously - □ Celebrate achievements along the way!

End Goal: Dr. [Your Name], PhD in Computer Science - Expert in AI-Based Network Security! □

References: See `gantt_chart.md` for detailed timeline and `../resources/budget.md` for resource allocation per milestone. # Tools and Frameworks ## Comprehensive Software Stack for AI-IDS Development

Complete Technology Stack and Tools
Last Updated: December 22, 2025

1.93 Table of Contents

1.94 1. Deep Learning Frameworks

1.94.1 1.1 TensorFlow

Version: 2.13+ (Latest stable)
Purpose: Primary deep learning framework for model development
Usage: - CNN, LSTM, Transformer implementation - Model training and inference - TensorFlow Lite for edge deployment - TensorBoard for visualization

Installation:

```
pip install tensorflow==2.13.0 tensorflow-gpu==2.13.0
```

Key APIs: - `tf.keras`: High-level API for model building - `tf.data`: Efficient data pipeline - `tf.distribute`: Distributed training - `tf.lite`: Model conversion for edge devices

---

1.94.2 1.2 PyTorch

Version: 2.0+ (Latest stable)
Purpose: Alternative deep learning framework, preferred for research
Usage: - Flexible model prototyping - Custom loss functions and layers - ONNX export for interoperability - Dynamic computational graphs

Installation:

```
pip install torch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2
```

Key Libraries: - `torch.nn`: Neural network modules - `torch.optim`: Optimization algorithms - `torchvision`: Computer vision utilities - `torch.distributed`: Distributed training

---

1.94.3 1.3 Keras

Version: Included with TensorFlow 2.x
Purpose: High-level neural networks API
Usage: - Rapid prototyping - Sequential and Functional API - Transfer learning - Pre-trained models

Key Features: - User-friendly - Modular and composable - Backend-agnostic (TensorFlow, Theano)

1.94.4 1.4 ONNX Runtime

Version: 1.15+
Purpose: Cross-platform model inference
Usage: - Deploy models across frameworks - Optimize inference performance - Hardware acceleration (CPU, GPU, NPU)

Installation:

```
pip install onnxruntime==1.15.0 onnxruntime-gpu==1.15.0
```

1.95 2. Classical Machine Learning

1.95.1 2.1 Scikit-learn

Version: 1.3+
Purpose: Machine learning for Python
Usage: - Random Forest, SVM, Decision Trees - Feature selection and engineering - Model evaluation metrics - Preprocessing utilities

Installation:

```
pip install scikit-learn==1.3.0
```

Key Modules: - `sklearn.ensemble`: Ensemble methods - `sklearn.svm`: Support Vector Machines - `sklearn.tree`: Decision Trees - `sklearn.metrics`: Evaluation metrics - `sklearn.preprocessing`: Data preprocessing - `sklearn.model_selection`: Cross-validation

---

1.95.2 2.2 XGBoost

Version: 2.0+
Purpose: Gradient boosting framework
Usage: - High-performance boosting - Feature importance - Tree visualization

Installation:

```
pip install xgboost==2.0.0
```

1.95.3 2.3 LightGBM

Version: 4.0+
Purpose: Fast gradient boosting
Usage: - Large-scale datasets - Categorical feature support - Efficient memory usage

Installation:

```
pip install lightgbm==4.0.0
```

1.95.4 2.4 CatBoost

Version: 1.2+
Purpose: Gradient boosting with categorical features
Usage: - Native categorical handling - Robust to overfitting - GPU acceleration

Installation:

```
pip install catboost==1.2
```

1.96 3. Network and SDN Tools

1.96.1 3.1 ONOS (Open Network Operating System)

Version: 2.7+ (Trellis)
Purpose: SDN controller for production networks
Usage: - OpenFlow switch control - Network topology management - REST API for integration - Real-time flow statistics

Installation:

```
# Download and install
wget https://repo1.maven.org/maven2/org/onosproject/onos-releases/2.7.0/onos-2.7.0.tar.
tar xzf onos-2.7.0.tar.gz
cd onos-2.7.0
./bin/onos-service start
```

Key Features: - Carrier-grade SDN controller - High availability and scalability - Intent-based networking - Service applications

---

1.96.2 3.2 OpenDaylight

Version: Argon (latest)
Purpose: Modular SDN controller
Usage: - Multi-protocol southbound (OpenFlow, NETCONF, BGP) - Flexible northbound REST APIs - Programmable network services

Installation:

```
# Download and run
wget https://nexus.opendaylight.org/content/repositories/opendaylight.release/org/openc
unzip karaf-0.18.0.zip
cd karaf-0.18.0
./bin/karaf
```

1.96.3 3.3 Ryu

Version: 4.34+
Purpose: Python-based SDN controller
Usage: - Lightweight controller - Python API for app development - OpenFlow protocol support

Installation:

```
pip install ryu==4.34
```

Usage Example:

```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import set_ev_cls

class SimpleSwitch(app_manager.RyuApp):
    def __init__(self, *args, **kwargs):
        super(SimpleSwitch, self).__init__(*args, **kwargs)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        # Handle packet-in events
        pass
```

1.96.4 3.4 Mininet

Version: 2.3+
Purpose: Network emulator for SDN prototyping
Usage: - Virtual network creation - Topology design and testing - OpenFlow switch emulation - Custom topologies in Python

Installation:

```
sudo apt-get install mininet
# Or from source
git clone https://github.com/mininet/mininet
cd mininet
sudo util/install.sh -a
```

Usage Example:

```
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.cli import CLI

class CustomTopo(Topo):
    def build(self):
        # Add switches
        s1 = self.addSwitch('s1')
        # Add hosts
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        # Add links
        self.addLink(h1, s1)
        self.addLink(h2, s1)

topo = CustomTopo()
net = Mininet(topo=topo)
net.start()
```

```
CLI(net)
net.stop()
```

1.96.5 3.5 NS-3 (Network Simulator 3)

Version: 3.38+
Purpose: Discrete-event network simulator
Usage: - Large-scale network simulation - Protocol testing - Performance evaluation

Installation:

```
git clone https://gitlab.com/nsnam/ns-3-dev.git
cd ns-3-dev
./ns3 configure --enable-examples
./ns3 build
```

1.96.6 3.6 OMNET++

Version: 6.0+
Purpose: Component-based discrete event simulator
Usage: - Network modeling - Protocol simulation - Visualization

Installation: Download from https://omnetpp.org/

1.97 4. Data Processing and Analysis

1.97.1 4.1 Pandas

Version: 2.0+
Purpose: Data manipulation and analysis
Usage: - DataFrame operations - CSV/JSON parsing - Time-series analysis - Data cleaning

Installation:

```
pip install pandas==2.0.3
```

1.97.2 4.2 NumPy

Version: 1.25+
Purpose: Numerical computing
Usage: - Array operations - Linear algebra - Fourier transform - Random number generation

Installation:

```
pip install numpy==1.25.0
```

1.97.3 4.3 Apache Spark (PySpark)

Version: 3.4+
Purpose: Distributed data processing
Usage: - Large-scale data processing - Distributed ML (MLlib) - Stream processing

Installation:

```
pip install pyspark==3.4.0
```

1.97.4 4.4 Dask

Version: 2023.7+
Purpose: Parallel computing in Python
Usage: - Parallel pandas operations - Distributed arrays - Task scheduling

Installation:

```
pip install dask==2023.7.0 distributed==2023.7.0
```

1.97.5 4.5 Scapy

Version: 2.5+
Purpose: Packet manipulation and analysis
Usage: - PCAP file reading - Packet crafting - Network sniffing

Installation:

```
pip install scapy==2.5.0
```

Usage:

```
from scapy.all import rdpcap, IP, TCP

packets = rdpcap('traffic.pcap')
for pkt in packets:
    if IP in pkt and TCP in pkt:
        print(f"SRC: {pkt[IP].src}, DST: {pkt[IP].dst}, PORT: {pkt[TCP].dport}")
```

1.98 5. Explainable AI (XAI) Libraries

1.98.1 5.1 SHAP

Version: 0.42+
Purpose: SHapley Additive exPlanations
Usage: - Feature importance - Global and local explanations - Model-agnostic

Installation:

```
pip install shap==0.42.0
```

Usage:

```
import shap

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

1.98.2 5.2 LIME

Version: 0.2+
Purpose: Local Interpretable Model-agnostic Explanations
Usage: - Instance-level explanations - Any black-box model

Installation:

```
pip install lime==0.2.0.1
```

Usage:

```
from lime import lime_tabular

explainer = lime_tabular.LimeTabularExplainer(X_train, feature_names=features)
exp = explainer.explain_instance(X_test[0], model.predict_proba, num_features=10)
exp.show_in_notebook()
```

1.98.3 5.3 Captum

Version: 0.6+
Purpose: Model interpretability for PyTorch
Usage: - Attribution algorithms - Layer-wise relevance propagation - Integrated gradients

Installation:

```
pip install captum==0.6.0
```

1.99 6. Distributed Computing and Federated Learning

1.99.1 6.1 TensorFlow Federated

Version: 0.57+
Purpose: Federated learning framework
Usage: - Distributed model training - Privacy-preserving ML - FedAvg implementation

Installation:

```
pip install tensorflow-federated==0.57.0
```

1.99.2 6.2 PySyft

Version: 0.8+
Purpose: Privacy-preserving ML
Usage: - Federated learning - Differential privacy - Secure multi-party computation

Installation:

```
pip install syft==0.8.0
```

1.99.3 6.3 Flower (Fl ower)

Version: 1.5+
Purpose: Federated learning framework
Usage: - Framework-agnostic FL - Scalable federation - Research and production

Installation:

```
pip install flwr==1.5.0
```

1.100 7. Network Simulation and Testbed

1.100.1 7.1 tcpdump

Purpose: Packet capture
Installation:

```
sudo apt-get install tcpdump
```

1.100.2 7.2 Wireshark/tshark

Purpose: Network protocol analyzer
Installation:

```
sudo apt-get install wireshark tshark
```

1.100.3 7.3 CICFlowMeter

Purpose: Network traffic flow generator
Usage: Convert PCAP to CSV with flow features
Installation: Download from Canadian Institute for Cybersecurity

---

1.101 8. Containerization and Orchestration

1.101.1 8.1 Docker

Version: 24+
Purpose: Containerization platform
Usage: - Package IDS as containers - Reproducible environments - Easy deployment

Installation:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

1.101.2 8.2 Kubernetes

Version: 1.28+
Purpose: Container orchestration
Usage: - Distributed IDS deployment - Auto-scaling - Service discovery

Installation:

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

1.102 9. Monitoring and Visualization

1.102.1 9.1 Matplotlib

Version: 3.7+
Purpose: Static, animated, and interactive visualizations
Installation:

```
pip install matplotlib==3.7.2
```

1.102.2 9.2 Seaborn

Version: 0.12+
Purpose: Statistical data visualization
Installation:

```
pip install seaborn==0.12.2
```

1.102.3 9.3 Plotly

Version: 5.15+
Purpose: Interactive visualizations
Installation:

```
pip install plotly==5.15.0
```

1.102.4 9.4 Grafana

Version: 10.0+
Purpose: Metrics dashboards
Installation:

```
sudo apt-get install -y grafana
sudo systemctl start grafana-server
```

1.102.5 9.5 Prometheus

Version: 2.45+
Purpose: Time-series monitoring
Installation:

```
wget https://github.com/prometheus/prometheus/releases/download/v2.45.0/prometheus-2.45
tar xvfz prometheus-*.tar.gz
cd prometheus-*
./prometheus --config.file=prometheus.yml
```

1.102.6 9.6 ELK Stack (Elasticsearch, Logstash, Kibana)

Purpose: Log aggregation and analysis
Installation:

```
# Elasticsearch
wget https://artifacts.elastic.co/downloads/elasticsearch/elasticsearch-8.9.0-linux-x86
tar -xzf elasticsearch-8.9.0-linux-x86_64.tar.gz
cd elasticsearch-8.9.0/
./bin/elasticsearch

# Kibana
wget https://artifacts.elastic.co/downloads/kibana/kibana-8.9.0-linux-x86_64.tar.gz
tar -xzf kibana-8.9.0-linux-x86_64.tar.gz
cd kibana-8.9.0/
./bin/kibana
```

1.103 10. Development Tools

1.103.1 10.1 Jupyter Notebook/Lab

Version: Latest
Purpose: Interactive development environment
Installation:

```
pip install jupyter jupyterlab
```

1.103.2 10.2 VS Code

Version: Latest
Purpose: Code editor
Extensions: - Python - Jupyter - Docker - GitLens

1.103.3 10.3 PyCharm

Version: Professional/Community
Purpose: Python IDE

1.104 11. Version Control and Collaboration

1.104.1 11.1 Git

Version: 2.40+
Purpose: Version control
Installation:

```
sudo apt-get install git
```

1.104.2 11.2 GitHub

Purpose: Code hosting and collaboration
Features: - Repository hosting - Issue tracking - Pull requests - GitHub Actions (CI/CD)

1.104.3 11.3 DVC (Data Version Control)

Version: 3.0+
Purpose: Version control for data and models
Installation:

```
pip install dvc==3.0.0
```

1.105 Summary: Complete Environment Setup

1.105.1 Installation Script

```
#!/bin/bash
# Complete environment setup for AI-IDS development

# Update system
sudo apt-get update && sudo apt-get upgrade -y

# Python and pip
sudo apt-get install python3.10 python3-pip -y

# Deep Learning
pip install tensorflow==2.13.0 torch==2.0.1 onnxruntime==1.15.0

# Classical ML
pip install scikit-learn==1.3.0 xgboost==2.0.0 lightgbm==4.0.0

# Data Processing
pip install pandas==2.0.3 numpy==1.25.0 dask==2023.7.0

# XAI
pip install shap==0.42.0 lime==0.2.0.1

# Federated Learning
pip install tensorflow-federated==0.57.0 flwr==1.5.0

# Visualization
```

```
pip install matplotlib==3.7.2 seaborn==0.12.2 plotly==5.15.0

# Network Tools
pip install scapy==2.5.0 ryu==4.34

# Development
pip install jupyter jupyterlab ipython

# Docker
curl -fsSL https://get.docker.sh -o get-docker.sh
sudo sh get-docker.sh

echo "Environment setup complete!"
```

1.106 Conclusion

This comprehensive toolset provides everything needed for developing, testing, and deploying the AI-based IDS. The combination of deep learning frameworks, classical ML libraries, network simulation tools, XAI libraries, and monitoring solutions enables end-to-end research and development.

Key Stack Highlights: - **Deep Learning:** TensorFlow, PyTorch for model development - **SDN:** ONOS, OpenDaylight, Mininet for network control - **XAI:** SHAP, LIME for explainability - **Federated:** TensorFlow Federated, Flower for distributed learning - **Monitoring:** Grafana, Prometheus, ELK for system observability - **Deployment:** Docker, Kubernetes for production deployment

---

Next: Refer to `budget.md` for cost allocation and `collaboration.md` for partnership opportunities. # Budget Estimate ## Detailed Budget Breakdown for 4-Year PhD Program

Comprehensive Financial Planning for AI-IDS Research
Last Updated: December 22, 2025
Total Estimated Budget: $53,000 over 4 years

1.107 Table of Contents

7. Publications
8. Miscellaneous
9. Budget Timeline
10. Funding Sources
11. Justification

1.108 1. Budget Summary

Category	Amount	Percentage
Equipment & Hardware	$18,000	34%
Computational Resources (Cloud)	$15,000	28%
Conference Travel	$10,000	19%
Publications (Open Access)	$3,500	7%
Personnel (Undergrad Support)	$3,000	6%
Software Licenses	$2,500	5%
Miscellaneous	$1,000	2%
TOTAL	**$53,000**	**100%**

1.109 2. Personnel Costs

1.109.1 2.1 PhD Student Stipend

Amount: $0 (Covered by Research Assistantship)
Notes: - Typical PhD stipend: $25-35K/year - Covered by department/advisor funding - Not included in research budget

1.109.2 2.2 Undergraduate Research Assistant

Amount: $3,000
Breakdown: - Summer research support (10 weeks × $15/hour × 20 hours/week) = $3,000 - Help with data collection, testbed setup, documentation

Timeline: - Year 2 Summer: $1,500 (testbed setup) - Year 3 Summer: $1,500 (experiments and documentation)

Justification: Undergraduate assistance accelerates testbed deployment and documentation, freeing PhD student for advanced research tasks.

1.110 3. Equipment and Hardware

Total: $18,000

1.110.1 3.1 Network Testbed Equipment

Item	Quantity	Unit Price	Total	Justification
OpenFlow SDN Switches	5	$1,000	$5,000	Enable SDN experiments, OpenFlow protocol testing
High-Performance Servers	2	$4,000	$8,000	Fog nodes for federated learning aggregation
Raspberry Pi 4 (8GB)	10	$100	$1,000	Edge computing nodes for distributed IDS
IoT Devices (Mixed)	50	$50	$2,500	Smart sensors, cameras for IoT traffic generation
Network Cables & Accessories	Bulk	-	$500	CAT6 cables, patch panels, connectors
External Storage (Backup)	2	$500	$1,000	10TB NAS for dataset and model backup

Subtotal: $18,000

Purchase Timeline: - Year 1 Q4: External storage ($1,000) - Year 2 Q3: All testbed equipment ($17,000)

Justification: Physical testbed is essential for real-world validation of distributed IDS. Cannot be replaced by simulation alone. Equipment enables: - SDN attack scenarios (controller DoS, flow table overflow) - Edge-fog-cloud hierarchical deployment - IoT malware propagation studies - Latency and throughput measurements on real hardware

---

1.111 4. Computational Resources (Cloud)

Total: $15,000

1.111.1 4.1 Cloud GPU Instances

Provider	Instance Type	Usage	Cost
AWS EC2	p3.8xlarge (4× V100)	5,000 hours	$12,000
Google Cloud	n1-standard-8 (CPU backup)	2,000 hours	$1,500
Storage	S3/Cloud Storage (5TB)	4 years	$1,500

Usage Breakdown: - Year 1: 1,000 hours (model prototyping) - $2,400 - Year 2: 1,500 hours (distributed training) - $3,600 - Year 3: 2,000 hours (extensive experiments) - $4,800 - Year 4: 500 hours (final experiments) - $1,200

Justification: - Deep learning models (CNN-LSTM, Transformer) require GPU acceleration - Large-scale experiments need parallel processing - University GPU cluster has limited availability and queue times - Cloud provides on-demand access for deadline-driven research

Cost Optimization: - Use spot instances (60-70% discount) - Apply for AWS/Google Cloud research credits ($3-5K potential) - Batch experiments during off-peak hours

---

1.112 5. Software Licenses

Total: $2,500

Software	Annual Cost	Years	Total	Purpose
MATLAB (if needed)	$500	2	$1,000	Signal processing, network simulation
Network Analysis Tools	$750	2	$1,500	Commercial packet analysis, deep inspection
Total	-	-	**$2,500**	-

Notes: - Most tools are open-source (TensorFlow, PyTorch, Scikit-learn) - MATLAB may not be needed if Python alternatives suffice - University site licenses may cover some costs

---

1.113 6. Conference Travel

Total: $10,000

1.113.1 6.1 Conference Breakdown

Year	Conference	Location	Estimated Cost	Purpose
Year 1	IEEE Workshop	Domestic	$1,500	First paper presentation
Year 2	IEEE INFOCOM	International	$3,000	Major networking conference
Year 3	USENIX Security	US/International	$3,000	Top security venue
Year 3	Local Workshop	Domestic	$500	Additional exposure
Year 4	IEEE S&P	US	$2,000	Top-tier security conference

Cost Breakdown per Conference: - Airfare: $400-800 (domestic), $1,000-1,500 (international) - Hotel: $150-250/night × 4 nights = $600-1,000 - Registration: $500-800 (student rate) - Meals: $50/day × 5 days = $250 - Ground transportation: $100-200

Funding Sources: - Conference travel grants: $500-1,000/conference - Department travel funds: $500-1,000/year - Advisor research grants: $1,000-2,000/year - Out-of-pocket: Remainder

Justification: - Conference presentations establish research credibility - Networking with leading researchers in field - Feedback improves research quality - Required for career development

---

1.114 7. Publications

Total: $3,500

1.114.1 7.1 Open Access Fees

Publication Type	Cost per Paper	Number	Total
Conference Open Access	$400	2	$800
Journal Open Access	$1,500	2	$3,000
Total	-	**4**	**$3,800**

Breakdown: - IEEE Transactions (TIFS, TMC): $1,495-1,995 per paper - ACM journals: $1,500-2,000 per paper - Conference open access: $300-500 per paper - arXiv preprints: Free

Budget Allocation: - Year 2: $800 (1 conference open access) - Year 3: $1,500 (1 journal open access) - Year 4: $1,200 (1 journal, 1 conference)

Justification: - Open access increases citation count and impact - Required by some funding agencies (e.g., NSF, EU grants) - Broader dissemination to industry and practitioners

Cost Reduction: - Some journals offer waivers for students - Author's accepted manuscript on institutional repository (free) - Negotiate with publishers for reduced fees

---

1.115 8. Miscellaneous

Total: $1,000

Item	Amount	Purpose
Books and Online Resources	$300	Textbooks, O'Reilly subscriptions
Printing and Materials	$200	Poster printing, thesis binding
Professional Memberships	$200	IEEE, ACM student membership
Contingency Fund	$300	Unexpected expenses

1.116 9. Budget Timeline

1.116.1 Year 1: $4,000

- Cloud GPU: $2,400
- Storage (backup): $1,000
- Books & resources: $200
- Professional memberships: $100
- Conference travel: $300 (local workshop)

1.116.2 Year 2: $17,900

- **Major spending year (testbed equipment)**
- Testbed hardware: $17,000
- Cloud GPU: $3,600
- Undergrad support: $1,500
- Software licenses: $1,000
- Conference travel: $3,000

1.116.3 Year 3: $18,300

- Cloud GPU: $4,800
- Undergrad support: $1,500
- Conference travel: $3,500
- Publications (OA): $1,500
- Software licenses: $1,000

1.116.4 Year 4: $12,800

- Cloud GPU: $1,200
- Conference travel: $2,000
- Publications (OA): $1,200
- Thesis printing: $200
- Miscellaneous: $300

Cumulative Total: $53,000

1.117 10. Funding Sources

1.117.1 10.1 Primary Funding

University Research Assistantship - Covers stipend and tuition - $25-35K/year (living expenses) - Health insurance

1.117.2 10.2 Research Funding

Source	Amount	Likelihood	Timeline
Department Research Grants	$5-10K	High	Year 1-2
Advisor's Research Grants	$10-20K	High	Year 2-4
External Fellowships	$10-30K	Medium	Year 2-3
Cloud Provider Credits	$3-5K	High	Year 1-2
Conference Travel Grants	$2-4K	Medium	Year 2-4
Industry Partnerships	$5-15K	Medium	Year 3-4

1.117.3 10.3 Fellowship Opportunities

NSF Graduate Research Fellowship Program (GRFP) - Award: $37K/year stipend + $12K tuition - Apply: Year 1 or 2 - Likelihood: Competitive (15-20% acceptance)

Industry Fellowships - Google PhD Fellowship: $50K/year - Facebook Fellowship: $42K/year - Microsoft Research Fellowship: $28K/year - Cisco Research Fellowship: $25K/year

1.117.4 10.4 Cloud Research Credits

- **AWS Educate:** $100-200 credits
- **AWS Research Credits:** $5,000-10,000 (application required)
- **Google Cloud for Research:** $5,000-20,000
- **Microsoft Azure for Research:** $10,000-20,000

1.118 11. Justification

1.118.1 11.1 Equipment Justification

Why Physical Testbed? ($18,000) - Simulation cannot replicate real-world latency, packet loss, hardware constraints - IoT device behavior differs significantly in simulation vs. reality - Required for latency validation (<100ms target) - Edge device resource constraints (CPU, memory) only testable on real hardware - Publications expect real-world validation for acceptance

Why Multiple Devices? - 5 SDN switches: Create realistic network topology with multiple paths - 10 Edge nodes: Test scalability and federated learning with realistic number of participants - 50 IoT devices: Generate realistic heterogeneous traffic patterns

1.118.2 11.2 Computational Resources Justification

Why Cloud GPU? ($15,000) - Deep learning models require 100-1000× speedup with GPUs - University GPU cluster: - Long queue times (days to weeks) - Limited to 24-48 hour jobs - Shared resources impact reproducibility - Cloud provides: - On-demand access for deadlines - Latest GPU architectures (V100, A100) - Scalability for large experiments

Cost-Benefit Analysis: - Purchasing 4× V100 GPUs: $20,000-30,000 - Power, cooling, maintenance: $2,000/year - Cloud: Pay only for usage, no maintenance - Break-even: >2 years of continuous use

1.118.3 11.3 Travel Justification

Why 4 Conferences? ($10,000) - Academic career requires conference presentations - Networking critical for: - Postdoc/faculty positions - Industry collaborations - Staying current with field - Student rate travel grants reduce out-of-pocket costs

1.118.4 11.4 Alternative Budget Scenarios

Minimum Budget ($35,000): - Reduce testbed ($10K): 3 switches, 5 edge nodes, 20 IoT devices - Cloud ($10K): Use only spot instances, apply for all credits - Travel ($8K): 3 conferences instead of 4 - Publications ($2K): Only essential open access

Maximum Budget ($70,000): - Enhanced testbed ($30K): 10 switches, 20 edge nodes, 100 IoT devices - Cloud ($25K): More GPU hours for extensive hyperparameter tuning - Travel ($15K): 5-6 conferences, international workshops

---

1.119 Conclusion

The $53,000 budget is realistic and justified for a 4-year PhD program in AI-based network security. The major expenses (testbed equipment and cloud GPU) are essential for validating the proposed distributed IDS framework in real-world scenarios. Multiple funding sources (department grants, fellowships, cloud credits, industry partnerships) will be pursued to cover the budget.

Budget Highlights: - ✅ **Justified:** Every expense tied to research objectives - ✅ **Realistic:** Based on current market prices - ✅ **Flexible:** Minimum and maximum scenarios provided - ✅ **Fundable:** Multiple funding sources identified

Cost Optimization Strategies: - Apply for cloud research credits (potential $10K savings) - Seek industry equipment donations - Use conference travel grants - Apply for competitive fellowships (NSF GRFP, industry)

Return on Investment: - **Academic:** 5-10 publications, PhD degree - **Practical:** Open-source framework benefiting community - **Career:** Strong foundation for post-doc/faculty/industry positions - **Impact:** Improved security for billions of IoT devices and 5G networks

---

Next: Refer to `collaboration.md` for potential funding and equipment partnerships, and `tools_and_frameworks.md` for software requirements. # Collaboration Opportunities ## Potential Partners for Research, Funding, and Deployment

Academic, Industry, and Community Partnerships
Last Updated: December 22, 2025

1.120 Table of Contents

1.121 1. Overview

Successful development and deployment of AI-based IDS for next-generation networks requires collaboration across academia, industry, standards organizations, and open-source communities. This document identifies potential partners and outlines mutually beneficial collaboration opportunities.

1.121.1 1.1 Collaboration Objectives

- **Data Access:** Obtain real-world network traffic datasets
- **Equipment Donation:** Secure testbed hardware (SDN switches, IoT devices)
- **Financial Support:** Research funding, fellowships, cloud credits
- **Validation:** Deploy IDS in production networks for real-world testing
- **Knowledge Exchange:** Access to domain experts, joint publications
- **Technology Transfer:** Industry adoption of research outcomes

---

1.122 2. Academic Institutions

1.122.1 2.1 Leading Universities in Network Security

Carnegie Mellon University (CMU) - **Department:** CyLab Security and Privacy Institute - **Expertise:** Network security, ML for security - **Potential Collaboration:** - Joint research projects - Student exchange programs - Dataset sharing - Joint publications - **Contact:** Faculty in Computer Science Department

University of California, Berkeley - **Department:** International Computer Science Institute (ICSI) - **Expertise:** Network measurement, security, machine learning - **Potential Collaboration:** - Access to network traces - Collaboration with NetSys Lab - Joint workshops - **Contact:** Prof. Vern Paxson (security expert)

MIT - **Department:** Computer Science and Artificial Intelligence Laboratory (CSAIL) - **Expertise:** AI, cybersecurity, networks - **Potential Collaboration:** - AI/ML methodology guidance - Research seminars and visits - Joint publications

Georgia Tech - **Department:** Institute for Information Security & Privacy - **Expertise:** Network security, SDN, IoT security - **Potential Collaboration:** - SDN testbed collaboration - Joint experiments - Faculty mentorship

University of Illinois Urbana-Champaign - **Department:** Coordinated Science Laboratory - **Expertise:** Network security, distributed systems - **Potential Collaboration:** - Federated learning research - Distributed IDS deployment

1.122.2 2.2 International Collaborations

UNSW Sydney (Australia) - **Expertise:** IDS datasets (UNSW-NB15), security research - **Collaboration:** Dataset usage, joint evaluation

Technical University of Munich (Germany) - **Expertise:** 5G/6G security, network slicing - **Collaboration:** 5G testbed experiments, joint papers

National University of Singapore (NUS) - **Expertise:** AI, IoT security - **Collaboration:** IoT malware research, federated learning

---

1.123 3. Industry Partners

1.123.1 3.1 Telecommunications Operators

Verizon - **Interest:** 5G network security, edge computing - **Potential Collaboration:** - Access to anonymized 5G traffic data - Testbed deployment in real 5G network - Joint research on 5G-specific attacks - **Benefits to Verizon:** - Advanced threat detection for 5G core - Protection of network slicing infrastructure - Reduced operational security costs - **Contact:** Verizon Innovation Program, CTO Office

AT&T - **Interest:** Network security, SDN/NFV protection - **Potential Collaboration:** - AT&T Research collaboration - Dataset provision - Proof-of-concept deployment - **Benefits to AT&T:** - Enhanced security for virtualized networks - AI-driven automation for threat response - Competitive advantage in secure 5G - **Contact:** AT&T Labs Research

Deutsche Telekom - **Interest:** European 5G security, GDPR compliance - **Potential Collaboration:** - Privacy-preserving IDS validation - Federated learning deployment across regions - **Benefits:** - GDPR-compliant threat detection - Cross-border threat intelligence - **Contact:** Deutsche Telekom Innovation Labs

T-Mobile - **Interest:** IoT connectivity, mobile network security - **Potential Collaboration:** - IoT traffic analysis - Edge-based IDS for massive IoT - **Benefits:** - Protection for IoT platform - Scalable security for millions of devices

1.123.2 3.2 Network Security Vendors

Cisco Systems - **Interest:** Next-gen firewalls, SD-WAN security, Cisco Umbrella - **Potential Collaboration:** - Equipment donation (switches, routers) - Joint development of AI-powered Cisco IDS - Cisco Research grant - Integration with Cisco Security products - **Benefits to Cisco:** - AI/ML enhancements to product portfolio - Academic validation of technology - Early access to cutting-edge research - **Contact:** Cisco Research, DevNet

Palo Alto Networks - **Interest:** ML-powered threat prevention, cloud security - **Potential Collaboration:** - Integration with Cortex XDR - Joint research on adversarial ML - Funding for explainable AI research - **Benefits:** - Enhanced AI capabilities in products - Explainability for regulatory compliance - **Contact:** Palo Alto Networks Research

Fortinet - **Interest:** SD-WAN security, FortiGuard Threat Intelligence - **Potential Collaboration:** - Threat intelligence data sharing - Joint threat hunting research -

Equipment provision (FortiGate devices) - **Benefits:** - AI-enhanced threat detection - Improved threat intelligence accuracy

Darktrace - **Interest:** AI-driven autonomous response, self-learning IDS - **Potential Collaboration:** - Reinforcement learning for threat response - Benchmark comparisons - Sponsored research - **Benefits:** - Academic validation of AI approach - Enhancements to self-learning algorithms

1.123.3 3.3 Cloud and Technology Giants

Amazon Web Services (AWS) - **Interest:** Cloud security, AWS Shield, GuardDuty - **Potential Collaboration:** - AWS Research Credits ($10-20K) - AWS Security Heroes program - Integration with AWS security services - **Benefits to AWS:** - Enhanced threat detection for VPC - AI models for GuardDuty - **Contact:** AWS Research, AWS Security team

Google Cloud - **Interest:** GCP security, Chronicle Security - **Potential Collaboration:** - Google Cloud Research Credits - Joint research with Chronicle team - TensorFlow optimization for IDS - **Benefits:** - Improved security for Google Cloud customers - TensorFlow use case showcase

Microsoft - **Interest:** Azure Security, Microsoft Defender - **Potential Collaboration:** - Microsoft Research Fellowship - Azure credits for research - Integration with Azure Sentinel - **Benefits:** - AI/ML enhancements to Defender - Azure security product improvement - **Contact:** Microsoft Research, Azure Security team

IBM - **Interest:** QRadar, Watson for Cyber Security - **Potential Collaboration:** - IBM Research collaboration - Quantum-safe security research - Funding for AI security research - **Benefits:** - Watson AI enhancements - Integration with QRadar SIEM

1.123.4 3.4 IoT and Edge Computing

NVIDIA - **Interest:** Jetson edge AI, security for autonomous systems - **Potential Collaboration:** - Jetson device donation for edge IDS - NVIDIA GPU Cloud credits - Joint optimization for embedded inference - **Benefits:** - Showcase for Jetson AI capabilities - Security for NVIDIA edge platforms - **Contact:** NVIDIA Research, Jetson Developer Program

Arm - **Interest:** IoT security, TrustZone, secure edge computing - **Potential Collaboration:** - Arm Research grant - Optimization for Arm Cortex-M/Cortex-A - Integration with Arm Mbed OS security - **Benefits:** - AI security for Arm-based IoT devices - Efficient inference on Arm processors

Qualcomm - **Interest:** 5G chipsets, edge AI acceleration - **Potential Collaboration:** - Access to 5G test equipment - On-device AI optimization - **Benefits:** - Security for Qualcomm 5G modems - AI acceleration for Snapdragon

---

1.124 4. Research Laboratories

1.124.1 4.1 Industrial Research Labs

Bell Labs (Nokia) - **Expertise:** 5G/6G, network security, AI - **Potential Collaboration:** - Joint research on 6G security - Access to experimental 5G networks - Visiting researcher program - **Benefits:** - Early insights into 6G security challenges - Real-world 5G deployment validation

IBM Research - **Expertise:** AI, quantum computing, security - **Potential Collaboration:** - IBM Research Fellowship - Quantum-resistant IDS research - Joint publications - **Benefits:** - AI/quantum expertise - High-impact research collaboration

Microsoft Research - **Expertise:** ML, systems, security - **Potential Collaboration:** - Research internship - Joint projects on adversarial ML - Access to large-scale datasets - **Benefits:** - Leading-edge AI research - Industry perspective on deployability

Google Research - **Expertise:** Deep learning, federated learning, privacy - **Potential Collaboration:** - Federated learning research - Privacy-preserving ML - Intern/visiting scholar programs - **Benefits:** - Expertise in FL and differential privacy - TensorFlow optimization guidance

1.124.2 4.2 Government and National Labs

NSF (National Science Foundation) - **Programs:** CNS (Computer and Network Systems), SaTC (Secure and Trustworthy Cyberspace) - **Potential Collaboration:** - NSF Graduate Research Fellowship (GRFP) - Research grants (CAREER, SaTC) - **Benefits:** - Significant research funding ($500K-1M for CAREER)

DARPA (Defense Advanced Research Projects Agency) - **Programs:** AI Next, Assured Autonomy - **Potential Collaboration:** - DARPA Young Faculty Award - Collaborative research programs - **Benefits:** - High-risk, high-reward research funding - National security impact

NIST (National Institute of Standards and Technology) - **Interest:** Cybersecurity standards, AI standards - **Potential Collaboration:** - AI Risk Management Framework input - Evaluation of IDS against NIST standards - **Benefits:** - Standards alignment - Government credibility

---

1.125 5. Standardization Bodies

1.125.1 5.1 IETF (Internet Engineering Task Force)

- **Working Groups:** DOTS (DDoS Open Threat Signaling), I2NSF (Interface to Network Security Functions)
- **Potential Collaboration:**
  - Propose draft RFC for AI-based IDS interfaces
  - Participate in working groups
  - Standardize federated threat intelligence sharing

- **Benefits:**
  - Industry adoption through standardization
  - Shape future IDS protocols

1.125.2 5.2 3GPP (3rd Generation Partnership Project)

- **Focus:** 5G/6G security specifications
- **Potential Collaboration:**
  - Contribute to 5G security standards (TS 33.xxx series)
  - Propose AI-based security functions for 6G
- **Benefits:**
  - Influence 5G/6G security architecture
  - Early access to specifications

1.125.3 5.3 ETSI (European Telecommunications Standards Institute)

- **Groups:** NFV Security, Cyber Security Technical Committee
- **Potential Collaboration:**
  - NFV security standardization
  - AI in cybersecurity best practices
- **Benefits:**
  - European adoption
  - Industry recognition

1.125.4 5.4 IEEE

- **Standards:** IEEE 802.1X (Network Access Control), IEEE 1609 (WAVE Security)
- **Potential Collaboration:**
  - Participate in standards committees
  - Submit proposals for AI-enhanced security
- **Benefits:**
  - Academic and industry visibility
  - Standards-compliant research

1.126 6. Open-Source Communities

1.126.1 6.1 ONOS Community

- **Project:** Open Network Operating System (SDN controller)
- **Potential Collaboration:**
  - Contribute AI-IDS app to ONOS
  - Present at ONOS Build events
  - Collaborate with SDN developers
- **Benefits:**
  - Community adoption

  - Real-world deployments
- **Repository:** https://github.com/opennetworkinglab/onos

1.126.2 6.2 OpenDaylight

- **Project:** Modular SDN controller
- **Potential Collaboration:**
  - Develop security service module
  - Join ODL community calls
- **Benefits:**
  - Alternative SDN platform
  - Diverse user base

1.126.3 6.3 TensorFlow/PyTorch Communities

- **Potential Collaboration:**
  - Contribute IDS models to TensorFlow Hub / PyTorch Hub
  - Write tutorials for network security ML
  - Participate in Google Summer of Code
- **Benefits:**
  - Showcase work to AI/ML community
  - Broader impact beyond security

1.126.4 6.4 Zeek (formerly Bro)

- **Project:** Open-source network security monitoring
- **Potential Collaboration:**
  - Integrate AI models with Zeek
  - Develop Zeek plugin for ML-based detection
- **Benefits:**
  - Large user base (enterprises, ISPs)
  - Established IDS platform

1.127 7. Data Sharing Agreements

1.127.1 7.1 Datasets Needed

- **5G Core Network Traffic:** Signaling messages, user plane data
- **IoT Botnet Traffic:** Real-world IoT malware propagation
- **Enterprise Network Logs:** Anonymized intrusion attempts
- **Industrial Control Systems (ICS):** SCADA/ICS attack traces

1.127.2 7.2 Data Sharing Partners

Telecom Operators: - Provide anonymized 5G traffic under NDA - Joint data analysis projects

CERTs (Computer Emergency Response Teams): - Access to incident response data - Collaborative threat intelligence

Canadian Institute for Cybersecurity (CIC): - CICIDS datasets provider - Collaborate on new dataset creation

1.127.3 7.3 Privacy and Legal Considerations

- **Anonymization:** Remove PII before sharing
- **NDA:** Non-disclosure agreements with industry partners
- **IRB Approval:** Obtain approval for human subjects data
- **GDPR Compliance:** Follow EU data protection regulations

1.128 8. Collaboration Benefits

1.128.1 8.1 Benefits to Academic Research

- **Data Access:** Real-world datasets for validation
- **Equipment:** Hardware for testbed (saves $10-20K)
- **Funding:** Sponsored research, fellowships
- **Deployment:** Real-world validation in production networks
- **Expertise:** Domain knowledge from practitioners
- **Career:** Industry connections for postdoc/employment

1.128.2 8.2 Benefits to Industry Partners

- **Innovation:** Early access to cutting-edge research
- **Talent Pipeline:** Recruit PhD students/postdocs
- **Product Enhancement:** Integrate research into products
- **Credibility:** Academic validation of technology
- **Publications:** Co-authored papers boost reputation
- **IP:** Potential patents from joint research

1.128.3 8.3 Benefits to Community

- **Open-Source:** Free, validated IDS for researchers and SMEs
- **Knowledge:** Publications, tutorials, documentation
- **Tools:** Reusable code, models, datasets
- **Standards:** Influence on future security standards

1.129 9. Engagement Strategy

1.129.1 9.1 Timeline

Year 1: - Attend conferences, network with potential collaborators - Apply for cloud credits (AWS, Google, Microsoft) - Join open-source communities (ONOS, Zeek)

Year 2: - Reach out to industry partners for equipment donation - Apply for industry fellowships (Google, Microsoft, Cisco) - Propose collaborations with telecom operators

Year 3: - Deploy pilot IDS with industry partner - Engage with standards bodies (IETF, 3GPP) - Publish open-source framework

Year 4: - Finalize industry collaborations - Transfer technology to partners - Plan postdoc/industry position

1.129.2 9.2 Communication Channels

- **Email:** Direct outreach to researchers, product managers
- **LinkedIn:** Professional networking
- **Conferences:** Face-to-face meetings at IEEE, USENIX, ACM events
- **Workshops:** Present work at industry workshops
- **GitHub:** Open-source contributions
- **Blog/Medium:** Write about research for broader audience

1.129.3 9.3 Proposal Template

Email Template for Industry Collaboration:

```
Subject: PhD Research Collaboration: AI-Based IDS for 5G/IoT Networks

Dear [Name],

I am a PhD student at [University] working on AI-based intrusion detection
for next-generation networks (5G/6G, IoT, SDN). My research aims to develop
a distributed, privacy-preserving IDS using deep learning and federated learning.

I believe [Company] would be an excellent partner for this research. Potential
collaboration areas include:
- [Specific area relevant to company]
- [Data sharing / Equipment / Funding]
- [Joint publications]

The benefits to [Company] would be:
- [Benefit 1: e.g., Enhanced security for products]
- [Benefit 2: e.g., Academic validation]
- [Benefit 3: e.g., Early access to research]

I would appreciate the opportunity to discuss this further. Are you available
```

```
for a 15-minute call next week?

Best regards,
[Your Name]
PhD Candidate, [University]
[Email] | [Website] | [LinkedIn]
```

1.130 Conclusion

Successful PhD research requires strategic collaborations across academia, industry, standards bodies, and open-source communities. This document outlines a comprehensive engagement strategy to secure:

- **Data:** Real-world network traffic from telecom operators and enterprises
- **Equipment:** $10-20K in donated hardware for testbed
- **Funding:** Fellowships, cloud credits, sponsored research
- **Validation:** Real-world deployment in production networks
- **Impact:** Industry adoption and standards influence

Key Partnerships to Pursue: 1. **Academic:** CMU, Berkeley, MIT for joint research 2. **Industry:** Cisco, Palo Alto Networks for equipment and funding 3. **Telecom:** Verizon, AT&T for data and deployment 4. **Cloud:** AWS, Google, Microsoft for computational resources 5. **Standards:** IETF, 3GPP for standardization 6. **Open-Source:** ONOS, Zeek for community adoption

Next Steps: - Year 1: Network at conferences, apply for cloud credits - Year 2: Reach out to industry for testbed equipment - Year 3: Deploy pilot IDS with partner, engage with standards - Year 4: Finalize collaborations, plan career next steps

---

Contact: idriss5214@github.com
Repository: github.com/idriss5214/AI-IDS-NextGen-Networks-PhD-Proposal

For more information, refer to: - `../proposal/full_proposal.md` for research details - `budget.md` for financial planning - `tools_and_frameworks.md` for technical requirements